

BACKEND DEVELOPER MANUAL

Wealth OS

Personal Wealth Intelligence Platform for Indian Markets

Java 25

Spring Boot 3.5.3

PostgreSQL 16

Redis 7

Generated	15 September 2026
Automated tests	474 passing
Deprecation warnings	0
Frontend type-check	clean

*"Do not fabricate, estimate, duplicate, silently drop, or overwrite financial records.
Every financial number must be traceable. Every recommendation must have a reason.
Every import must be reconcilable. Every decision must use verified data."*

Contents

01	Orientation	5
	Backend Onboarding	6
	What Wealth OS Is	9
	Business Logic Rules	11
02	Architecture — as built	15
	Architecture Overview	16
	Architecture Decisions (ADRs)	20
	Database Architecture	24
	Data Flow & Integrations	34
03	Architecture — target	40
	Target Architecture: How To Read It	41
	Target System Architecture	43
	Target Data Architecture	59
	The Financial Ledger	70
04	Backend internals	78
	Backend Structure	79
	Financial Engine	83
	Gmail Ingestion Pipeline	86
	Jobs & Schedulers	95
	API Specification	99
05	End-to-end flows	110
	Gmail to Portfolio	111
	Transaction Processing	114
	Card & AI Recommendations	119
06	Operations	127
	DevOps Onboarding	128
	Environment Variables	130
	Deployment	135
	Monitoring & Logging	138
07	Why it is built this way	140

Research: How To Read It	141
Product Gap Matrix	143
Technical Gap Matrix	150

Assembled directly from the documentation in the repository, so it cannot drift from it. Every chapter names the file it came from, and that path is a live link. Read **The Financial Ledger** before changing anything that touches money. It documents the single-writer invariant the rest of the system depends on, and the seven properties that must remain true.

01

Orientation

Start here: what the product is, who it's for, and the rules every change must respect.

01 Backend Onboarding

02 What Wealth OS Is

03 Business Logic Rules

Backend Onboarding

 Source file · docs/02-backend/BACKEND_ONBOARDING.md

Prerequisites (verified versions)

TOOL	VERSION	SOURCE
JDK	25	<code>pom.xml</code> : <code>java.version</code> / <code>maven.compiler.release=25</code>
Maven	3.6+	standard
PostgreSQL	16	<code>docker-compose.yml</code>
Redis	7	<code>docker-compose.yml</code> (optional in dev — <code>spring.cache.type=simple</code> doesn't require it)
Ollama	any, with <code>qwen2.5:7b</code> pulled	<code>app.llm.provider=ollama</code> default

Known discrepancy: `backend/Dockerfile` builds/runs on `eclipse-temurin:21-*--alpine`, which does not match `pom.xml`'s Java 25 requirement. Verify which is authoritative before relying on the Docker build — see `04-devops/DEPLOYMENT_GUIDE.md`.

Quick start

1. DATABASE (VIA DOCKER COMPOSE, FROM REPO ROOT)

```
docker compose up -d postgres redis
```

Postgres initializes from `./database/schema.sql` then `./database/seed.sql` on first boot (mounted as `docker-entrypoint-initdb.d`).

2. ENVIRONMENT

Copy `.env.example` to `.env` — **note it is incomplete** relative to what `application.yml` actually reads (see `04-devops/ENVIRONMENT_VARIABLES.md` for the full, corrected table). At minimum for local dev you additionally need: `MAIL_CLIENT_ID`, `MAIL_CLIENT_SECRET` (if testing Gmail sync), and nothing else strictly required — `JWT_SECRET` auto-generates and persists to a gitignored `gmail.env` if left unset.

3. RUN THE BACKEND

```
cd backend
mvn spring-boot:run
```

Default profile is `dev` (`SPRING_PROFILES_ACTIVE` unset → `dev`). Server starts on `:8080` .

4. VERIFY

```
curl http://localhost:8080/actuator/health
```

API docs (Swagger UI): `http://localhost:8080/swagger-ui.html` .

5. REGENERATE THE TLS TRUSTSTORE (ONLY IF YOU SEE PKIX/SSL ERRORS ON OUTBOUND CALLS)

If your network intercepts TLS (corporate proxy), run:

```
backend/tools/regen-truststore.sh
```

This rebuilds `backend/truststore.p12` (JDK cacerts + your machine's trusted corporate roots). Off a corporate network, this file/setup is not needed — the `-Djavax.net.ssl.*` JVM args can be dropped from `pom.xml`'s `spring-boot-maven-plugin` config.

Running tests

```
mvn test
```

Test dependencies: `spring-boot-starter-test` , `spring-security-test` , `h2database` (in-memory, test scope only — production always runs on PostgreSQL).

Local AI (Ollama) setup

```
ollama pull qwen2.5:7b
ollama serve # if not already running as a service
```

Set `LLM_PROVIDER=none` to disable AI-dependent paths entirely (deterministic parsers keep working) if you don't want to run Ollama locally.

First things to read, in order

1. `BACKEND_STRUCTURE.md` — the layering pattern and per-domain package map.

2. `API_SPECIFICATION.md` — every endpoint, to know what's already there before adding a new one.
3. `FINANCIAL_ENGINE.md` — the calculation rules you must not violate (ledger-replay, net worth, XIRR, signal scoring).
4. `GMAIL_INGESTION_PIPELINE.md` — if you're touching anything email-related, including adding a new bank/broker parser (step-by-step guide included).

What Wealth OS Is

 [Source file · docs/00-pmo-product/PRODUCT_OVERVIEW.md](#)

Vision

A single, trustworthy source of truth for a user's entire financial life in the Indian market context — equities, mutual funds, fixed-income (FD/RD/EPF), cash, credit cards, and loans — built from transactions the system actually observes (via Gmail) rather than manual data entry, with an advisory layer that tells the user what to do next and why.

Problem solved

Indian retail investors' financial data is fragmented across broker apps, bank apps, AMC portals, and credit card statements, none of which talk to each other. Consolidation today typically means manual spreadsheets or trusting a black-box aggregator. This product instead:

1. Ingests the email trail every bank/broker/AMC already sends (`[IMPLEMENTED]` — 18 institution-specific parsers plus a generic bank-alert catch-all, see `02-backend/GMAIL_INGESTION_PIPELINE.md`).
2. Derives holdings and net worth from a replayable transaction ledger rather than accepting a hand-entered number (`[IMPLEMENTED]` — `PortfolioService.recomputeFromLedger`).
3. Refuses to guess: every ambiguous or low-confidence import goes to a human review queue instead of being silently booked (`[IMPLEMENTED]` — `EmailReviewItem / ReviewStatus`).
4. Surfaces a daily, risk-adjusted action list (buy/sell/hold with rationale) instead of a wall of raw numbers (`[IMPLEMENTED]` — Today's Investment Actions, `TodayActionsService`).

Core features (at a glance — see `FEATURE_MATRIX.md` for the authoritative status of each)

FEATURE	ONE-LINE DESCRIPTION
Portfolio & Net Worth	Stocks, MF, FD/RD, EPF, cash, loans consolidated into one number via <code>PortfolioContextService</code>
Gmail Ingestion	OAuth-connected inbox scan → parse → dedup → ledger commit, with push (real-time) and scheduled (30-min safety net) triggers
AI Email Classification	Local LLM (Ollama, <code>qwen2.5:7b</code> default) fallback for emails no deterministic parser recognizes, gated at 85% confidence
Human Review Queue	Anything below the confidence bar, or of a type that must never be auto-booked (e.g. internal transfers), is queued for Accept/Edit/Reject

FEATURE	ONE-LINE DESCRIPTION
Signal Engine	Multi-factor (trend/momentum/structure/volume/volatility) BUY/SELL/HOLD scoring with ATR-based stop/target, confidence capped by data availability
Card Optimizer	Net-value verdict (Keep/Downgrade/Cancel/Underused) per credit card against the user's own 90-day spend
Today's Investment Actions	Daily buy/sell/hold/watch queue with execute/snooze/note actions
FD/RD Lifecycle	Automatic renewal detection (old FD → new FD linking) so a renewed deposit is never double-counted
MF Redemption & Tax	STCG/LTCG tax computation per redemption, with a staged reinvestment ("deployment plan") tracker
Financial Planning	Goals, reminders, tax summary, recurring investments (SIP/PPF/NPS)
Privacy Masking	One global toggle hides every rupee figure app-wide (<code>useMaskedText</code> / <code>Amount</code>)

Who this is for

A single authenticated user managing their own finances (multi-tenant by `User` FK throughout, but no team/shared-account concept exists in the schema). The product is not a robo-advisor — it never places trades; every recommendation requires the user to act on it themselves (or, at most, mark it executed after the fact).

Business Logic Rules

 [Source file · docs/00-pmo-product/BUSINESS_LOGIC_RULES.md](#)

Every formula below is quoted from the actual implementation, not paraphrased business intent. File references point to `backend/src/main/java/com/marketai/`.

Net Worth

`PortfolioContextService.build()`

```
stocksValue, mfValue = sum of Holding.currentValue, split by symbol suffix ".MF"
equityInvested/Current = sum across ALL holdings (stocks + MF)
fd    = firstNonNull(totalFdCurrentValue, totalFdPrincipal)
rd    = totalRdCurrentValue
epf   = totalEpf
other = totalOtherAssets
loans = totalLoanOutstanding
cash  = sum of CashAccount.balance

totalAssets = stocksValue + mfValue + fd + rd + epf + other + cash
netWorth    = totalAssets - loans

equityPercent = (stocksValue + mfValue) / totalAssets × 100
debtPercent   = (fd + rd + epf) / totalAssets × 100
otherPercent  = other / totalAssets × 100      (cash is NOT its own bucket – it's inside
totalAssets only)

equityPnl      = equityCurrent - equityInvested
equityPnlPercent = equityPnl / equityInvested × 100 (null if equityInvested ≤ 0)
```

Edge case: `pct(part, whole)` returns `null` (never `0`) when the denominator is ≤ 0 — an undefined percentage is never displayed as "0%".

Concentration flags (thresholds are hardcoded constants): | Flag | Trigger | Severity | |---|---|---| |

`SINGLE_STOCK` | one stock > 10% of total assets **or** > 25% of the equity book | HIGH | | `SECTOR` | one sector > 30% of *known-sector* equity | HIGH | | `ASSET_ALLOCATION` | equity > 85% of total assets | MODERATE | | `LEVERAGE` | loans > 50% of total assets | MODERATE |

Sector concentration is only trusted if $\geq 80\%$ of equity value has a known sector — below that, a data-gap note is appended instead of silently treating unknown-sector value as zero-risk.

Holdings (ledger replay)

`PortfolioService.recomputeFromLedger(Holding h)` — the *only* place `quantity` / `averageCost` is ever set. Replays `Transaction` rows ordered by date ascending:

```

BUY: totalCost += price*qty ; netQty += qty
SELL: sellQty = min(t.qty, netQty)
      if netQty > 0: avgCostAtSale = totalCost / netQty (scale 4)
                    totalCost -= avgCostAtSale * sellQty
                    netQty -= sellQty (floored at 0)

if netQty ≤ 0 → holding deleted, method returns null (fully sold)
else          → quantity = netQty, averageCost = totalCost / netQty (scale 2)

```

This is weighted-average-cost replay, not FIFO/LIFO lot tracking. Each sale reduces the pooled cost basis proportionally, not by matching a specific buy lot. [Known limitation – FIFO/LIFO cost basis is [PLANNED / NOT IMPLEMENTED] .]

XIRR

XirrCalculator — Newton-Raphson on $\sum \text{cashflow}_i / (1+r)^{(\text{days}_i/365)} = 0$. - Requires ≥ 2 cash flows with at least one positive and one negative sign, else `null` . - `MAX_ITERATIONS=100` , `PRECISION=1e-7` , `INITIAL_GUESS=0.1` (10%), rate floored at `-0.999999` . - Cash flows for a holding: every BUY (negated) and SELL (positive) transaction, plus a synthetic "sold today at current value" flow. Falls back to the stored `Holding.xirr` field only if the ledger is empty or Newton's method fails to converge.

FD/RD Renewal Detection

TrackingService — shared tolerances: `RENEWAL_DATE_WINDOW_DAYS = 10` , `RENEWAL_AMOUNT_TOLERANCE = 0.05` (5%, chosen to cover standard TDS deduction plus rounding without false-matching an unrelated similarly-sized new FD).

```

Candidate FD: same user, same bank (case-insensitive), status in {ACTIVE, MATURED}
Gate 1: |candidate.maturityDate - newFd.startDate| ≤ 10 days
Gate 2: |newFd.principal - candidate.projectedMaturityValue| / projectedMaturityValue ≤ 0.05
Pick:   among all candidates clearing both gates, the smallest date difference

```

On a match: old deposit → `status=MATURED_RENEWED` , `renewedToId` set (excluded from net worth from then on); new deposit → `renewedFromId` set. **No link is ever invented if nothing clears both thresholds** — the FD is simply left unlinked, never guessed.

RD renewal compares the candidate's **projected corpus** against the new RD's **total committed value** (`monthlyAmount × tenureMonths`), not its first installment — "since an RD's principal is the whole schedule, not one payment."

FD maturity/current value — compound interest $P \times (1 + r/n)^{(n \times t)}$, `n` = 12/4/1 for monthly/quarterly/annually (quarterly is the default when unspecified). Falls back to simple 1-year interest if either date is null.

MF Redemption Tax

RedemptionService — `STCG_RATE=0.15` , `LTCG_RATE=0.125` , `LTCG_EXEMPTION=₹1,25,000` .

```

redeemedAmount    = nav × unitsRedeemed
investedPortion    = holding.averageCost × unitsRedeemed
gain               = redeemedAmount - investedPortion
isLongTerm         = daysHeld ≥ 365 → LTCG, else STCG

if gain > 0:
    LTCG tax = max(gain - 125000, 0) × 0.125
    STCG tax = gain × 0.15

```

Known limitation: the ₹1.25L LTCG exemption is applied *per redemption*, not tracked cumulatively across the financial year. [PARTIALLY IMPLEMENTED – cumulative-year exemption tracking is [PLANNED / NOT IMPLEMENTED]].

Deployment plan for redeemed-but-not-reinvested cash: a rule-based 20% (invest now) / 30% (after a correction, trigger = Nifty ATR-derived pullback level) / 50% (SIP over 6 months) split — explicitly documented as **not** live AI market monitoring.

Signal Engine Scoring

SignalEngine scores by *family*, not by individual indicator — deliberately, because correlated oscillators (RSI/CCI/Stochastic/ROC/MACD-line, pairwise correlation >0.9) would otherwise silently over-weight momentum ~4:1 under naive per-indicator averaging.

```

Weights: trend=0.25, momentum=0.25, structure=0.25, volume=0.15, volatility=0.10 (sum=1.00)

composite = round(Σ(score_i × weight_i) / Σ(weight_i available))    clamped [-100,100]
ceiling    = round(weightAvailable / totalWeight × 100)
confidence = min(|composite|, ceiling)    ← can never exceed how much data was actually available

BUY if composite ≥ 20, SELL if ≤ -20, else HOLD

```

Guardrails (can downgrade BUY → HOLD): - **LOW_VOLUME_TRAP**: volume ratio < 0.7× the 20-bar average. - **COUNTER_TREND**: BUY signal while market structure reads DOWNTREND. - **STALE_DATA**: newest bar older than 5 days (flagged, not suppressed). - **INSUFFICIENT_BARS**: fewer bars than the minimum → whole signal becomes **INSUFFICIENT_DATA** before any scoring runs.

ATR execution: stop = 2×ATR(14), target = 4×ATR(14) — fixed 2:1 reward:risk by construction. Entry zone = last close ± 0.25×ATR. **Position sizing is never computed** — **limitedBy="NOT_SIZED"**, documented as "needs an equity figure and a tracked cash balance; sizing without one would be invented."

Card Net-Value Optimizer

CardOptimizerService — **MIN_TRUSTWORTHY_COVERAGE = 0.50**.

```

projected = Σ over categories( category.projectedAnnualSpend × card.rateForCategory / 100 )
netValue  = projected - annualFee
breakEvenSpend = annualFee × 100 / bestRate    (null if fee=0 or bestRate=0)

```

Verdict precedence: 1. Spend coverage < 50% → UNKNOWN ("not enough categorized spend to judge this card") 2. `fee == 0` → KEEP 3. `netValue > 0` → KEEP 4. `bestRate == 0` → CANCEL 5. `projected == 0` (fee but no rewarded spend) → UNDERUSED 6. else → DOWNGRADE ("ask issuer about a no-fee variant rather than outright cancelling")

Spend categories are **inferred from merchant names, not real card-network MCCs** — every category is flagged `inferred=true`. Annualization ($365/90 \times \text{observed}$) is explicitly **not seasonally adjusted**.

Internal Transfers (Net-Worth Neutrality)

LedgerTransferService — enforced structurally, not by a runtime check: the only balance mutations are equal-and-opposite.

```
source.balance      -= amount      (if a tracked source account)
destination.balance += amount      (if a tracked destination account)
```

For a transfer into a non-cash asset (e.g., buying a mutual fund), only the cash side is touched here — the asset side is created independently by that asset's own import path. This is what keeps a bank→MF transfer from inflating net worth by the transferred amount.

02

Architecture — as built

The system as it exists in the repository today, file-verified.

01 Architecture Overview

02 Architecture Decisions (ADRs)

03 Database Architecture

04 Data Flow & Integrations

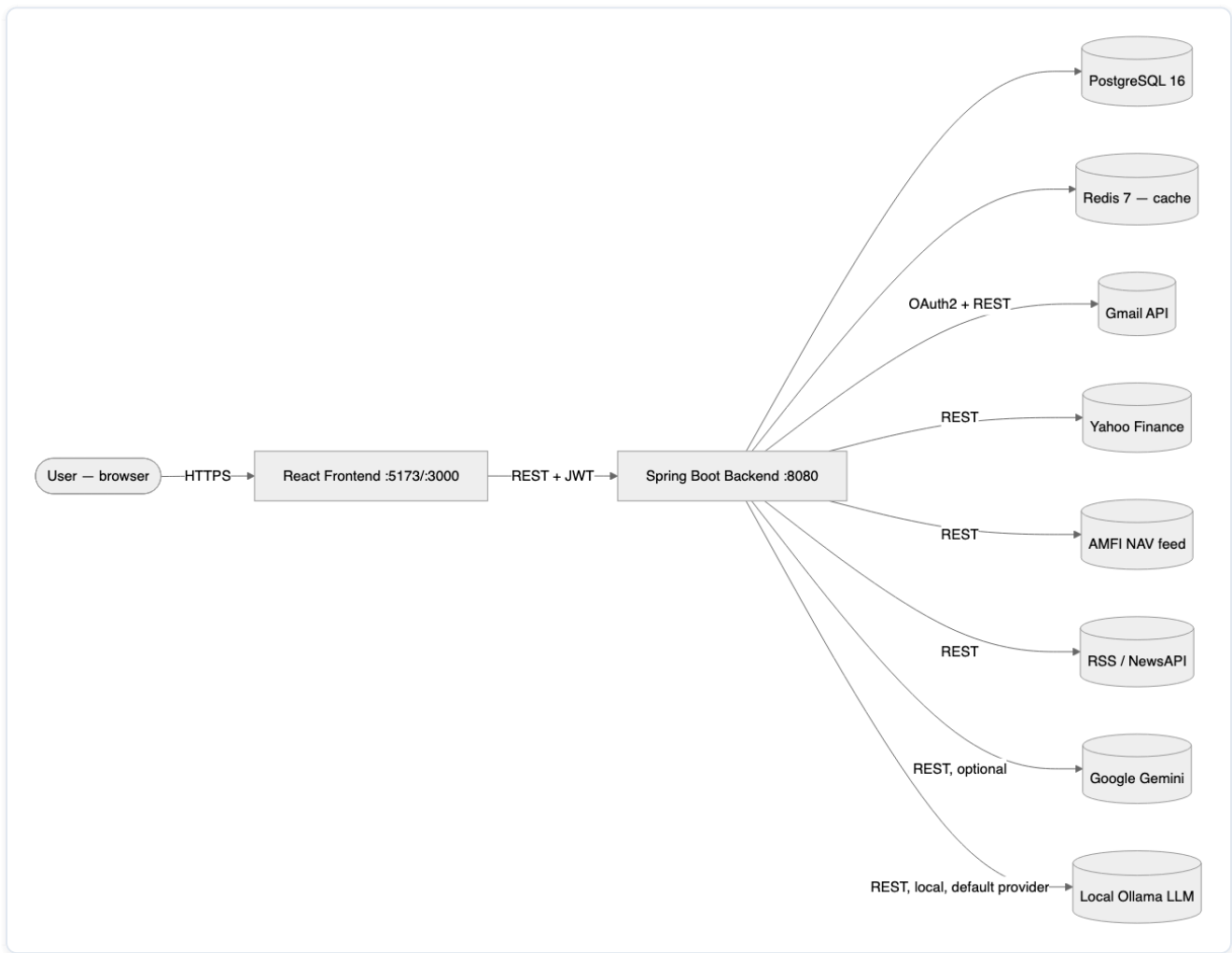
Architecture Overview

 Source file · docs/01-architecture/ARCHITECTURE_OVERVIEW.md

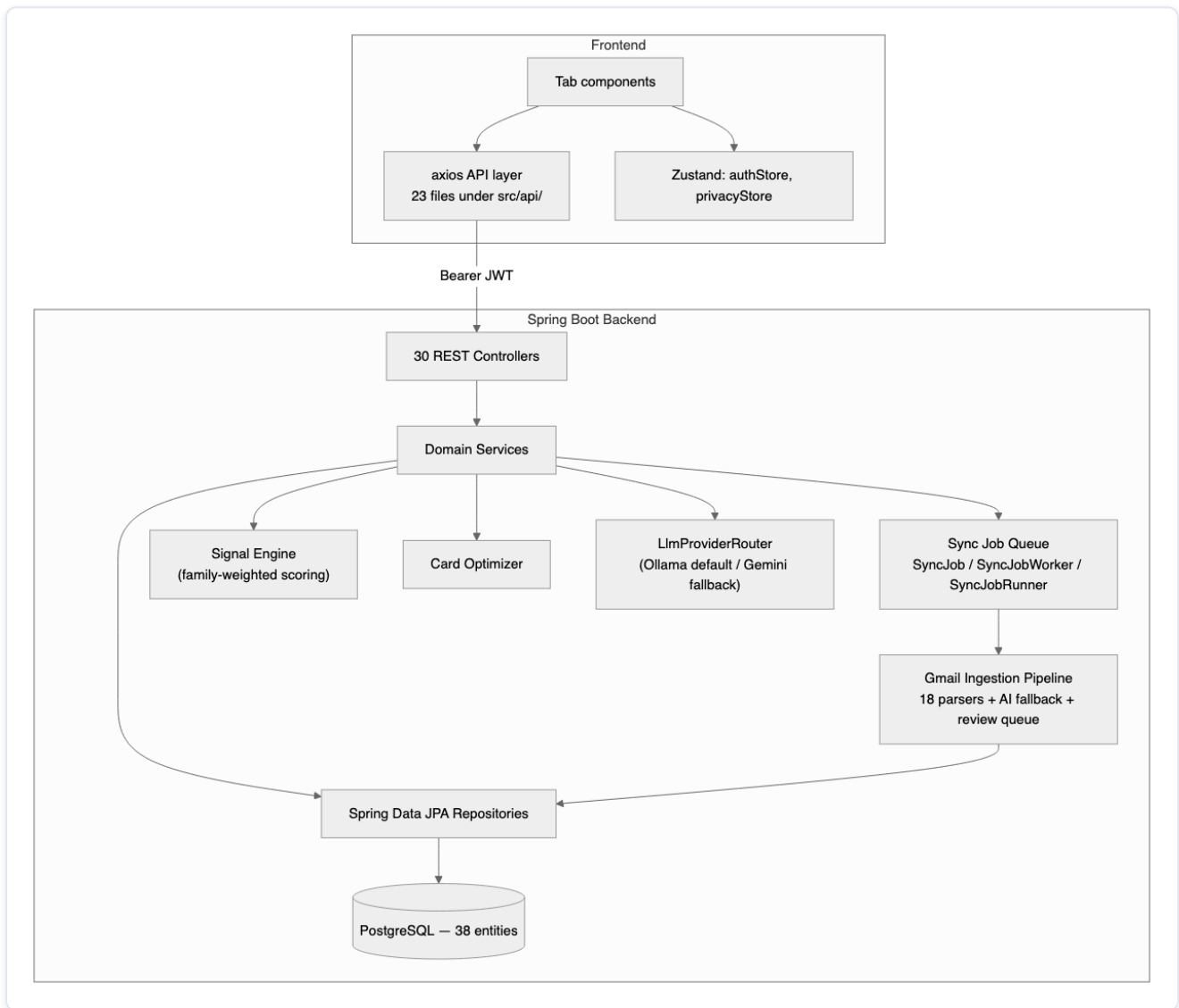
Stack

LAYER	TECHNOLOGY	VERSION (VERIFIED)
Backend runtime	Java	25 (<code>pom.xml</code> : <code>java.version / maven.compiler.release=25</code>)
Backend framework	Spring Boot	3.5.3
ORM	Hibernate/JPA (jakarta namespace)	via Spring Boot 3.5.3 BOM
Database	PostgreSQL	16 (<code>postgres:16-alpine</code> in <code>docker-compose.yml</code>)
Cache/queue substrate	Redis	7 (<code>redis:7-alpine</code>)
Auth	JJWT	0.12.6
API docs	springdoc-openapi	2.8.9
Frontend	React	19
Build tool	Vite	8
Styling	Tailwind CSS	3 (light-mode token palette)
Charts	Recharts	—
Client state	Zustand	2 stores (<code>authStore</code> , <code>privacyStore</code>)
Server-state fetching	None — raw <code>useState</code> + <code>useEffect</code> + <code>axios</code>	[PLANNED / NOT IMPLEMENTED] : no React Query/ SWR anywhere
Local AI	Ollama	<code>qwen2.5:7b</code> default, config-swappable to Gemini

System Context



Component Architecture



Layered pattern (per domain — see `ARCHITECTURE_DECISIONS.md` for deviations)

```

Controller  (@RestController, @AuthenticationPrincipal User for auth)
  ↓
Service     (@Service, @RequiredArgsConstructor, business logic)
  ↓
Repository  (JpaRepository<Entity, Long>, derived-name + @Query finders)
  ↓
Entity      (@Entity, Lombok @Data/@Builder, JPA annotations directly — no separate mapper layer)

```

30 controllers exist; not every domain has the full stack — `mf`, `forecast`, `recommendation`, `reconciliation`, `analyst`, `amfi`, `reminder`, `tax`, `signal`, `technical` are read-only/derived domains with no owned entity/repository. See `BACKEND_STRUCTURE.md` for the full per-domain map.

Security model

- **Stateless JWT** (`SessionCreationPolicy.STATELESS`), `BCryptPasswordEncoder(12)` .
- `JwtAuthFilter` runs before `UsernamePasswordAuthenticationFilter` in the chain.
- Explicit `permitAll` paths: `/api/auth/**` , `/api/gmail/callback` , `/api/gmail/push` (protected instead by a query-string shared secret, since Google Pub/Sub can't carry a JWT), `/api-docs/**` , `/swagger-ui/**` , `/actuator/health` .
- Everything else: `anyRequest().authenticated()` .
- No method-level `@PreAuthorize` / `@Secured` usage anywhere — auth is enforced entirely by the path allowlist plus `@AuthenticationPrincipal User user` being a required controller argument.
- `/api/admin/**` → `hasRole("ADMIN")` is configured but **no controller maps under it** (`[PARTIALLY IMPLEMENTED]` — dead rule).

Cross-cutting design principles (see `ARCHITECTURE_DECISIONS.md` for the "why")

1. **Ledger is the single source of truth** for holdings — never hand-set.
2. **One net-worth formula** (`PortfolioContextService`) consumed identically everywhere.
3. **Transfers are structurally net-worth-neutral** (equal-and-opposite balance mutation).
4. **A missing/uncertain input is `null` with a reason, never a guessed default.**
5. **Confidence is capped by data availability**, both in email classification (85% threshold) and signal scoring (`confidence = min(|composite|, ceiling)`).
6. **Idempotency by content, not by message ID** — a forwarded email with a new Gmail message ID still collides on the SHA-256 transaction fingerprint.

Architecture Decisions (ADRs)

 Source file · docs/01-architecture/ARCHITECTURE_DECISIONS.md

Each ADR reflects a decision actually visible in the code (a comment, a structural choice, or a migration), not a hypothetical.

ADR-1: Holdings are always derived from the transaction ledger, never hand-set

Decision: `PortfolioService.recomputeFromLedger(Holding)` is the *only* method permitted to set `Holding.quantity` / `averageCost`. It replays every `Transaction` row (BUY/SELL) in date order to arrive at a weighted-average cost basis.

Why: A holding's displayed quantity/cost must always be traceable to a specific, auditable set of buy/sell events. If a UI or API allowed directly setting these fields, the number could drift from what actually happened — the exact failure mode the standing rule ("never fabricate, silently drop, or overwrite financial records") is designed to prevent.

Consequence: A nightly `HoldingReconciliationScheduler` re-runs this replay for every user and logs when the previously-stored value had drifted from what the ledger implies — this is the system's built-in drift detector.

Trade-off accepted: this is weighted-average-cost, not FIFO/LIFO lot tracking. Tax reporting that requires per-lot cost basis is not supported (`[PLANNED / NOT IMPLEMENTED]`).

ADR-2: One net-worth formula, everywhere

Decision: `PortfolioContextService.build()` is the single computation for total assets, net worth, and asset-class percentages. `/api/wealth/summary` exposes it; every screen that shows a net-worth figure consumes this endpoint rather than computing its own.

Why: consolidating stocks + MF + FD/RD + EPF + cash + other assets – loans into one number, in one place, guarantees the dashboard, the advisory engine, and any report agree by construction — rather than by convention across N call sites.

ADR-3: Cash accounts exist specifically to prevent transfer-inflated net worth

Decision: `CashAccount` and `LedgerTransferService` were added so that moving money from a bank account into a mutual fund purchase does not increase net worth by the transferred amount.

Why (from the code's own comment in `PortfolioContextService`): before cash was tracked, an internal transfer looked identical to new income — the destination asset increased in value with nothing decreasing to

offset it. `LedgerTransferService.applyCashEffect` enforces the fix structurally: source account balance decreases, destination increases, by the same amount, in the same operation, guarded by an `applied` flag so it can never double-apply.

Consequence: any future asset type that money can move *into* (e.g., a new investment product) must route through `LedgerTransfer` with the correct `destinationType`, or this invariant breaks again for that asset class.

ADR-4: Idempotency by content fingerprint, not by message ID

Decision: `TransactionFingerprinter` hashes the *financial content* of a parsed email (type, date, amount, symbol/folio/bank, quantity — explicitly excluding Gmail message ID, sender, and free-text description) into a SHA-256 stored in `ImportedTransactionFingerprint`, checked before any transaction is written.

Why: a forwarded or resent copy of the same email gets a new Gmail message ID but describes the same real-world transaction. Deduplicating on message ID alone would double-book it. Content fingerprinting catches this regardless of delivery path.

Consequence: per-domain dedup checks (e.g., `isDuplicateExpense`, `isDuplicateTrade`) still exist as defense-in-depth in case a future import path bypasses the central fingerprint gate — documented as intentional redundancy, not oversight.

ADR-5: Local LLM by default, with a mandatory confidence floor and human review queue

Decision: Email classification falls back to a local Ollama model (`qwen2.5:7b`, `temperature=0.0`) only when no deterministic parser matches. Anything below 85% confidence, or of a type that must never be auto-booked (e.g., `INTERNAL_TRANSFER`), routes to `EmailReviewItem` instead of being imported.

Why: an LLM classification is inherently uncertain. Auto-importing a wrong classification would silently corrupt the ledger — worse than not importing at all. The system treats "the model didn't return a confidence score" identically to "confidence was too low" (never as implicit permission to import).

Trade-off accepted: local-first means no per-call API cost and no data leaving the machine, at the cost of depending on a local Ollama install. `Gemini` is available as a config-swappable fallback provider.

ADR-6: Async job queue for Gmail sync, not an inline request-blocking call

Decision: Syncing now goes through `SyncJob` → `SyncJobWorker` (bounded thread pool) → `SyncJobRunner`, with atomic job claiming via a conditional `UPDATE ... WHERE status = QUEUED` and automatic retry/stale-job reclaim.

Why: the original inline sync blocked the HTTP request for however long a full mailbox scan took — worse once the local LLM fallback was added, since each unparsed email costs real classification time. A durable

queue survives process restarts and lets the sync run in the background while the UI shows progress instead of hanging.

Consequence: `POST /api/gmail/sync` (inline) still exists for backward compatibility, but `POST /api/sync/gmail` (async, returns a job to poll) is the intended path going forward.

ADR-7: Incremental sync via Gmail's history API, with a strict monotonic watermark

Decision: `GmailIncrementalSyncService` tracks `GmailToken.lastHistoryId` and only ever moves it forward (`advanceWatermark`), never backward, and only after `history.list` results were fully paginated ("complete"). A stale/expired historyId (Gmail 404) forces a full re-scan rather than silently skipping the gap.

Why: Gmail's history API can return partial pages, and a historyId can age out of Gmail's retention window. Advancing the watermark on partial or stale data would create a silent gap in ingested transactions — the exact class of bug this ADR exists to prevent.

ADR-8: Signal confidence is capped by data availability, not by the score alone

Decision: `SignalEngine.confidence = min(|composite score|, confidence ceiling)`, where the ceiling is the percentage of total factor-family weight that had usable data. A signal computed from 3 of 5 families (order-book/microstructure is *always* unavailable — no data source exists) cannot report a confidence higher than what fraction of factors were actually computed.

Why: presenting a strong-looking composite score as if it were fully corroborated, when a whole factor family (e.g., microstructure) was silently unavailable, would overstate the engine's actual certainty.

Related: `SignalEngine` scores by *factor family* (trend/momentum/structure/volume/ volatility) rather than by individual indicator, specifically because correlated oscillators (RSI/CCI/Stochastic/ROC/MACD-line all measure similar things) would otherwise silently over-weight momentum under naive per-indicator averaging.

ADR-9: `Long userId` vs. real `@ManyToOne User` — accepted inconsistency, not yet unified

Decision (implicit — visible as a pattern, not a single commit): newer modules (portfolio, ledger, actions, tracking, sync) use a real JPA `@ManyToOne` relationship to `User`; older modules (gmail, card, expense, income, redemption, ai/audit, ai/review, goal, network) use a bare `Long userId` column with an index but no FK constraint.

Why this exists: reflects incremental modernization over time rather than a deliberate two-tier design.

[PARTIALLY IMPLEMENTED — no migration plan to unify this was found in the codebase; documented here as a known inconsistency for future architects to resolve deliberately.]

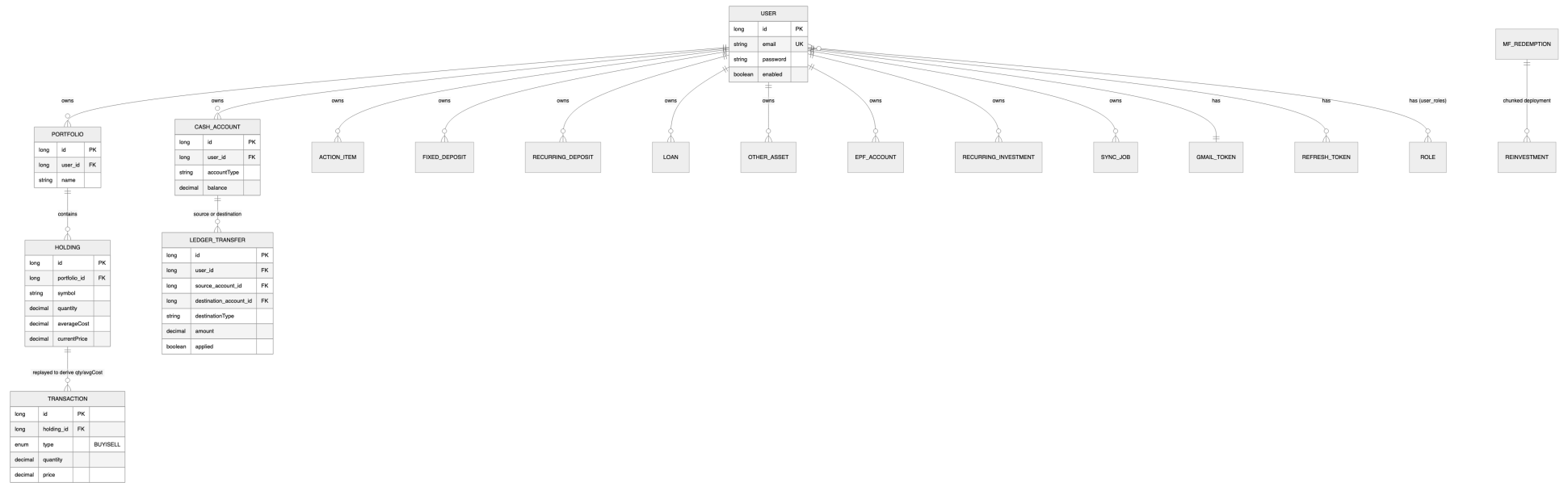
Risk carried: without an FK constraint, deleting a `User` row would not cascade or block on these tables — an orphaned row is possible at the DB level (though no code path for deleting a `User` was found during this audit).

Database Architecture

 [Source file · docs/01-architecture/DATABASE_ARCHITECTURE.md](#)

38 `@Entity` classes, PostgreSQL 16, Hibernate `ddl-auto=update` (dev) / `validate` (prod — schema must be pre-migrated in prod, no auto-DDL).

Entity-relationship diagram (core financial domain)



Note on bare `userId` entities: many entities below (see §"User FK pattern") use a plain `Long userId` column with an index, but **no** `@ManyToOne /FK constraint` to `User`. These are omitted from the ERD's relationship lines above for clarity but are documented in the table below.

Full entity inventory by domain

AUTH

ENTITY	TABLE	KEY COLUMNS	RELATIONSHIPS
Role	roles	name (unique, enum <code>RoleName{ROLE_USER,ROLE_ADMIN,ROLE_PREMIUM}</code>)	inverse of <code>User.roles</code>
User	users	email (unique), password, enabled, emailVerified	<code>@ManyToMany (EAGER)</code> → <code>Role</code> via <code>user_roles</code> join table
RefreshToken	refresh_tokens	token (unique, len 512), expiresAt, revoked	<code>@ManyToOne</code> → <code>User</code> (real FK)

PORTFOLIO

ENTITY	TABLE	KEY COLUMNS	RELATIONSHIPS
Portfolio	portfolios	name, description	<code>@ManyToOne</code> → <code>User</code> ; <code>@OneToMany (cascade=ALL, orphanRemoval)</code> → <code>Holding</code>
Holding	holdings	unique (portfolio_id, symbol); quantity (4dp), averageCost (2dp), xirr	<code>@ManyToOne</code> → <code>Portfolio</code> ; <code>@OneToMany (cascade=ALL, orphanRemoval)</code> → <code>Transaction</code> . Transient: <code>getInvestedValue/</code> <code>getCurrentValue/getPnl/</code> <code>getPnlPercent</code>
Transaction	transactions	type (enum BUY/SELL), quantity, price, charges	<code>@ManyToOne</code> → <code>Holding</code>

MARKET

ENTITY	TABLE	KEY COLUMNS
Stock	stocks	symbol (unique), OHLC + fundamentals (all nullable — Yahoo omits many), active
PriceHistory	price_history	symbol, date, barStart (intraday), interval (default "1d", column named <code>bar_interval</code> to dodge the PG reserved word), OHLC+volume

ENTITY	TABLE	KEY COLUMNS
MarketIndex	market_indices	symbol (unique), value/change fields

MF

ENTITY	TABLE	KEY COLUMNS
MfNavHistory	mf_nav_history	unique (scheme_code, date), nav (6dp — lossless vs. AMFI's 4-5dp)

GMAIL

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
GmailToken	gmail_tokens	accessToken / refreshToken (2000-char, plaintext), lastHistoryId, watchExpiration	@OneToOne → User (real FK)
ProcessedEmail	processed_emails	unique (user_id, gmail_message_id), status (IMPORTED/SKIPPED/FAILED)	bare userId
ExcludedSender	excluded_senders	unique (user_id, pattern)	bare userId
PendingPdf	pending_pdfs	unique (user_id, gmail_message_id, attachment_id), status, pipelineSteps (JSON text)	bare userId
SavedPdfPassword	saved_pdf_passwords	unique (user_id, provider_key), encryptedPassword (AES-GCM ciphertext, never returned to frontend)	bare userId
ImportedTransactionFingerprint	imported_transaction_fingerprints	unique (user_id, fingerprint) — SHA-256, 64 chars	bare userId

CARD

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
CreditCard	credit_cards	annualFee , pointsBalance , pointValue	bare <code>userId</code> ; <code>@ElementCollection</code> → <code>card_reward_rates</code> map (category→rate)
CardRewardRule	card_reward_rules	cardName , category , rewardRate , monthlyCapAmount , effectiveFrom / effectiveTo , lastVerifiedAt	none — global reference data. Methods: <code>isCurrentOn(date)</code> , <code>isStale(today,</code> <code>maxAgeDays)</code>

LEDGER

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
CashAccount	cash_accounts	accountType , balance (authoritative — only changed via transfer or explicit correction)	<code>@ManyToOne</code> → <code>User</code> (real FK)
LedgerTransfer	ledger_transfers	destinationType (string, not FK — far side varies), amount , applied (guards double-apply)	<code>@ManyToOne</code> → <code>User</code> , optional <code>@ManyToOne</code> → <code>CashAccount</code> x2 (source/ destination)

ACTIONS

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
ActionItem	action_items	unique (user_id, action_type, symbol, action_date) , status (enum <code>ActionStatus</code>)	<code>@ManyToOne</code> → <code>User</code> (real FK — code comment notes this follows the newer pattern, not the older bare- <code>userId</code> style)

AI / AUDIT / REVIEW

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
AiHistory	ai_history	queryType (enum), prompt / response (TEXT)	<code>@ManyToOne</code> → <code>User</code> (real FK, nullable)

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
AiAuditTrail	ai_audit_trail	task, provider, model, confidence, status (plain string, not enum), prompt/output columns declared columnDefinition="text" deliberately not @Lob (a @Lob String on Postgres becomes an orphan-prone oid large object)	bare userId
EmailReviewItem	email_review_items	unique (user_id, gmail_message_id), status (enum ReviewStatus), extractedFields (raw JSON text)	bare userId

TRACKING

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
EpfAccount	epf_accounts	rate (default 8.25 = EPFO FY2024-25)	@ManyToOne → User (real FK)
FixedDeposit	fixed_deposits	status (plain string: ACTIVE/CLOSED/MATURED_RENEWED), renewedToId / renewedFromId (bare Long self-refs, no FK)	@ManyToOne → User (real FK)
RecurringDeposit	recurring_deposits	status (ACTIVE/MATURED/CLOSED/MATURED_RENEWED), tenureMonths . Transient getMaturityDate()	@ManyToOne → User (real FK)
Loan	loans	emi, outstanding, rate, remainingMonths	@ManyToOne → User (real FK)
OtherAsset	other_assets	category (plain string: ppf/epf/nps/gold/realstate/...)	@ManyToOne → User (real FK)

REDEMPTION

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
MfRedemption	mf_redemptions	gainType (STCG/LTCG), status (ACTIVE/COMPLETED, plain string). Method getCashRemaining() = redeemedAmount – reinvestedAmount, floored at 0	bare userId ; @OneToMany(cascade=ALL, orphanRemoval) → Reinvestment

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
Reinvestment	reinvestments	amount, date, targetFund	@ManyToOne → MfRedemption

SCHEDULED

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
RecurringInvestment	recurring_investments	type (enum SIP/PPF/NPS), status (plain string ACTIVE/PAUSED/COMPLETED)	@ManyToOne → User (real FK)

SYNC

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
SyncJob	sync_jobs	type / status / trigger (all enums), attempts / maxAttempts (default 3), claimedBy / claimedAt (crash detection). Method isTerminal()	@ManyToOne → User (real FK)

EXPENSE / INCOME

ENTITY	TABLE	KEY COLUMNS	FK PATTERN	NOTE
Expense	expenses	category — typed as ExpenseCategory enum with no @Enumerated annotation	bare userId	[PARTIALLY IMPLEMENTED — modeling quirk]: without @Enumerated or a registered @Convert, Hibernate's default mapping behavior for this field should be verified; it is not annotated as ordinal- or string-mapped at the field level
Income	incomes	source — typed as IncomeSource enum, persisted via IncomeSourceConverter (maps to display label, so legacy free-text rows read back unchanged)	bare userId	Correctly converter-backed, unlike Expense.category

GOAL / NETWORTH / NEWS

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
FinancialGoal	financial_goals	targetAmount, currentSaved, expectedReturn (default 10%)	bare userId

ENTITY	TABLE	KEY COLUMNS	FK PATTERN
NetWorthSnapshot	net_worth_snapshots	unique (user_id, snapshot_date)	bare user_id
News	news	url (unique), sentiment (enum POSITIVE/NEGATIVE/NEUTRAL)	none — global content

User FK pattern: real FK vs. bare Long user_id

Real @ManyToOne / @OneToOne + @JoinColumn to User: RefreshToken, Portfolio, GmailToken (@OneToOne), CashAccount, LedgerTransfer, ActionItem, AiHistory, EpfAccount, FixedDeposit, Loan, OtherAsset, RecurringDeposit, RecurringInvestment, SyncJob.

Bare Long user_id (no FK, no relationship object): ProcessedEmail, ExcludedSender, PendingPdf, SavedPdfPassword, ImportedTransactionFingerprint (gmail), CreditCard (card), Expense, Income, MfRedemption, AiAuditTrail, EmailReviewItem, FinancialGoal, NetWorthSnapshot.

This is a clean split by module age: newer/refactored modules (auth, portfolio, ledger, actions, tracking, sync, ai/history) use real FKs; older modules (gmail, card, expense, income, redemption, ai/audit, ai/review, goal, networth) still use bare longs. [PARTIALLY IMPLEMENTED – architectural inconsistency, not a bug: bare-user_id tables still have an index on the column, so query performance is unaffected, but referential integrity at the DB level is not enforced for those tables.]

Every enum type in the codebase

ENUM	FILE	VALUES
Role.RoleName (nested)	auth/entity/Role.java	ROLE_USER, ROLE_ADMIN, ROLE_PREMIUM
AiHistory.QueryType (nested)	ai/entity/AiHistory.java	STOCK_ANALYSIS, MARKET_SUMMARY, PORTFOLIO_REVIEW, FORECAST, CHAT
Transaction.TransactionType (nested)	portfolio/entity/Transaction.java	BUY, SELL
News.Sentiment (nested)	news/entity/News.java	POSITIVE, NEGATIVE, NEUTRAL
RecurringInvestment.Type (nested)	scheduled/entity/RecurringInvestment.java	SIP, PPF, NPS
ActionStatus	actions/entity/ActionStatus.java	PENDING, EXECUTED, SKIPPED, SNOOZED
ReviewStatus	ai/review/entity/ReviewStatus.java	PENDING, ACCEPTED, EDITED, REJECTED

ENUM	FILE	VALUES
<code>SyncJobStatus</code>	<code>sync/entity/ SyncJobStatus.java</code>	<code>QUEUED</code> , <code>RUNNING</code> , <code>SUCCEEDED</code> , <code>FAILED</code> , <code>CANCELLED</code>
<code>SyncJobType</code>	<code>sync/entity/ SyncJobType.java</code>	<code>GMAIL_FULL_SYNC</code> , <code>GMAIL_INCREMENTAL_SYNC</code> , <code>GMAIL_RETRY_FAILED</code>
<code>SyncTrigger</code>	<code>sync/entity/ SyncTrigger.java</code>	<code>MANUAL</code> , <code>SCHEDULED</code> , <code>PUSH</code> , <code>RECOVERY</code>
<code>ExpenseCategory</code>	<code>expense/entity/ ExpenseCategory.java</code>	<code>FOOD</code> , <code>SHOPPING</code> , <code>TRAVEL</code> , <code>FUEL</code> , <code>BILLS</code> , <code>MEDICAL</code> , <code>ENTERTAINMENT</code> , <code>EMI</code> , <code>UPI</code> , <code>INVESTMENT</code> , <code>UNCATEGORIZED</code>
<code>IncomeSource</code>	<code>income/entity/ IncomeSource.java</code>	<code>SALARY</code> , <code>BONUS</code> , <code>FREELANCE</code> , <code>BUSINESS</code> , <code>DIVIDEND</code> , <code>INTEREST</code> , <code>CAPITAL_GAIN</code> , <code>RENTAL</code> , <code>OTHER</code>
<code>EmailIntelType</code>	<code>ai/intel/ EmailIntelType.java</code>	26 values (see <code>02-backend/ GMAIL_INGESTION_PIPELINE.md §6</code>) — not persisted, in-memory classification result only

Not real enums (plain `String` columns with comment-documented allowed values): `MfRedemption.status`, `RecurringInvestment.status`, `FixedDeposit.status`, `RecurringDeposit.status`, `PendingPdf.status`, `AiAuditTrail.status`.

Lifecycle callbacks (`@PrePersist` / `@PreUpdate`)

ENTITY	CALLBACK	EFFECT
<code>ActionItem</code> , <code>GmailToken</code> , <code>CashAccount</code>	both <code>onCreate</code> + <code>onUpdate</code>	timestamps set/ refreshed
<code>AiAuditTrail</code> , <code>EmailReviewItem</code> , <code>ImportedTransactionFingerprint</code> , <code>ProcessedEmail</code> , <code>LedgerTransfer</code> , <code>CardRewardRule</code> , <code>SyncJob</code>	<code>onCreate</code> only	<code>createdAt</code> set if null
<code>User</code> , <code>Portfolio</code> , <code>OtherAsset</code>	<code>onUpdate</code> only	<code>updatedAt</code> refreshed

All other entities rely on `@Builder.Default = LocalDateTime.now()` at construction — no lifecycle callback.

Indexing summary

Every multi-tenant table indexes on `user_id` (or `(user_id, status)` / `(user_id, date)` composites) regardless of whether the FK is real or bare. Unique constraints exist specifically to enforce idempotency: `(user_id, gmail_message_id)` on 3 gmail tables, `(user_id, fingerprint)` on the fingerprint table, `(portfolio_id, symbol)` on `Holding`, `(user_id, action_type, symbol, action_date)` on `ActionItem`.

Data Flow & Integrations

 Source file · [docs/01-architecture/DATA_FLOW_AND_INTEGRATIONS.md](#)

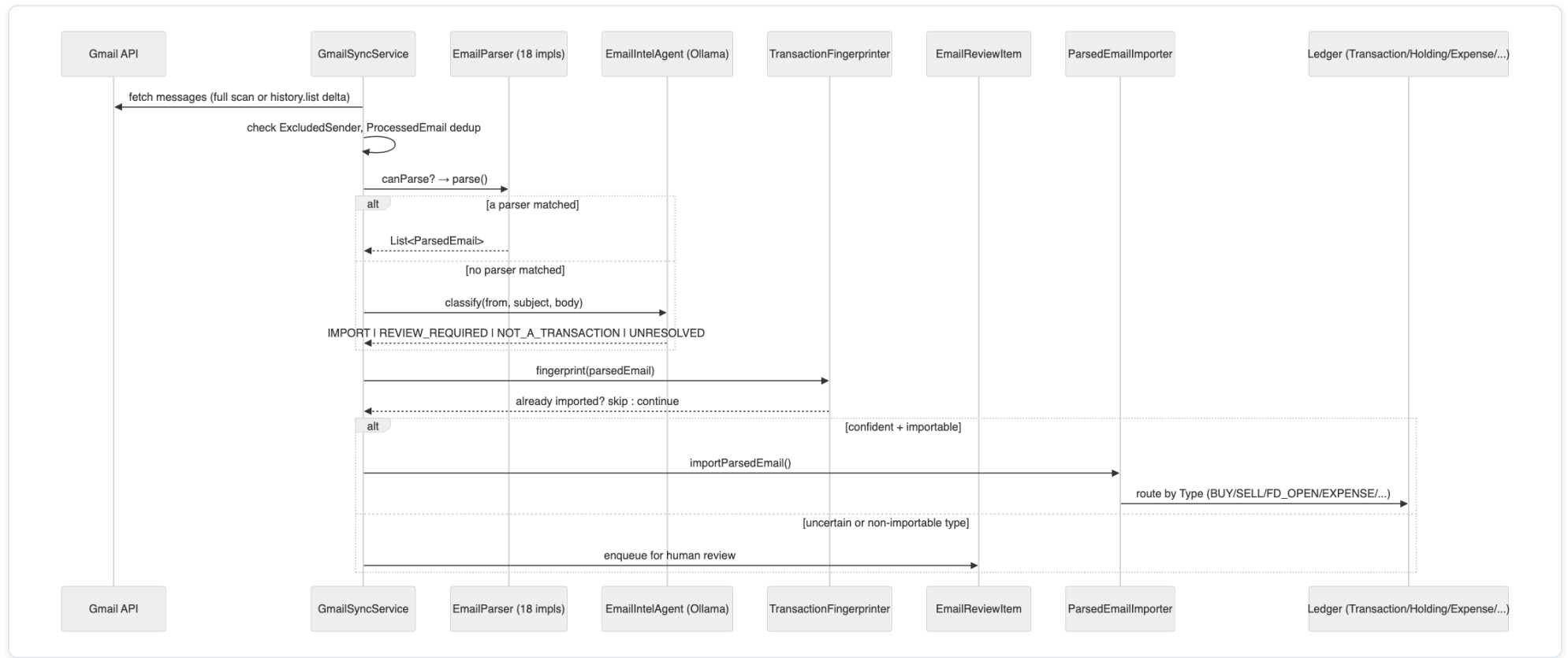
Integration inventory

INTEGRATION	PURPOSE	AUTH	TRIGGER	STATUS
Gmail API	Email discovery, OAuth, push watch	OAuth2 (offline access, refresh token)	Manual sync, 30-min scheduled poll, Gmail push (optional)	[IMPLEMENTED]
Yahoo Finance (<code>query1.finance.yahoo.com</code>)	Quotes, historical OHLC, index data	None (public endpoints)	On-demand + scheduled cache refresh (15 min) + daily sector-history refresh (18:30 IST weekdays)	[IMPLEMENTED]
AMFI (<code>portal.amfiindia.com</code> — see note)	Mutual fund NAV	None	Daily at 21:30 (<code>AmfiNavService.refresh</code>)	[IMPLEMENTED]
RSS feeds (Google News, Economic Times, Moneycontrol x2, LiveMint, Business Standard)	Free news source, dev default	None	Every 30 min (<code>app.news.refresh-rate-ms</code>)	[IMPLEMENTED]
NewsAPI (<code>newsapi.org</code>)	Prod news source	API key	Same 30-min cycle, prod profile only	[IMPLEMENTED] (prod-only; RSS)
Google Gemini	AI narrative, portfolio review, chat, optional email classification	API key	On-demand	[IMPLEMENTED]
Ollama (local)	Default LLM provider — email classification, ambiguity resolution	None (local)	On-demand	[IMPLEMENTED] — default <code>qwen2.5:7b</code> at <code>localhost</code>
Google Cloud Pub/Sub	Gmail push notification transport	Shared-secret query token	Real-time (if enabled)	[IMPLEMENTED], disabled by <code>(app.gmail.push.enable</code>

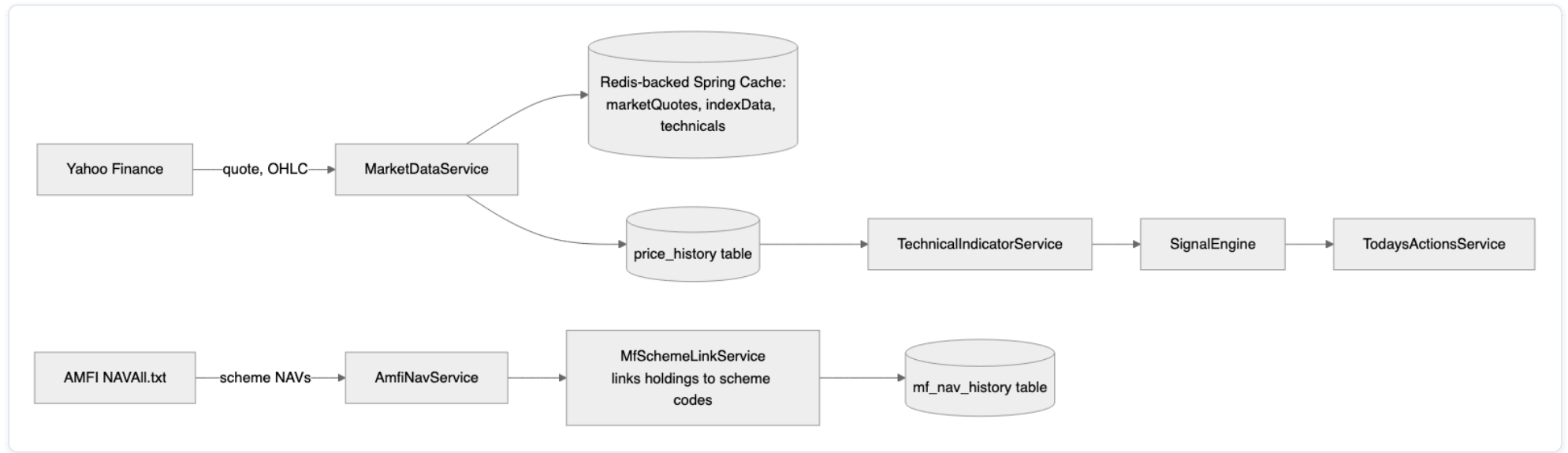
Note on AMFI URL: `AmfiNavService` fetches from `portal.amfiindia.com`, not `www.amfiindia.com` — the `www` host 302-redirects, and `WebClient` does not follow redirects by default, so the code fetches the canonical redirect target directly (with `redirect-following` explicitly enabled as a defensive measure).

Data flow — Gmail ingestion → ledger (high-level; see `05-flows/`

`GMAIL_TO_PORTFOLIO_FLOW.md` for the full trace)



Market data flow



AI provider routing

`LlmProviderRouter` selects between providers based on `app.llm.provider`:

VALUE	PROVIDER	NOTES
<code>ollama</code> (default)	Local Ollama, <code>qwen2.5:7b</code>	No API cost, <code>temperature=0.0</code> for reproducible classification
<code>gemini</code>	Google Gemini	Requires <code>GEMINI_API_KEY</code>
<code>none</code>	Disabled	All AI-dependent paths (email classification fallback) are skipped; deterministic parsers still work

`app.llm.fallback-enabled=true` allows falling back between providers if the primary fails (exact fallback behavior: see `02-backend/BACKEND_STRUCTURE.md` `ai/llm` package).

Caching

Spring Cache (`spring.cache.type=simple` — in-memory, not Redis-backed despite Redis being provisioned) covers `marketQuotes`, `indexData`, `technicals` — evicted every 15 minutes by `MarketDataService.refreshMarketData()` (`@CacheEvict(allEntries=true)` , a pure cache-bust with no fetch of its own). Redis itself (`spring-boot-starter-data-redis`) is a live dependency but its concrete usage beyond the `spring.cache.type=simple` default was not verified beyond connection config — **[PARTIALLY IMPLEMENTED – Redis is provisioned but Spring Cache is configured to use the in-memory simple provider, not Redis, in the profiles inspected]**.

03

Architecture — target

What we are building toward, and the evidence behind each change.

01 Target Architecture: How To Read It

02 Target System Architecture

03 Target Data Architecture

04 The Financial Ledger

Target Architecture: How To Read It

 [Source file · docs/architecture/README.md](#)

This directory is the architecture we are designing toward. `../01-architecture/` documents the system as it exists today and remains the source of truth for current behaviour. Where the two disagree, `01-architecture` describes reality and this directory describes intent.

Produced in the DESIGN phase, after [RESEARCH](#) → [AUDIT](#) → [BENCHMARK](#). No code has been written against these documents yet.

DOCUMENT	SCOPE
SYSTEM_ARCHITECTURE.md	Layer map, what changes and why, data-acquisition topology, build sequence
DATA_ARCHITECTURE.md	The three data tiers, document/identity/reference entities, MF holdings, attribution, tax lots, migration posture
FINANCIAL_LEDGER.md	The single-writer invariant, replay semantics, transfer neutrality, the seven properties to preserve
diagrams/	Standalone Mermaid sources (all render-verified)

Reading order

1. `SYSTEM_ARCHITECTURE.md` §1–2 — the organising principle and the layer map
2. `FINANCIAL_LEDGER.md` — what already works and must not break
3. `SYSTEM_ARCHITECTURE.md` §3 — the changes, each traced to research evidence
4. `DATA_ARCHITECTURE.md` — the schema that supports them

Conventions

Sections marked **TARGET** are proposals. Everything else describes verified current behaviour, cited as `file:line`. Evidence for design choices is labelled **[A]** documented / **[B]** inference / **[C]** speculation in `../research/` and referenced rather than restated here.

Status

PHASE	STATE
Research	Complete — 14 agents, 4 deliverables
Audit	Complete — verified against code

PHASE	STATE
Benchmark	Complete — disposition recorded in <code>../research/COMPETITIVE_RESEARCH.md</code> §9
Design	Complete — this directory
Implement	Phases 0–3 complete. Phases 4–7 core engines built and tested. 416 tests green, zero deprecation warnings, zero DDL failures. Remaining: per-AMC MF ingestion (3–6wk), effective-dated card rules, tax-aware rebalancing, routing cutover (needs real mail), sector scoring (needs a fundamentals feed)

The build sequence is in `SYSTEM_ARCHITECTURE.md` §6. Phase 0 is cheap and protective; Phase 4 explicitly keeps the existing scorer serving until a replacement passes a comparison harness.

Target System Architecture

 Source file · [docs/architecture/SYSTEM_ARCHITECTURE.md](#)

This directory is the *target* architecture — what we are designing toward.

`docs/01-architecture/` documents the system **as built today** and remains the source of truth for current behaviour. Where the two disagree, `01-architecture` describes reality and this document describes intent.

Every design choice here traces to a finding in `../research/COMPETITIVE_RESEARCH.md` or a verified gap in `../research/TECHNICAL_GAPS.md`. **No code has been written against this yet.**

1. The organising principle

The system has one job that everything else is subordinate to:

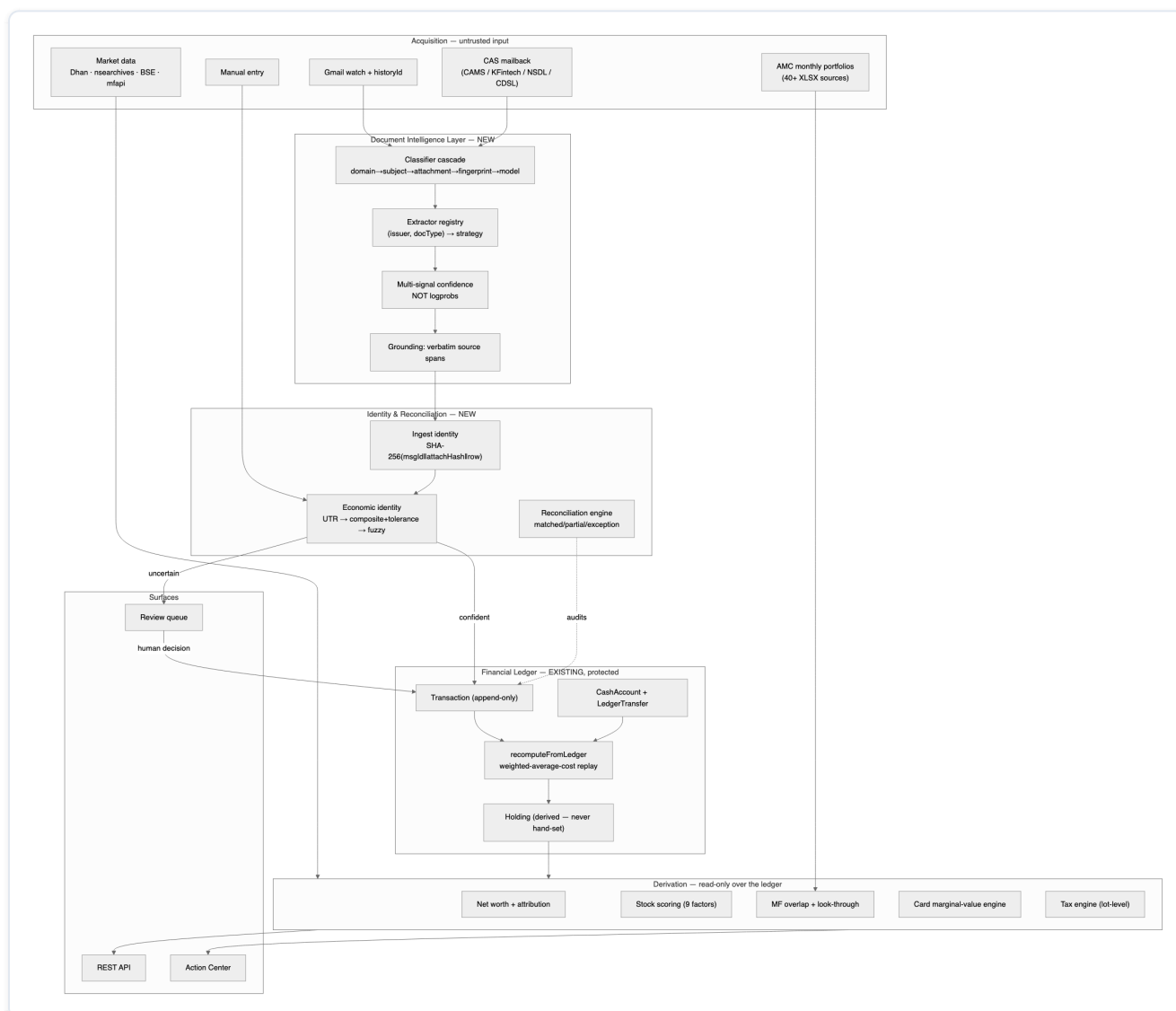
Produce financial numbers that are traceable to a source, reconcilable against that source, and honest about what they don't know.

That is the load-bearing constraint. Three architectural consequences follow, and they are not negotiable:

- 1. The ledger is the only writer of derived financial state.** Verified intact today — `Holding.quantity / averageCost` are written at exactly one place, `PortfolioService.java:258–259`, inside `recomputeFromLedger`.
- 2. Every number carries its provenance.** Today provenance stops at the email. Target is the **verbatim source span** within the document.
- 3. Every computation can decline.** The types now exist — `common/quality/DataQuality` and `Sufficiency<T>`. Migration of remaining call sites to `Sufficiency<T>` is ongoing; the magic-string `dataQuality` markers are already gone.

Two market leaders independently validate #3: Value Research publishes **five exclusions** that make a stock unratable, and Zerodha **suppresses** portfolio XIRR rather than show a misleading one. Refusing to answer is a feature, not a failure.

2. Layer map



The rule the diagram encodes: everything above the ledger is *untrusted and reversible*; everything below it is *derived and recomputable*. Nothing writes to `Holding` directly. Nothing in the derivation layer writes at all.

3. What changes, and why

3.1 NEW: DOCUMENT INTELLIGENCE LAYER

Problem (verified): no generic `Document` abstraction exists — only `PendingPdf`, plus **18 parsers each with independent logic**. Adding an issuer means a new parser class; nothing enumerates "all documents in flight"; there is no shared lifecycle, provenance, or retry.

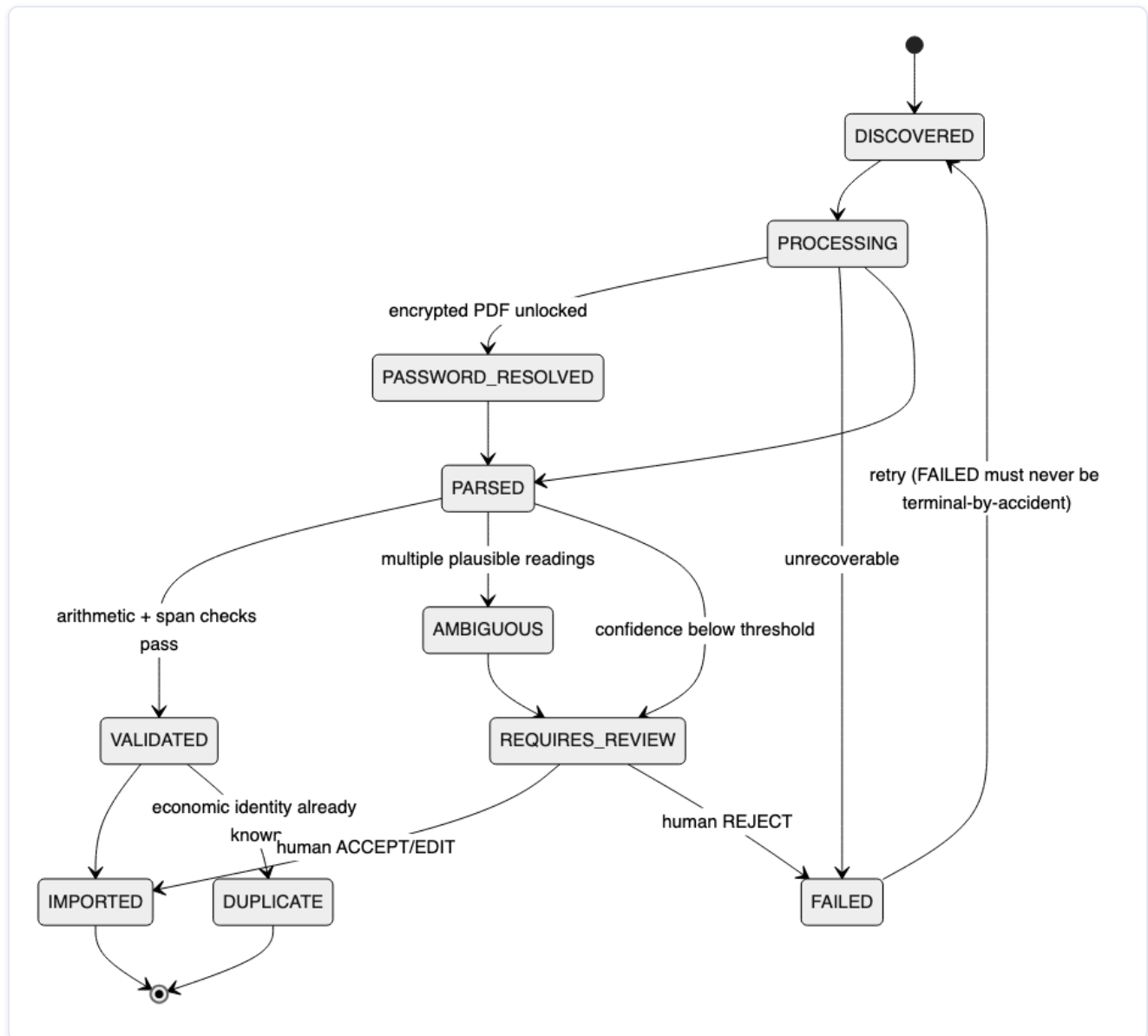
Design:

```

FinancialDocument (aggregate root)
├─ source: GMAIL | CAS_MAILBACK | MANUAL_UPLOAD
├─ sourceRef: gmailMessageId | uploadId
├─ status: one of the 10 spec'd states
├─ classification: (docType, issuer, confidence, matchedStage)
├─ extractions: List<ExtractedField>
│   └─ { name, value, sourceSpan, page, bbox?, confidence }
└─ provenance: { receivedAt, classifiedAt, extractedAt, decidedAt, decidedBy }

```

The ten statuses, as a state machine rather than a loose enum:



FAILED must be retryable. This was a real bug class already fixed once in this codebase (a transient failure permanently blocking an email); the state machine makes it structural rather than a convention.

Classifier cascade — ordered stages, each may abstain:

#	STAGE	COST	PRECISION
1	Sender domain + DKIM-verified domain	~0	Highest
2	Subject regex	~0	High
3	Attachment filename / MIME / PDF producer metadata	low	High
4	First-page text fingerprint	low	Medium
5	Small-model classifier	8.2s measured	Variable

Jupiter — the only Indian fintech to publish its actual Gmail query — runs **stages 1–2 only** and stops there. We keep 3–5 behind them for long-tail coverage, but the cheap stages must win the common case. This also caps the cost of our measured 8.2s/email LLM latency by making it rare rather than routine.

3.1A SENDER AUTHORITY — A WEAKNESS FOUND WHILE BUILDING THIS

Not identified during the design phase; found while grounding the domain registry in what the parsers actually match.

The legacy routing asks `ParserUtil.containsIgnoreCase(from, "zerodha")`, and `from` is the **raw From header** as returned by `GmailClientService.getFrom` — display name included. The display name is set by whoever sent the message.

```
"Zerodha Alerts" <noreply@attacker.example>    → matches containsIgnoreCase(from, "zerodha")
                                                → routed to ZerodhaParser
                                                → whatever it extracts heads toward the ledger
```

Substring matching also accepts `notzerodha.com` and `zerodha.com.evil.example`.

Mitigating factors: the message must already be in the user's own mailbox, and the fingerprint check plus the review queue sit downstream. **It still matters**, because the whole premise of the system is that a stored financial record is trustworthy, and this is a path to planting one.

`SenderDomainStage` matches the **registrable domain of the envelope address** instead — parsing inside `<...>` when present, and requiring an exact or true-subdomain match. `SenderDomainSpoofingTest` pins the behaviour against display-name spoofs and lookalike domains.

IMPLEMENTED — `SenderTrustEvaluator` now gates the live sync path.

The gate sits in `GmailSyncService` *after* parsing and *before* import, so a held-back email reaches the review queue carrying its extracted content rather than as a bare "something was blocked" row. **Nothing is dropped** — each parsed item becomes a review row the user can accept if the sender is in fact genuine.

Scope is deliberately narrow — only the **impersonation shape** is blocked:

SENDER	TRUST	IMPORTS?
"Zerodha" <noreply@zerodha.com>	VERIFIED_DOMAIN	Yes

SENDER	TRUST	IMPORTS?
statements@somenewbroker.in	UNKNOWN_DOMAIN	Yes — recorded, not blocked
"Zerodha Alerts" <noreply@attacker.example>	IMPERSONATION_SUSPECTED	No → review
"Zerodha Support" <noreply@groww.in>	IMPERSONATION_SUSPECTED	No → review
alerts@zerodha.com.evil.example	IMPERSONATION_SUSPECTED	No → review

UNKNOWN_DOMAIN passing is the load-bearing decision. The issuer registry will always lag reality, and blocking unrecognised domains would mean any bank not yet catalogued silently stops importing — a cost far larger than the risk it removes. Only a sender making a *claim it cannot support* is held back, because that is the only case with positive evidence of a lie.

GmailSenderTrustGateTest proves the wiring, not just the evaluator: disabling the gate makes the test go red showing importParsedEmail **was** called on the spoofed message — direct evidence that the trade would otherwise have reached the ledger.

Selection routing — built, running in shadow mode, not yet deciding.

ParserRouter orders candidates using the classification: the issuer's own parser first, then the cross-issuer generics (BankTransactionParser , CardBillParser , DividendParser , UpiAppParser), then everything else as a last resort.

Routing reorders the candidate list; it never removes from it. When the issuer is unknown the full legacy list is returned unchanged. Narrowing is precisely how a routing change silently stops importing a real sender, so the invariant is asserted directly rather than inferred from sampled outcomes.

SelectionComparator runs both paths on every synced email and tallies the verdicts — AGREE / AGREE_NEITHER / ROUTED_ONLY / LEGACY_ONLY / DISAGREE . It calls canParse only, never parse , so it cannot double-apply a parser side effect or double the cost of a sync, and it swallows its own failures so an observability feature can never break an import.

The cutover gate is GET /api/gmail/selection-comparison , which reports safeToCutOver: regressions == 0 && total > 0 . Cutover happens on that evidence over real mail, not on passing unit tests.

ParserRoutingCoverageTest runs both paths over **all 19 real parsers** across 26 realistic samples: zero regressions. One honest caveat recorded in that test — a deliberately narrowed router was caught by the invariant assertion, not the sampling one, because producing a LEGACY_ONLY verdict requires a known issuer whose matching parser is neither its own nor a generic, and the parsers gate on issuer tokens in the From header. Both tests are kept; the invariant one is load-bearing.

Still open: the legacy substring loop is still what *decides*. Routing decides only once the tally is clean on real traffic.

3.2 CONFIDENCE: REPLACING THE FLAT 0.85 GATE

Problem: we gate on model-reported confidence. Research shows **logprob-based confidence achieves only 0.705 ROC AUC** on the DocILE benchmark, because the dominant error sources — unreadable scans, OCR noise — are invisible to the model. *A frontier LLM confidently transcribing OCR noise produces high log-probabilities for a wrong answer.*

Design — four independent signals, combined explicitly:

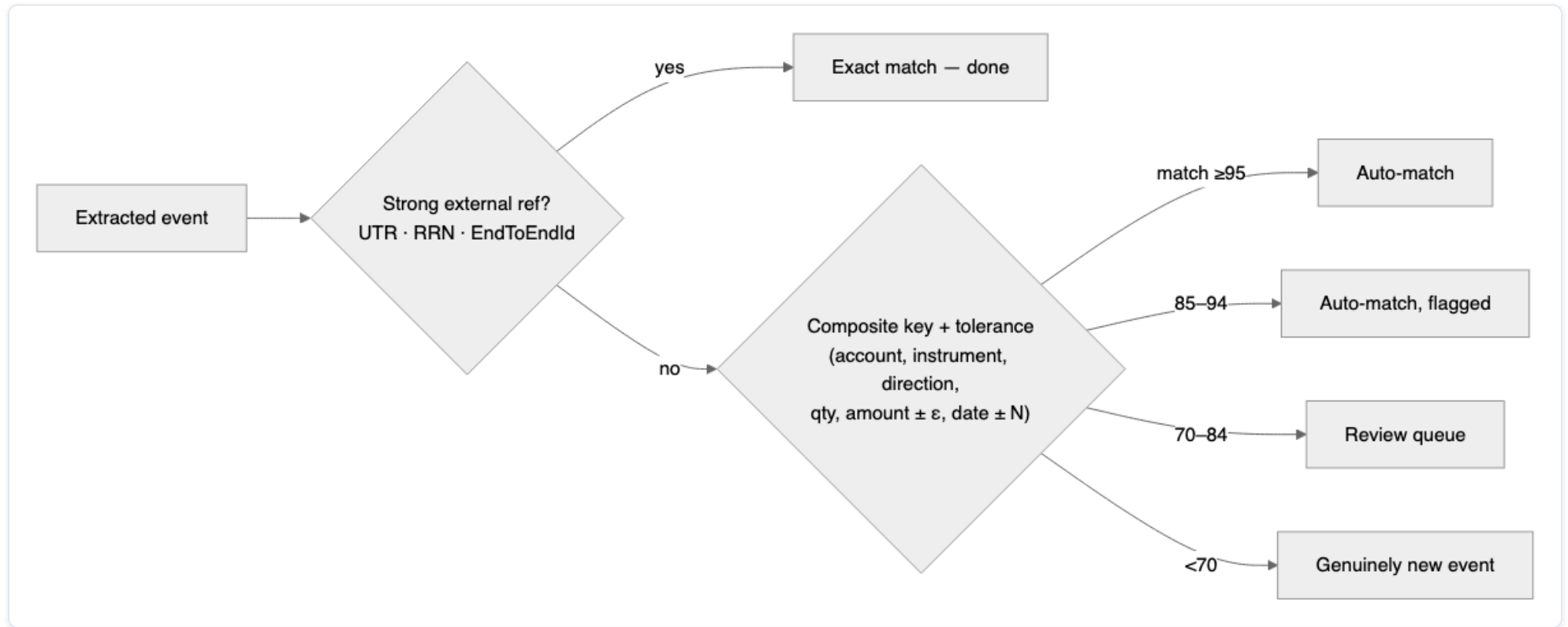
SIGNAL	MECHANISM	CATCHES
Arithmetic	Do debits + credits reconcile to the stated closing balance? Does $\text{qty} \times \text{price} + \text{charges} = \text{net amount}$?	Transcription errors, dropped rows
Parser agreement	Deterministic parser and LLM produce the same value	Model hallucination
Grounding	Every value's verbatim span is mechanically found in the source text	Fabrication — <i>"LLMs cannot cite what they haven't seen"</i>
Self-consistency	Repeated passes agree	Sampling instability

`ConfidenceVector` is stored, not collapsed to a scalar before persistence — so a later threshold change can be re-evaluated against history without re-extraction.

The threshold must be calibrated against a real sample before go-live, mapping confidence to observed error rate. **0.85 was chosen by feel and should not survive this redesign unexamined.**

3.3 IDENTITY: TWO IDENTITIES, NOT ONE

This is the central dedup gap. Content hashing **cannot** match the same trade arriving as a contract-note PDF and a broker email — the bytes differ.



Harvest UTR/RRN aggressively — it is the highest-value field on the page, because it converts fuzzy matching into exact matching and removes the tolerance-window guesswork entirely. Indian payment rails put it in many emails already.

Restatement is a link, never an overwrite. Following Plaid's `pending_transaction_id` precedent, a corrected document appends an event with `supersedes_id` pointing at the original. Plaid also documents that *"in some rare cases"* matching fails — **so the failure path is designed, not assumed away**. This directly serves "never overwrite financial records."

3.3A ECONOMIC IDENTITY — IMPLEMENTED

`ReferenceHarvester` + `EconomicIdentityResolver`, three tiers tried in order.

Reference harvesting is label-driven, never pattern-only. A bare twelve-digit number is just as likely to be an account number, a phone number, or an amount in paise. Because tier 1 treats a reference as *proof*, a false positive would merge two genuinely different transactions and **silently lose a financial record** — so a reference is harvested only where the document explicitly names it (`UTR No`, `RRN`, `UPI Ref`, ...).

References are split by strength. `UTR` / `RRN` / `UPI_TXN_ID` / `IMPS_REF` are globally unique and decisive. `CHEQUE` and `ISSUER_REF` are **not**: a cheque number repeats across accounts and two brokers can each mint order `12345`, so those are harvested and stored but never used for tier-1 matching.

TIER	BASIS	OUTCOME
1	Globally-unique rail reference matches	<code>EXACT_REFERENCE</code> , score 100
2	Composite key agrees exactly	<code>COMPOSITE_KEY</code> , score 100
2b	Composite key within tolerance (₹1.00, ±3 days)	<code>TOLERANT</code> — duplicate, and flagged
3	Scored similarity ≥70	<code>NEEDS_REVIEW</code>
—	Below 70	<code>NEW</code>

The asymmetry that sets every threshold: failing to spot a duplicate books a transaction twice, which reconciliation surfaces and a user can correct. Wrongly declaring a duplicate **discards a real transaction silently** — nothing reports it, because as far as the system is concerned nothing happened. So the tiers prefer a visible double-booking over an invisible loss.

Two rules that are gates, not weights:

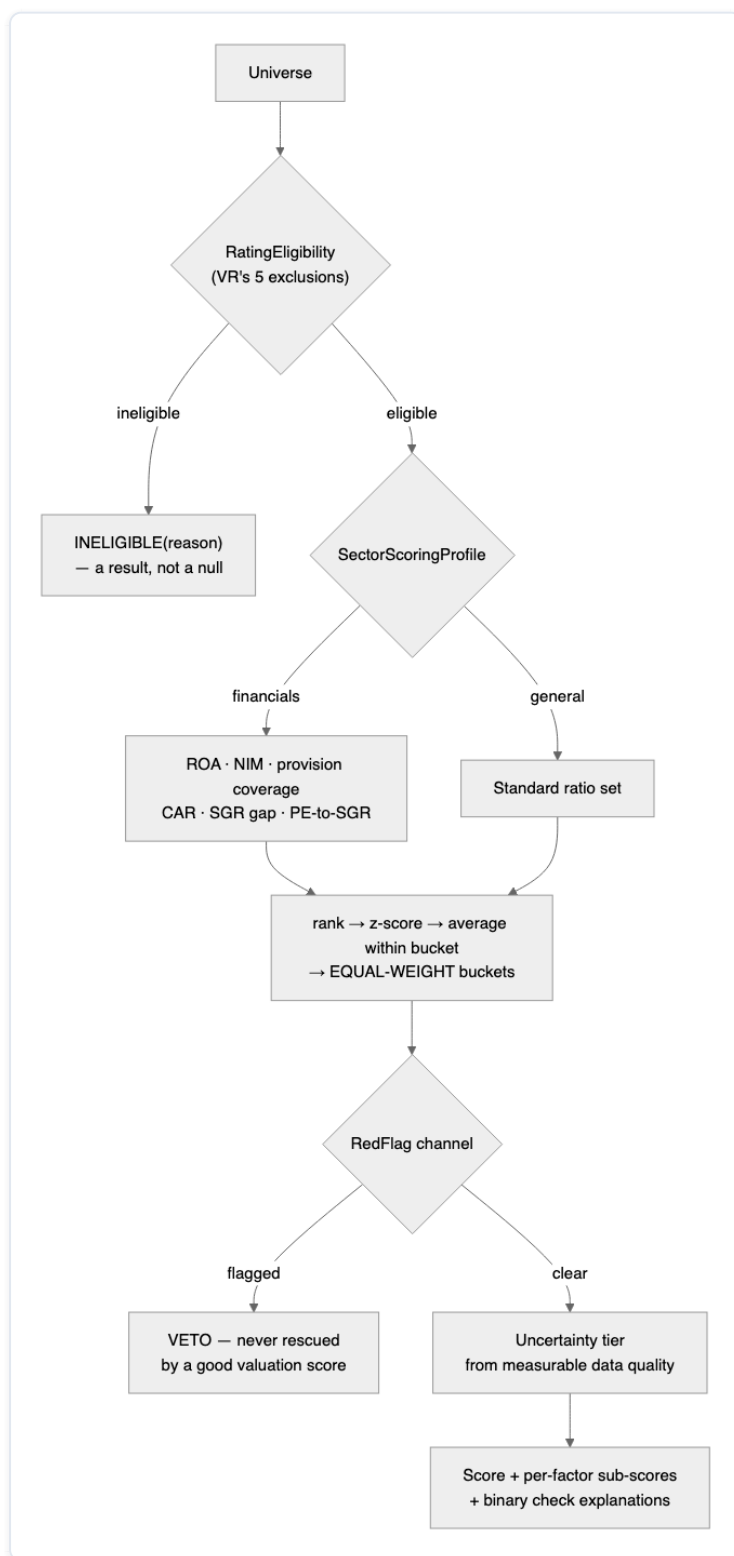
- **Direction.** A BUY and a SELL of the same quantity, amount and date are opposites, not near-duplicates. No amount of agreement elsewhere can average them together.
- **Amount-only difference.** When account, instrument, direction, quantity and date all agree and *only* the amount differs, the two likely explanations are opposites — one document includes brokerage (a bank debit against a contract note's net) and is the same event, or they are two separate trades of the same size that day. Nothing in the data distinguishes them, so this is escalated **by name** rather than decided by whichever way a weighted score falls. Sub-rupee gaps are excluded from this rule and left to the tolerant tier, since those are rounding rather than charges.

That second rule came out of a failing test, not the design. The first implementation scored the case at 55 and declared it `NEW` — i.e. it would have double-booked a charges-inclusive restatement. Tuning the threshold until the test passed would have hidden the real gap: the model had no way to express "this specific pattern is inherently ambiguous."

3.4 SCORING: REPLACING THE CURRENT COMPOSITE

Problem (verified at `AnalystService.java:142`): `tech×0.30 + mom×0.20 + val×0.20 + sent×0.30` — no quality factor, no growth factor, 30% on sentiment, and the momentum-family indicators are pairwise $\rho > 0.9$ so momentum is effectively **~4:1 over-weighted** by accident.

Design:



Four design decisions and their evidence:

DECISION	EVIDENCE
Equal-weight buckets, no optimisation	Measured on Ken French data (n=757): HML–RMW and RMW–CMA flip sign between 1963–99 and 2000–26. The covariance matrix you estimate is not the one you live with. AQR reached the same conclusion: <i>"a potential nightmare of choices and dimensionality... We choose the latter"</i>

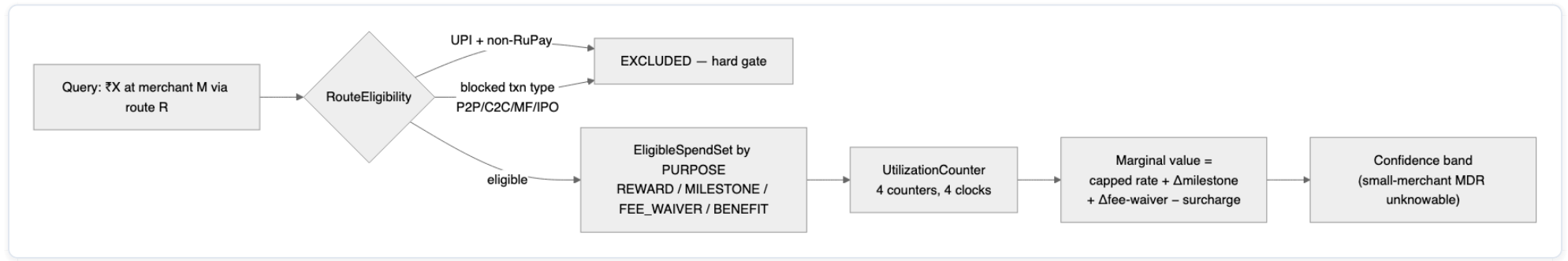
DECISION	EVIDENCE
Collapse each indicator family to one representative	Measured: HML+CMA ($p=0.68$) → Sharpe 0.41, zero gain ; HML+UMD ($p=-0.19$) → 0.67, nearly double . Correlated factors add nothing
Separate ratio set for financials	Value Research and Simply Wall St independently do this. Two sources converging makes it non-optional
Red flags veto, never contribute	Tickertape and Zerodha Nudge both treat ASM/GSM/pledge/default-probability as disqualifiers, not weighted inputs

Assign accruals exactly one slot. They appear independently in Piotroski (CFO>ROA), Beneish (TATA) and QMJ (ACC) — triple-counting them would be the same class of error as the current momentum over-weighting.

Explainability layer: continuous scores rank; **binary pass/fail checks explain.** Simply Wall St's model — 6 binary checks per axis — is auditable in a way a weighted z-score never is. "Failed 4 of these 6 named checks, here they are" *is* a reason, which is what "EVERY RECOMMENDATION MUST HAVE A REASON" actually requires.

3.5 CARD ENGINE: STATEFUL, NOT A RATE LOOKUP

Problem: `CardRewardRule.monthlyCapAmount` exists but there is **no counter entity**, zero MCC data, zero exclusion data, and no route-eligibility model.



Why marginal, not average: in December, a card whose caps are exhausted but which is ₹40k short of a ₹2L fee waiver is the right answer **at a 0% reward rate**. No static rate table can express that.

Why `EligibleSpendSet` is keyed by purpose: Kotak publishes **five different exclusion lists for five different purposes** — fee levy, reward exclusion, air-miles, milestone/fee-waiver, and a benefit-specific list — with per-card-variant carve-outs, and states the list *"is not exhaustive."* One exclusion list per card is the single most common modelling error in this domain.

Why every rule is effective-dated with `supersedes_id`: devaluations arrive with 30–60 days' notice, and our own research found the **SBI Cashback cap value disagreeing across three sources**. Backtesting requires answering "what was true last March?"

Why rewards are a band, not a number: NPCI's parity mandate exempts issuers earning no interchange, and small merchants (turnover below ~₹20 lakh) are MDR-exempt. **Whether a ₹5,000 UPI payment earns anything can depend on the merchant's turnover band — unobservable to the user and to us.** The honest output is a range with the reason stated.

3.6 NET-WORTH ATTRIBUTION

Status correction: `NetWorthSnapshot` **does** exist and stores a dated series (`snapshotDate`, `totalAssets`, `netWorth`) via `NetWorthService.record()`. What is missing is **decomposition**.

```
Δ NetWorth = net saving + revaluation
            = (contributions - withdrawals - expenses) + (market movement + income)
```

with `LedgerTransfer` ensuring internal transfers contribute **exactly zero** — already proven by an existing invariant test.

This is the largest unoccupied space found in the entire research phase: nobody in Indian retail ships it. And the reason nobody does is not the mathematics — Modified Dietz is trivial — it is **transfer-pair detection**, which this codebase already solved.

Modified Dietz for the revaluation term:

$$R = (B - A - F) / (A + \sum W_i \cdot F_i) \quad W_i = (D - d_i) / D$$

Benchmark comparison must use the user's own cash-flow schedule. INDmoney shows portfolio XIRR beside a raw Nifty index return, which is apples-to-oranges. Computing the benchmark's XIRR under the user's actual flows is both correct and rare.

4. Data acquisition topology

SOURCE	ACCESS	CONSTRAINT DISCOVERED
Dhan	Free with account	Daily history back to inception — the highest-value integration

SOURCE	ACCESS	CONSTRAINT DISCOVERED
Kite Connect	₹500/mo per app	Backup. Personal tier gives orders but no data
<code>nsearchives.nseindia.com</code>	Open	Bhavcopy, equity master, index constituents, corp actions
<code>www.nseindia.com/api/</code>	403 — Akamai, blocks datacenter IPs	Cookie-priming does not help. Residential IP only
<code>api.bseindia.com</code>	Open with UA + Referer	No datacenter block — preferred for filings
Screener.in	HTML scrape, <code>/company/<SYM>/</code> not robots-disallowed	12y fundamentals. ToS unverified (/terms/ 404s) — personal use plausibly fine, republishing is not
<code>portal.amfiindia.com</code>	Open	Moved from <code>www.</code> ; new Plan / Option columns break positional parsers
mfapi.in	Free, no key	NAV history + ISIN + category
AMC monthly portfolios	40+ inconsistent XLSX, some WAF'd	First-class subsystem: 3–6 weeks + 1–3 days/month
MF Central / AA	Closed / gated	AMFI stopped MF Central third-party sharing (Sept 2025); AA requires a regulated entity
Analyst estimates	None free	Scope out. Drop forward P/E and PEG from v1

Central PoliteFetcher with per-host rate limits, caching and a real user agent. A 403 from an AMC is a scheduling problem, not an error to retry hard.

5. Cross-cutting

5.1 DATASUFFICIENCY AS A FIRST-CLASS TYPE

Every computed number returns either a value or `Insufficient(reason, whatWouldFixIt)`. Today this exists only ad-hoc. The pattern is validated by VR's published exclusions, Zerodha's XIRR suppression, and Simply Wall St's auto-fail on under 10 years of dividend history.

5.2 EVENT SOURCING — DELIBERATELY PARTIAL

The ledger is already event-sourcing-shaped (append-only `Transaction` + replay). **It stays that way and does not spread.** Azure's guidance is explicit about the cost: projections become effectively immutable, and *"even a simple bug fix can cause a butterfly effect as it runs against every transaction since the beginning of time."* Event-source the ledger; keep CRUD elsewhere.

5.3 DEFENDING THE INVARIANT — IMPLEMENTED

`HoldingLedgerInvariantTest` fails the build if any class other than `PortfolioService` calls `Holding.setQuantity` or `setAverageCost`, checked against bytecode. Verified by injecting a violation and confirming it goes red. See `FINANCIAL_LEDGER.md` §1.1.

5.4 SECRETS AND PII

Unchanged and non-negotiable: PII sanitisation before any fallback LLM provider; no secrets in logs, git, or frontend; generated passwords use cryptographically secure randomness, are never derived from personal details, never persisted, never logged.

6. Build sequence

Ordered so that each phase is independently shippable and nothing depends on a later phase.

PHASE	CONTENTS	RATIONALE
0	<code>HoldingLedgerInvariantTest</code> · <code>DataQuality</code> + <code>Sufficiency</code> types	Done. Cheap, protective. (<code>portal.amfiindia.com</code> was already correct in <code>AmfiNavService</code> — no change needed)
1 (P0) 	<code>FinancialDocument</code> + 10-state machine · classifier cascade · grounding spans · multi-signal confidence · wiring into <code>GmailSyncService</code> still to do	Financial truth. Largest correctness surface
2 (P0) 	Economic identity resolver · UTR harvesting (tier 1 live in <code>ParsedEmailImporter</code>) · <code>supersedes_id</code> columns · reconciliation registry (+4 checks)	Dedup and reconcilability
3 (P0/ P1)	Net-worth attribution — decomposition, Modified Dietz, closure invariant, wired to <code>NetWorthSnapshot</code> and exposed at <code>GET /api/networth/attribution</code>	Biggest differentiator; dependencies already existed
4 (P1) 	Scoring — rating eligibility , red-flag veto , quality factor , binary-check explanations , weights rebalanced 30/20/20/30 → 25/15/40/20 with fail-fast sum validation ; sector profiles and quality/growth wiring blocked — no fundamentals feed	Fixes a model that was demonstrably wrong
5 (P1) 	Card marginal-value engine · UPI/RuPay route gate · utilisation counters · purpose-keyed spend types ; effective-dated rule storage outstanding	Fixes recommendations that can be wrong
6 (P1) 	Overlap calculator (Σ min, ISIN-keyed, staleness refusal) · X-Ray look-through ; per-AMC XLSX ingestion outstanding — the 3–6 week part	

PHASE	CONTENTS	RATIONALE
7 (P2) ●	Lot-level disposal calculator ✓ · FY exemption ledger ✓ · grandfathering ✓ · deferral prompt ✓; tax-aware rebalancing outstanding	

Phase 4 note: the existing `AnalystService` stays in place and serving until the replacement passes a comparison harness on real holdings. We do not swap a wrong-but-working scorer for an untested one.

Related

- `DATA_ARCHITECTURE.md` — entity design, identity model, provenance
- `FINANCIAL_LEDGER.md` — the invariant, replay semantics, attribution math
- `diagrams/` — standalone Mermaid sources
- `../research/` — evidence for every claim above
- `../01-architecture/` — the system as it exists today

Target Data Architecture

 [Source file · docs/architecture/DATA_ARCHITECTURE.md](#)

Target-state data design. See [../01-architecture/DATABASE_ARCHITECTURE.md](#) for the current schema (28 domains, entity inventory, index summary, the `Long userId` vs `@ManyToOne User` inconsistency recorded as ADR-9).

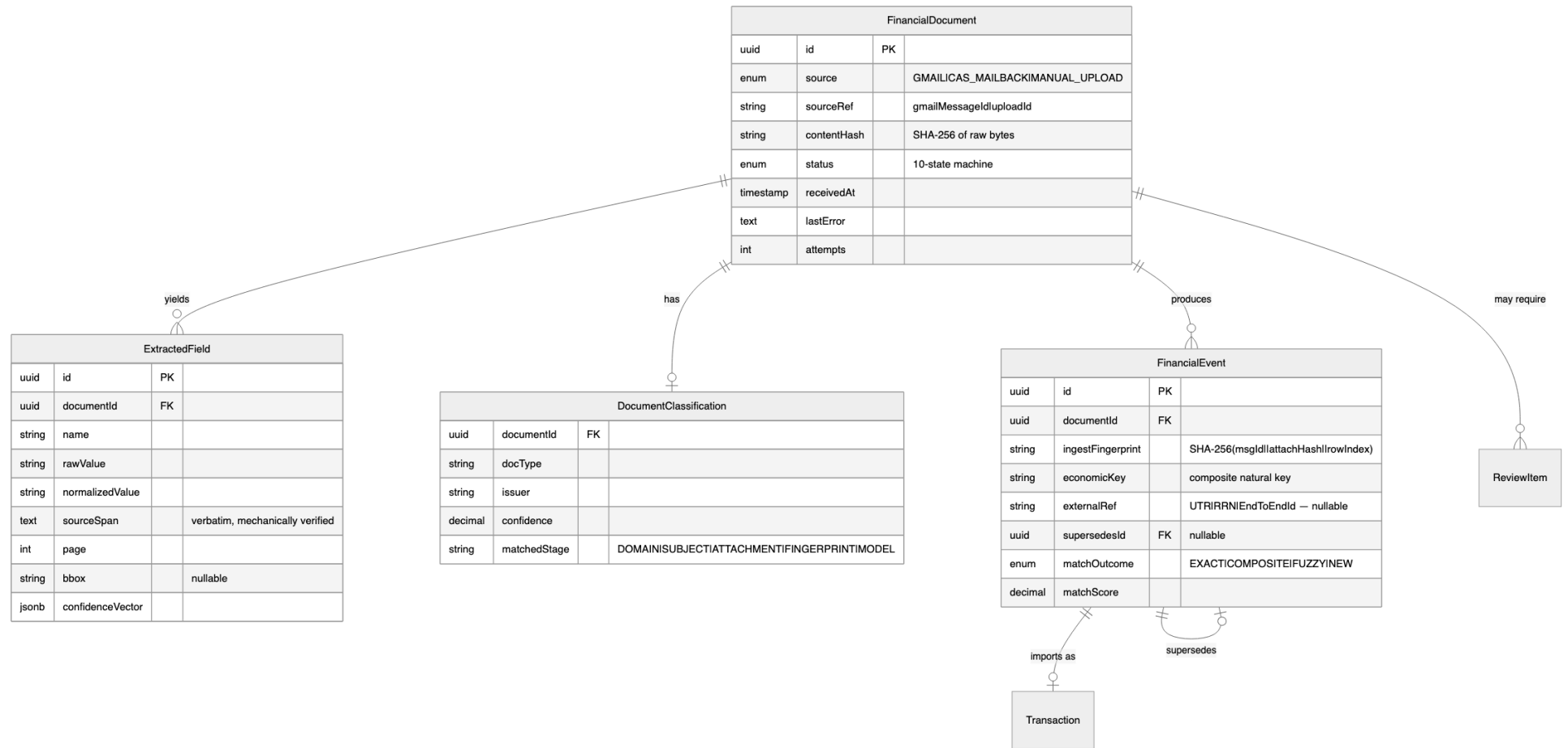
1. The three data tiers

Data in this system falls into exactly three tiers, and **the tier determines the mutation rules**. Most data bugs in financial software come from treating one tier like another.

TIER	MUTABILITY	EXAMPLES	RULE
Observed	Append-only, never edited	<code>FinancialDocument</code> , <code>ExtractedField</code> , <code>Transaction</code>	A correction is a new row linked to the old one , never an update
Derived	Recomputable, freely overwritten	<code>Holding.quantity</code> , <code>Holding.averageCost</code> , net worth, scores	Written by exactly one function; deleting and recomputing must produce identical output
Reference	Versioned with effective dates	<code>CardRewardRule</code> , <code>Cap</code> , <code>Milestone</code> , NAV, prices, tax rates	Every row carries <code>effective_from</code> / <code>effective_to</code> and provenance

The test for tier confusion: if you can delete a row and regenerate it exactly from other data, it's Derived. If you can't, it's Observed and must never be edited in place. `Holding` is Derived. `Transaction` is Observed. Conflating them is how ledgers drift.

2. Document Intelligence entities



2.1 WHY SOURCESPAN IS NOT NULL

This one column is what converts *"every financial number must be traceable"* from an aspiration into a mechanically enforced property. Extraction **fails** if the span cannot be located in the source text — which is also the cheapest hallucination defence available, because *"LLMs cannot cite what they haven't seen, regardless of plausibility."*

The alternative — email-level provenance, which is what exists today — answers "which email did this come from" but not "which words in it said ₹47,300."

2.2 WHY CONFIDENCEVECTOR IS JSONB, NOT A SCALAR

Storing the four signals separately means a threshold change can be re-evaluated against historical documents **without re-extraction**. Collapsing to a scalar at write time throws away the ability to answer "would the new rule have caught this?"

```
{
  "arithmetic": { "passed": true, "checks": ["debit_credit_balance", "qty_price_net"] },
  "parserAgreement": { "score": 1.0, "parser": "ZerodhaParser", "disagreements": [] },
  "grounding": { "score": 1.0, "unfoundSpans": [] },
  "selfConsistency": { "score": 0.94, "passes": 3 },
  "modelReported": 0.91
}
```

`modelReported` is **recorded but not trusted** — DocILE puts logprob-based confidence at only 0.705 ROC AUC.

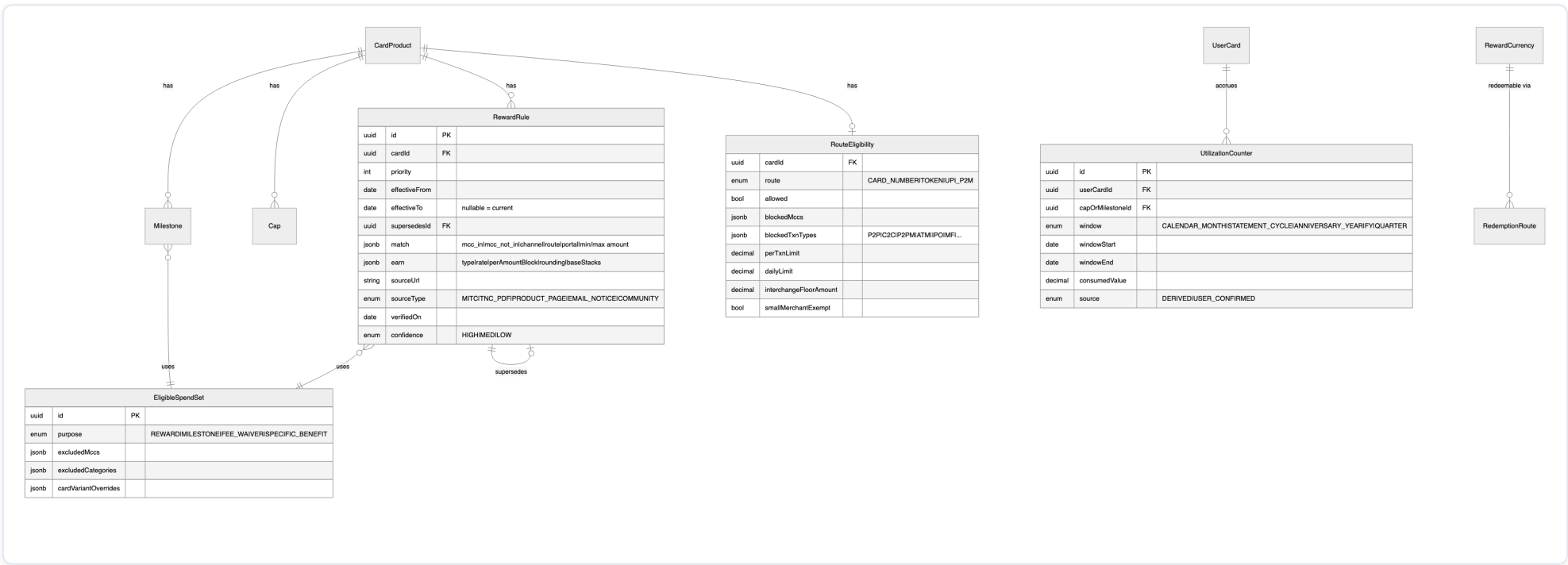
2.3 TWO FINGERPRINTS, TWO JOBS

COLUMN	FORMULA	ANSWERS
<code>ingestFingerprint</code>	<code>SHA-256(gmailMessageId attachmentHash rowIndex)</code>	"Have I processed this <i>artifact</i> before?"
<code>economicKey</code>	<code>(account, instrument, direction, quantity, amount, date) normalised</code>	"Have I recorded this <i>event</i> before, from any source?"

The existing `TransactionFingerprinter` already implements the economic side correctly — it hashes financial content and **deliberately excludes the Gmail message ID**, so a forwarded email collides with the original. That property is preserved. What's added is the document-layer fingerprint with row granularity, so a 40-row CAS statement doesn't collapse to one hash.

3. Reference data: effective dating

Every reference row is versioned. This is not optional for card rules — devaluations arrive with 30–60 days' notice, and our research found the **SBI Cashback cap disagreeing across three sources**, which is precisely the situation versioning exists to survive.



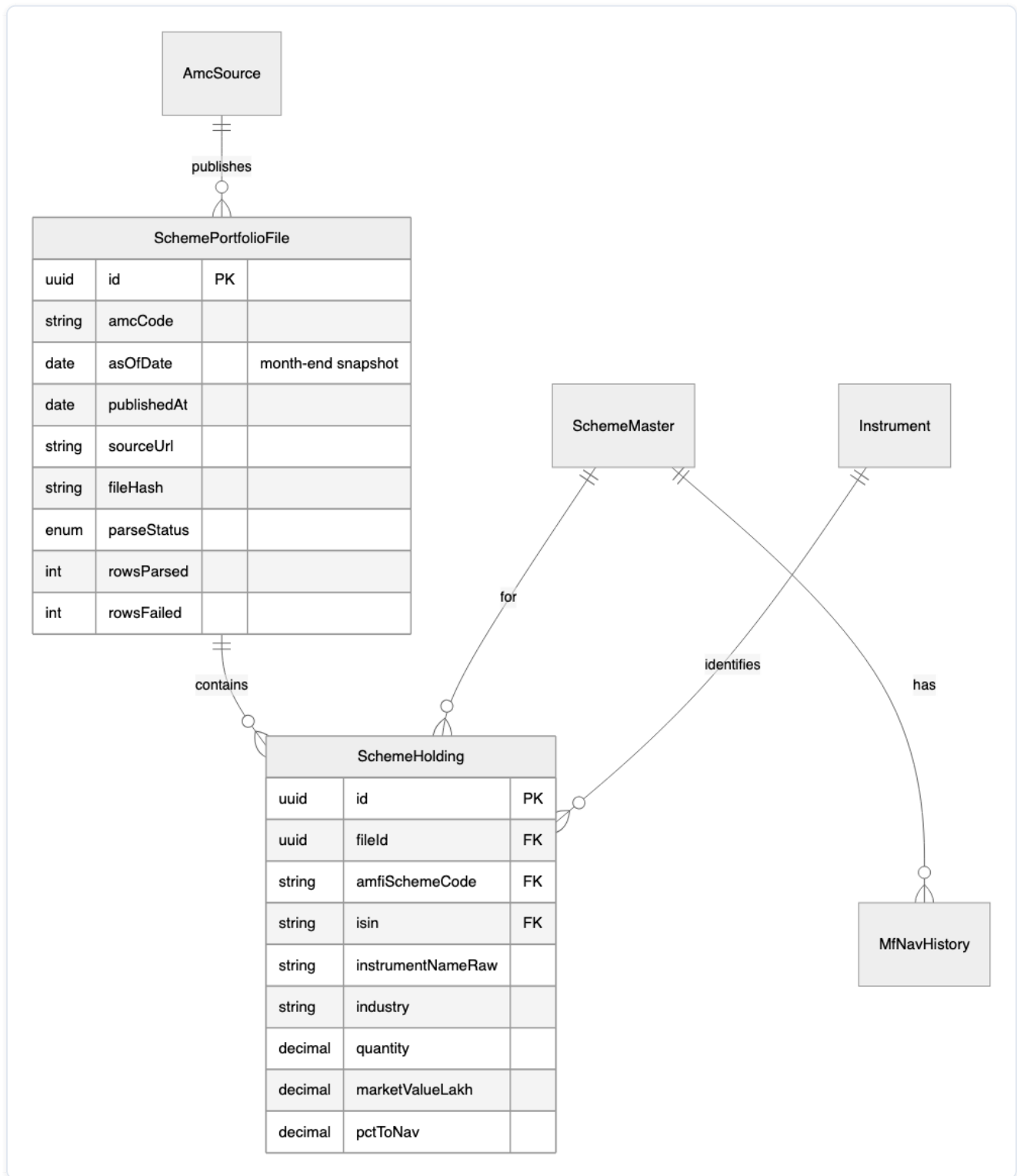
Three non-obvious requirements, each traceable to a specific finding:

1. **EligibleSpendSet.purpose** — Kotak maintains *five* lists for five purposes with per-variant carve-outs, and says the list "is not exhaustive." One exclusion list per card is the most common modelling error in this domain.
2. **window distinguishes CALENDAR_MONTH from STATEMENT_CYCLE** — SBI uses the statement cycle, HDFC uses the calendar month. Treating them as one is a silent off-by-days bug on every cap.
3. **confidence + sourceType on every rule** — where issuers don't publish MCC maps we are *inferring*. FinArt chose category-level modelling openly for this reason; we should be equally explicit rather than presenting an inference as a fact.

smallMerchantExempt drives a band, not a number. NPCI's reward-parity mandate exempts issuers earning no interchange, and merchants below ~₹20 lakh turnover are MDR-exempt — so whether a payment earns anything can depend on a merchant attribute the user cannot observe.

4. Mutual fund holdings

The subsystem that unlocks overlap, look-through and concentration. **ISIN is the join key.**



Four properties this schema enforces:

- **asOfDate is stored and surfaced.** Portfolios are month-end snapshots published up to 10 days later, and Advisorkhoj was observed showing a **~2.5-month-old** portfolio. The as-of date is not metadata, it's part of the answer.
- **rowsFailed is recorded, not swallowed.** The AMFI NAV outage in this codebase was caused by every row dying inside `catch (Exception ignored)` while the feed appeared healthy. Partial parse failures must be visible.

- **pctToNav** is stored raw. Weights are % of NAV, so cash and debt sit in the denominator — two funds each holding 8% cash mechanically cap overlap at 92%. Renormalisation is a presentation choice made downstream, not baked into storage.
- **quantity** is stored alongside the weight, enabling re-pricing at today's price rather than applying a stale month-end weight to a current fund value.

Overlap, computed not stored:

$$\text{Overlap}(A,B) = \sum \min(w_{A,i}, w_{B,i}) \quad \text{for } i \in (\text{holdings}_A \cap \text{holdings}_B)$$

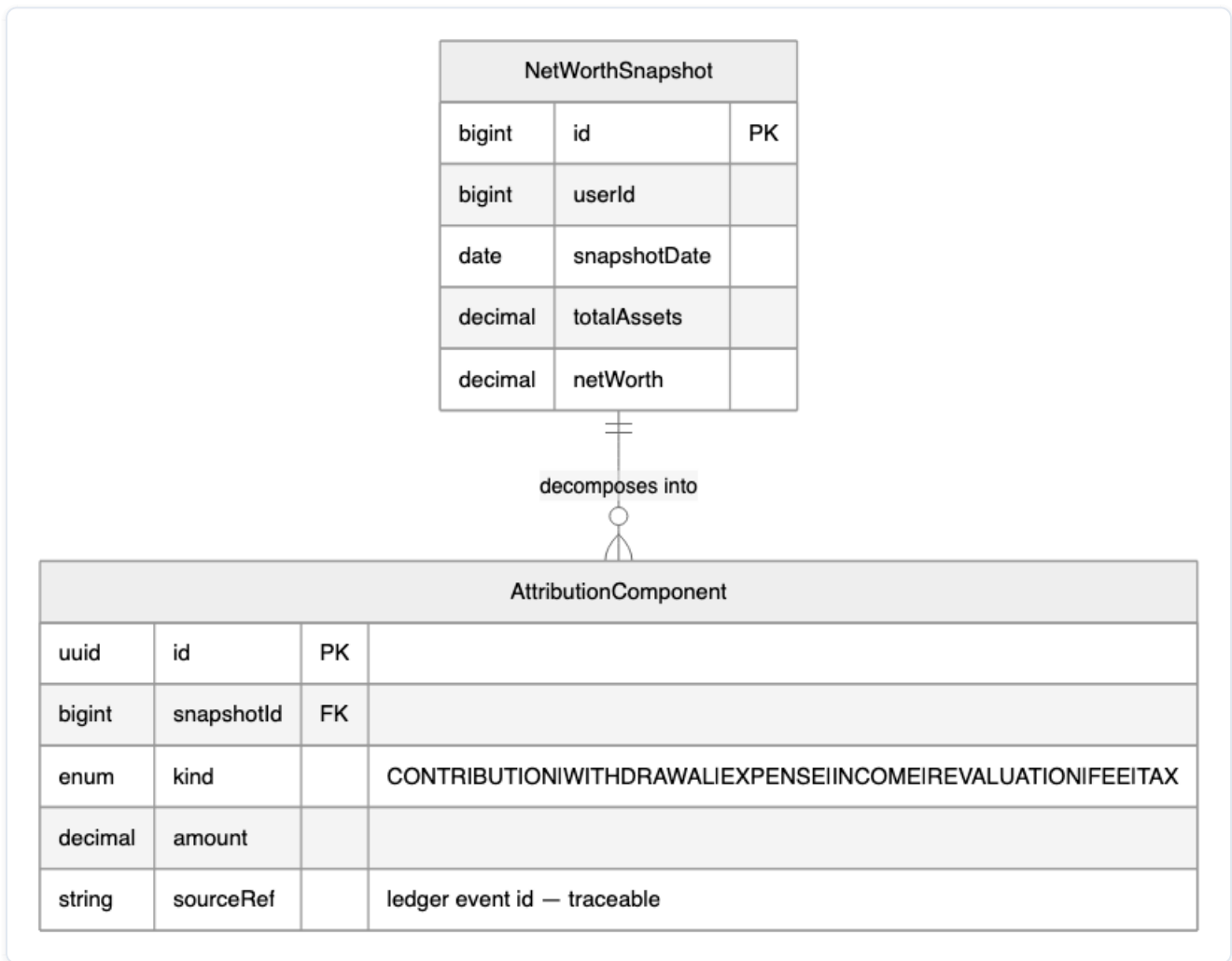
Publish the formula on the surface. Advisorkhoj shows **counts** with no published methodology, so our number will legitimately differ from theirs and users will ask why.

Look-through, the Morningstar X-Ray model:

$$\text{EffectiveExposure}(S) = \text{DirectValue}(S) + \sum_f (\text{Value}(f) \times w_{f,S})$$

Refuse to compute when the underlying portfolio is older than ~60 days — a fund that has exited a stock would still show exposure.

5. Net-worth attribution



NetWorthSnapshot **already exists** with the first four columns. **AttributionComponent** is the addition, and every component carries **sourceRef** back to a ledger event — so "*₹B came from market movement*" is itself traceable rather than a residual plug.

The invariant that must hold:

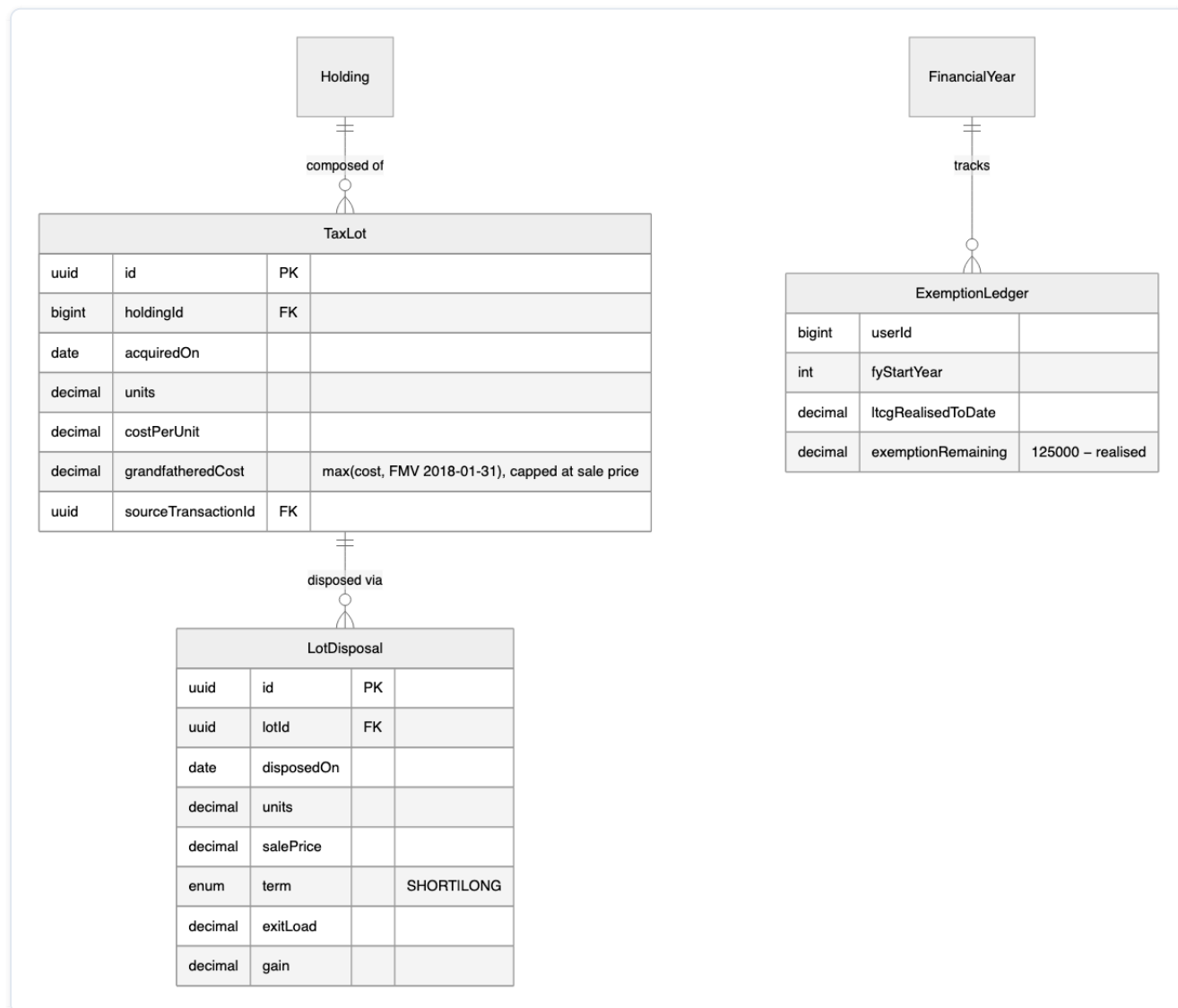
$$\text{netWorth}(T) - \text{netWorth}(T-1) == \sum \text{AttributionComponent.amount}$$

exactly, to the paisa. **LedgerTransfer** guarantees internal transfers contribute zero — already proven by an existing test showing a ₹50k bank→MF transfer produces ₹0 net-worth shift.

Design note: revaluation must be *computed*, not left as the residual. A residual absorbs every bug in every other component silently. Compute it from price deltas on held quantities, then assert the identity closes — and if it doesn't, that's a reconciliation exception, not a rounding adjustment.

6. Tax: lot-level

`TaxService` today aggregates `Income` rows by source string and returns an honest estimate *range*. It is **already on the correct rates** — 12.5% LTCG above ₹1,25,000 and 20% STCG, applied to the FY aggregate. **Those constants are right; do not "fix" them.** What's missing is lot granularity.



ExemptionLedger is the single most important table here. The ₹1,25,000 exemption is a **per-financial-year aggregate across all s.112A gains** — listed equity, equity MF, business trust units — **not per transaction, not per scheme, not per folio**. A per-redemption calculation is simply wrong, and it is the most commonly mis-implemented rule in Indian retail finance. Kuvera's own tax-harvesting page still displays the stale ₹1 lakh figure.

Store section references as data, not literals. The Income-tax Act 2025 replaces the 1961 Act from 1 April 2026: **111A → 196, 112A → 198**. Rates carry over unchanged, but hard-coded section strings in user-facing copy go stale on that date.

Also required for a correct final number: surcharge (capped at 15% for s.112A) plus 4% health and education cess. A bare "12.5%" is wrong for high-income users.

7. Provenance, applied uniformly

Every table whose rows originate outside the user gets the same five columns:

COLUMN	PURPOSE
<code>sourceUrl</code>	Where it came from
<code>sourceType</code>	<code>MITC</code> / <code>TNC_PDF</code> / <code>PRODUCT_PAGE</code> / <code>EMAIL_NOTICE</code> / <code>COMMUNITY</code> / <code>REGULATORY</code>
<code>verifiedOn</code>	When a human last confirmed it
<code>confidence</code>	<code>HIGH</code> / <code>MED</code> / <code>LOW</code> — inference is labelled as inference
<code>supersedesId</code>	The row this replaced

This is what makes a **public, dated changelog** possible — which our research found to be a real gap: TechnoFino's forum is the only structured change-tracking in the Indian card market.

8. Migration posture

PRINCIPLE	CONSEQUENCE
Additive first	New tables and nullable columns. No destructive schema change while the old path still serves
Backfill is a job, not a migration	Grounding spans can't be retrofitted to already-imported emails; the column is nullable for historical rows and <code>NOT NULL</code> going forward
Existing entities keep their identity	<code>Holding</code> , <code>Transaction</code> , <code>CashAccount</code> , <code>LedgerTransfer</code> , <code>EmailReviewItem</code> , <code>NetWorthSnapshot</code> are unchanged. <code>FinancialDocument</code> wraps the ingestion path rather than replacing it
<code>PendingPdf</code> becomes a projection	Not dropped. It keeps working while <code>FinancialDocument</code> takes over
ADR-9 stays open	The <code>Long userId</code> vs <code>@ManyToOne User</code> inconsistency is recorded and deliberately not bundled into this work

Related

- `SYSTEM_ARCHITECTURE.md` — layers, flow, build sequence
- `FINANCIAL_LEDGER.md` — the invariant and replay semantics
- `../research/TECHNICAL_GAPS.md` — evidence for each change
- `../01-architecture/DATABASE_ARCHITECTURE.md` — current schema

The Financial Ledger

 Source file · docs/architecture/FINANCIAL_LEDGER.md

Unlike the other documents in this directory, this one is **mostly descriptive**. The ledger is the part of Wealth OS that is already right. It is documented here because the target architecture depends on it staying right, and because the properties below are easy to break accidentally.

Sections marked **TARGET** are additions. Everything else describes verified current behaviour.

1. The invariant

`Holding.quantity` and `Holding.averageCost` are derived, never asserted.

Verified: both are written at exactly one place in the codebase — `PortfolioService.java:258–259`, inside the private method `recomputeFromLedger`. A grep for `setQuantity()` / `setAverageCost()` across the whole backend returns other hits only on request DTOs and on `ActionItem`, never on `Holding`.

The method's own comment records why it exists:

"The single place quantity/averageCost is ever derived — always by replaying the full BUY/SELL transaction ledger for this holding, never by hand-rolling weighted-average math inline (that pattern used to be duplicated across addHolding/sellHolding/merge, and could silently drift from what the ledger actually implies)."

This is the property that makes manual entry and Gmail import structurally incapable of disagreeing about a holding. Both paths call the same replay.

1.1 IMPLEMENTED — DEFENDED BY A TEST

`HoldingLedgerInvariantTest` asserts at the **bytecode** level that no type other than `PortfolioService` calls `Holding.setQuantity` or `Holding.setAverageCost`.

Bytecode rather than source grepping is deliberate: `setQuantity` is also declared on `ActionItem` and on several request DTOs, so matching the method name in source text produces false positives. ArchUnit resolves the receiver's actual type.

The test was verified by **injecting a deliberate violation** into an unrelated service and confirming both rules failed with their guidance message, then reverting. A green architecture test that cannot go red is worse than no test.

Its failure message points at the fix rather than the symptom:

Holding.quantity is derived from the transaction ledger by PortfolioService.recomputeFromLedger. Setting it directly makes stored state disagree with the ledger that is supposed to explain it. Record a Transaction instead and let the replay derive the quantity.

If it fails, the fix is almost never to add the offending class to an exception list.

2. Replay semantics

Weighted-average cost, not FIFO or LIFO.

```
netQty  = 0
totalCost = 0

for each transaction, ordered by transactionDate ascending:

    BUY:
        totalCost += price × quantity
        netQty    += quantity

    SELL:
        sellQty    = min(transaction.quantity, netQty)
        avgCostAtSale = totalCost / netQty    (4 dp, HALF_UP)
        totalCost  -= avgCostAtSale × sellQty
        netQty     -= sellQty
        clamp both at zero

if netQty <= 0 : delete the holding, return null
else          : quantity = netQty
               averageCost = totalCost / netQty    (2 dp, HALF_UP)
```

Four behaviours worth naming explicitly, because each is a decision:

BEHAVIOUR	CONSEQUENCE
Oversell is clamped, not rejected	<code>sellQty = min(txn.quantity, netQty)</code> ; negative quantities can never be persisted. A sell with no matching holding is booked as <code>Income(CAPITAL_GAIN)</code> by <code>ParsedEmailImporter</code> , not silently dropped
Full sale deletes the holding	A zero-quantity holding is not a valid state
Realised P&L is not stored	It is derived at read time from the average cost <i>as of that sale</i> during replay. There is no "profit" column to drift

BEHAVIOUR	CONSEQUENCE
Replay is total, not incremental	Every transaction for the holding is re-read on every change. $O(n)$ per write, and correct by construction

2.1 WHY WEIGHTED-AVERAGE AND NOT FIFO

Weighted-average is what the replay implements today, and it is internally consistent. **But it is not what Indian capital-gains tax requires.**

TARGET — this is a real divergence to resolve, not an oversight to paper over. Indian tax computation needs **lot-level** identification: acquisition date per lot, LT/ST classification per lot, grandfathering to `max(cost, FMV 2018-01-31)` per lot, and exit-load windows per lot. Weighted-average cost cannot express any of that.

Proposed resolution: keep weighted-average as the *portfolio* cost basis (it is the right model for "what did this position cost me on average") and add `TaxLot` as a **parallel derivation from the same ledger** for tax purposes. Two projections, one source of truth — not two sources of truth.

Both must replay from the same `Transaction` rows, and a test should assert that `Σ TaxLot.units == Holding.quantity` at all times.

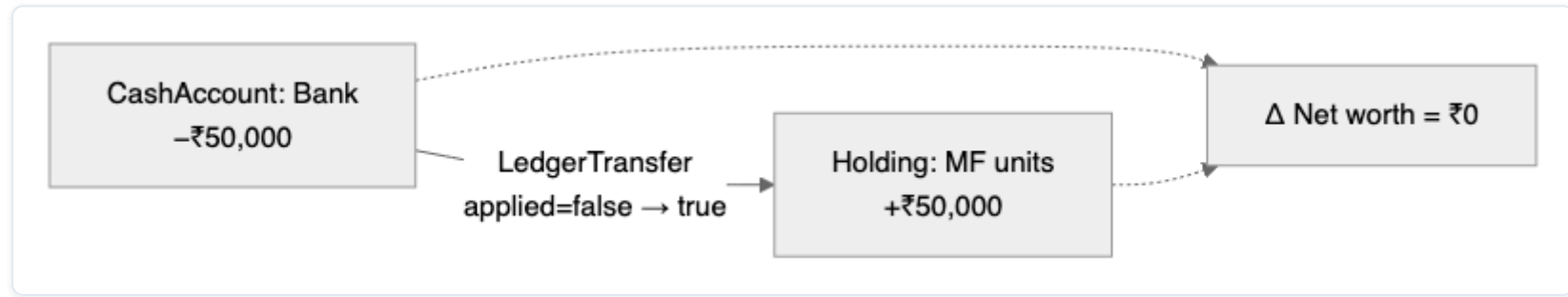
3. Net-worth neutrality of transfers

The problem this solves, found during an earlier audit: **net worth had no cash component at all**, so a ₹50,000 bank→MF transfer inflated net worth by the full ₹50,000 — the money appeared in the MF holding and never left the bank.

`LedgerTransferService.applyCashEffect` performs an equal-and-opposite mutation guarded by an `applied` flag. Its comment states the guarantee:

"...so replaying or re-saving a transfer can never move the money twice."

An invariant test proves a transfer produces **₹0** net-worth shift.



This is also the thing that makes net-worth attribution possible. Research found that nobody in Indian retail ships an attribution waterfall, and the blocker is not the maths — Modified Dietz is trivial — it is **transfer-pair detection**. A system that cannot tell "moved ₹50k" from "earned ₹50k" cannot produce an honest attribution. This codebase already solved that.

4. Single net-worth formula

`PortfolioContextService.build()` → `/api/wealth/summary` is the **only** net-worth computation. An earlier audit found six duplicate implementations that could disagree; they were collapsed into this one.

`NetWorthSnapshot` stores a dated series (`snapshotDate` , `totalAssets` , `netWorth`) written by `NetWorthService.record()`.

Net worth is computed fresh on request, never cached from an import event. A Gmail import does not trigger a "recompute net worth" step — the next `/api/wealth/summary` call simply sees the new holding.

5. TARGET — attribution

```
Δ NetWorth(T-1 → T) = net saving + revaluation

net saving = contributions - withdrawals - expenses + income
revaluation = Σ over held instruments of (price_T - price_{T-1}) × quantity_held
```

Modified Dietz for the period return:

$$R = (B - A - F) / (A + \sum W_i \cdot F_i) \quad W_i = (D - d_i) / D$$

where A = beginning value, B = ending value, F = net external flows, `di` = day of flow i, D = days in period.

Three rules that keep this honest:

1. **Revaluation is computed, not residual.** A residual absorbs every bug in every other component silently. Compute it from price deltas, then assert `ΔNetWorth == Σ components` to the paisa. **A failure to close is a reconciliation exception, not a rounding adjustment.**
2. **Every component carries `sourceRef`** back to a ledger event. "₹B came from market movement" must itself be traceable.
3. **Transfers contribute zero by construction** — guaranteed by §3, not by a filter that could miss a case.

5.1 TARGET — RETURN METRIC SELECTION

Zerodha's own product contradicts itself here: Console's *portfolio* XIRR covers current holdings only and "does not take into account historical buy and sell trades," while its *equity holdings* XIRR page covers all trades since FY2017. Two surfaces, two answers.

We have one ledger, so we have no excuse. **One definition, stated on the surface**, with the metric selected by horizon:

HORIZON	SINGLE FLOW	MULTIPLE FLOWS
< 1 year	Absolute	Absolute
≥ 1 year	CAGR ($\equiv$ XIRR)	XIRR

And **refuse** rather than mislead — Zerodha shows "-" when most holdings are under a year, because *"the XIRR would be greater than the actual percentage."*

Four XIRR pitfalls to engineer against:

- **Switches** must be modelled as paired outflow/inflow on the same date, or they double-count
- **IDCW payouts** are outflows in the flow vector; growth-plan NAV needs no adjustment
- **Partial redemptions** are outflows with the residual valued at the terminal date
- **SIP + STP chains** produce sign-alternating flows where XIRR can have **multiple roots** — a bracketed solver must detect this and decline, not return the first root it finds

Benchmark comparison must use the user's own cash-flow schedule. Comparing portfolio XIRR to a raw index return (as INDmoney does) is apples-to-oranges. Computing the benchmark's XIRR under the user's actual flows is correct and, per our research, rare in Indian retail.

6. Reconciliation

`HoldingReconciliationScheduler` runs daily (60s after startup) → `reconcileAllUsers()`: for every holding, capture stored values, `recomputeFromLedger`, compare, and `WARN` if they differed. A silent correction plus a log line is the only signal today.

`ReconciliationService.checkAll` covers **3 of ~17** checks specified.

6.1 TARGET — EXCEPTION MODEL

Replace binary pass/fail with the industry vocabulary:

OUTCOME	MEANING
matched	Ledger and source agree
unmatched	Present on one side only
partially-matched	One deposit covering several ledger entries
exception	Typed by cause: <code>TIMING_DIFFERENCE</code> / <code>AMOUNT_VARIANCE</code> / <code>UNIDENTIFIED</code>

Every exception carries the source rows needed to investigate it. And adopt the distinction between **matching** (pre-commit, blocking) and **reconciliation** (post-commit, comparative) — they are different controls that today are conflated.

Drift should not only be corrected silently. If stored state disagreed with the ledger, that is a fact worth surfacing to the user in Data Health, not just to a log file nobody reads.

7. Properties to preserve

Any future change must keep all seven true. This list is the regression contract.

#	PROPERTY	ENFORCED BY
1	<code>Holding.quantity</code> / <code>averageCost</code> have exactly one writer	<code>HoldingLedgerInvariantTest</code> (ArchUnit, bytecode)
2	Manual entry and Gmail import cannot disagree	Both call <code>recomputeFromLedger</code>
3	A holding can never hold negative quantity	<code>min(sellQty, netQty)</code> clamp
4	A sell with no holding is recorded, not dropped	<code>Income(CAPITAL_GAIN)</code> fallback
5	Internal transfers shift net worth by exactly ₹0	<code>applyCashEffect</code> + <code>applied</code> flag + invariant test
6	A forwarded duplicate email cannot double-book	<code>TransactionFingerprinter</code> excludes the Gmail message ID
7	Net worth has exactly one formula	<code>PortfolioContextService.build()</code>

Properties 1, 5 and 6 each fixed a real bug that had shipped. They are not theoretical.

8. What must not change

- **The ledger stays append-only.** Corrections are new rows; the existing manual-correction endpoint is explicitly temporary until the next replay overwrites it, and the documented fix is to correct the underlying `Transaction`, not the `Holding` snapshot.
- **Event sourcing stays confined to the ledger.** It does not spread to the rest of the system. Azure's guidance is explicit about why: projection code becomes effectively immutable, and *"even a simple bug fix can cause a butterfly effect as it runs against every transaction since the beginning of time."*
- **Derived state is never the source of a decision without recomputation.** If a score, a net worth, or a tax figure is stale, recompute it — do not read a cached row and act on it.

Related

- `SYSTEM_ARCHITECTURE.md` — layers and build sequence
- `DATA_ARCHITECTURE.md` — `TaxLot`, `AttributionComponent`, identity model
- `../01-architecture/ARCHITECTURE_DECISIONS.md` — ADR-1 through ADR-9
- `../05-flows/TRANSACTION_PROCESSING.md` — the flow trace

04

Backend internals

The layers, the engine, the ingestion pipeline, the jobs, the API surface.

01 Backend Structure

02 Financial Engine

03 Gmail Ingestion Pipeline

04 Jobs & Schedulers

05 API Specification

Backend Structure

 Source file · docs/02-backend/BACKEND_STRUCTURE.md

Package root: `com.marketai` . 30 `@RestController` classes found under `**/controller/` .

Standard layered pattern (representative: `portfolio` domain)

```
Controller  src/main/java/com/marketai/portfolio/controller/PortfolioController.java
Service      src/main/java/com/marketai/portfolio/service/PortfolioService.java  (@Service,
@RequiredArgsConstructor, @Slf4j – concrete class, no interface)
Repository  src/main/java/com/marketai/portfolio/repository/
{PortfolioRepository,HoldingRepository,TransactionRepository}.java
Entity       src/main/java/com/marketai/portfolio/entity/{Portfolio,Holding,Transaction}.java
DTO         src/main/java/com/marketai/portfolio/dto/
{AddHoldingRequest,PortfolioSummaryDto,TransactionDto,IntegrityReportDto,MergeSummaryDto}.java
Extras       scheduler/HoldingReconciliationScheduler.java, util/XirrCalculator.java
```

Note: controllers here are not strictly service-only — `PortfolioController` also injects `HoldingRepository` / `TransactionRepository` directly for a few read-only endpoints, and reaches into `income.repository.IncomeRepository` (a different domain's repository) as a collaborator passed into `portfolioService.sellHolding(...)` . Entities use Lombok directly with JPA annotations — no separate mapper/converter layer (DTOs are hand-built in controller/service code).

Domain package map (30 controllers + supporting-only domains)

DOMAIN	HAS CONTROLLER?	HAS ENTITY/REPOSITORY?	NOTES
<code>auth</code>	✓	✓	Extra <code>security/</code> package (<code>JwtAuthFilter</code> , <code>JwtService</code>)
<code>ai</code>	✓ (<code>/api/ai</code>)	✓ (<code>AiHistory</code>)	Extra <code>audit/</code> , <code>client/</code> , <code>intel/</code> , <code>llm/</code> packages; nested <code>ai.review</code> sub-domain has its own full stack + a <i>second</i> REST surface (<code>/api/review</code>)
<code>amfi</code>	✓	—	Read-only: AMFI NAV lookup, cached in-memory
<code>analyst</code>	✓	—	Routes through <code>RecommendationEngine</code> , no owned entity
<code>actions</code>	✓ (2 controllers)	✓ (<code>ActionItem</code>)	<code>ActionItemController</code> + <code>TodaysActionsController</code>

DOMAIN	HAS CONTROLLER?	HAS ENTITY/REPOSITORY?	NOTES
<code>card</code>	✓	✓	Extra <code>optimizer/</code> package (<code>CardOptimizerService</code> , <code>SpendAggregator</code>)
<code>expense</code>	✓	✓	
<code>forecast</code>	✓	—	Derived/computed, no owned persistence
<code>gmail</code>	✓ (2 controllers)	✓	Extra <code>ai/</code> , <code>parser/</code> , <code>scheduler/</code> , <code>security/</code> packages; <code>GmailController</code> + <code>GmailPushController</code>
<code>goal</code>	✓ (<code>/api/goals</code>)	✓ (<code>FinancialGoal</code>)	
<code>income</code>	✓	✓	
<code>ledger</code>	✓	✓	Request DTOs (<code>CashAccountRequest</code> , <code>TransferRequest</code>) are nested static classes inside the controller rather than in <code>dto/</code> — inconsistent with the rest of the app
<code>market</code>	✓	✓	Extra <code>client/</code> package (Yahoo Finance client)
<code>mf</code>	—	✓ (<code>MfNavHistory</code>)	No controller at all — pure internal/support domain consumed by other services
<code>networth</code>	✓	✓ (<code>NetWorthSnapshot</code>)	
<code>news</code>	✓	✓	Extra <code>client/</code> package
<code>planning</code>	(folder absent — reminders/goals/tax live in separate top- level domains, not a <code>planning</code> package)		
<code>portfolio</code>	✓	✓	The representative example above
<code>recommendation</code>	✓ (2 controllers)	—	<code>RecommendationController</code> + <code>WealthController</code> (<code>/api/</code> <code>wealth</code>)
<code>reconciliation</code>	✓	—	Cross-domain check, distinct from <code>gmail</code> 's own <code>/api/gmail/</code> <code>reconciliation-report</code>

DOMAIN	HAS CONTROLLER?	HAS ENTITY/REPOSITORY?	NOTES
redemption	✓	✓	
reminder	✓	—	[PARTIALLY IMPLEMENTED] — read-only, only GET /api/reminders exists
scheduled	✓ (/api/recurring-investments)	✓ (RecurringInvestment)	
signal	✓	—	SignalEngine computes on the fly, no owned entity
sync	✓	✓ (SyncJob)	SyncJobDto is a nested class inside the controller, not in dto/
tax	✓	—	
technical	✓	—	
tracking	✓	✓	FD/RD/Loan/EPF/OtherAsset — all sub-resources under one controller

Global exception handling

GlobalExceptionHandler (common/exception/) — @RestControllerAdvice :

EXCEPTION	STATUS	MESSAGE BEHAVIOR
ResourceNotFoundException	404	
DuplicateResourceException	409	
BadCredentialsException	401	Hardcoded "Invalid email or password" — never leaks ex.getMessage()
AccessDeniedException	403	Generic "Access denied"
MethodArgumentNotValidException	400	Includes a validationErrors field→message map
ExternalApiException	503	ERROR-logged
org.apache.catalina.connector.ClientAbortException	(no body)	DEBUG-logged only (client disconnected)

EXCEPTION	STATUS	MESSAGE BEHAVIOR
Generic <code>Exception</code>	500	"An unexpected error occurred", ERROR-logged

[PARTIALLY IMPLEMENTED]: the generic handler also contains a redundant string-based `ex.getClass().getName().contains("ClientAbort")` check duplicating the dedicated `ClientAbortException` handler above it — a minor fragility worth cleaning up.

Security configuration

See `01-architecture/ARCHITECTURE_OVERVIEW.md §"Security model"` for the full `permitAll` list and filter chain. Summary: stateless JWT, `BCryptPasswordEncoder(12)`, `JwtAuthFilter` before `UsernamePasswordAuthenticationFilter`, no method-level `@PreAuthorize` usage anywhere in the codebase (`@EnableMethodSecurity` is declared but unused).

Naming/registration conventions worth knowing

- Parsers (`gmail/parser/*Parser.java`) are plain `@Component` beans — Spring auto-discovers and injects the full `List<EmailParser>`; ordering is controlled by `@Order` (see `GMAIL_INGESTION_PIPELINE.md`).
- Schedulers live in a `scheduler/` sub-package per domain (`portfolio/scheduler/`, `tracking/scheduler/`, `gmail/scheduler/`, `mf/scheduler/`), not centralized.
- No mapper library (`MapStruct` is a dependency but its actual usage was not confirmed as widespread in the controllers inspected — DTO construction appears largely hand-written).

Financial Engine

 [Source file · docs/02-backend/FINANCIAL_ENGINE.md](#)

For the business-level rules and formulas in plain terms, see `00-pmo-product/BUSINESS_LOGIC_RULES.md`. This document is the engineering view: which classes own which calculation, their exact signatures, and the invariants a change must not break.

Net worth — `recommendation/service/PortfolioContextService.java`

Entry point: `build(Long userId) → PortfolioContext`, consumed by `recommendation/controller/WealthController` → `GET /api/wealth/summary`. This is the *only* sanctioned place to compute net worth — do not recompute it elsewhere.

Data sources it pulls from: `HoldingRepository` (stocks/MF), `TrackingService.getSummary()` (FD/RD/EPF/other/loans), `CashAccountRepository.sumBalanceByUser()` (cash).

Failure handling: if `TrackingService.getSummary()` throws, FD/RD/EPF/other/loans all default to `BigDecimal.ZERO` and a data-gap note is appended to the result rather than the call failing — net worth still computes from whatever data is available, with the gap surfaced, not hidden.

Constants (hardcoded in this class — change here if concentration thresholds need tuning): `SINGLE_STOCK` 10%/25%, `SECTOR` 30%, `ASSET_ALLOCATION` 85%, `LEVERAGE` 50%, `SECTOR_COVERAGE_TRUSTWORTHY` 80%.

Holdings — `portfolio/service/PortfolioService.java`

`recomputeFromLedger(Holding h)` is the single write path for `quantity / averageCost`. Any new code path that creates or modifies a `Transaction` must call this afterward — do not set `Holding` fields directly.

`reconcileAllUsers()` — called nightly by `portfolio/scheduler/HoldingReconciliationScheduler` (`fixedDelay=86_400_000`, 60s after startup). Iterates every user, replays every holding's ledger, logs a warning if the pre-replay stored values had drifted.

`computeRealXirr(Holding h)` builds the cash-flow list for `util/XirrCalculator` from the same transaction ledger, plus a synthetic "sold today at current value" flow. Falls back to the stored `Holding.xirr` field only if the ledger is empty or Newton-Raphson fails to converge — never silently returns 0 or null when a fallback value exists.

XIRR — `portfolio/util/XirrCalculator.java`

Pure static utility, no Spring dependency. `computeXirrPercent(List<CashFlow>) → Double` (nullable). Newton-Raphson, `MAX_ITERATIONS=100`, `PRECISION=1e-7`, floors the rate at `-0.999999`. Returns `null` (never a guessed value) when: fewer than 2 flows, all flows the same sign, or the iteration doesn't converge.

FD/RD lifecycle — `tracking/service/TrackingService.java`

`detectAndLinkRenewal(userId, newFdId)` / `detectAndLinkRdRenewal(userId, newRdId)` — call these after every FD/RD creation path (manual entry and Gmail import both go through `addFd` / `addRd`, which call these internally — do not bypass by inserting an entity directly).

Tolerances are class constants: `RENEWAL_DATE_WINDOW_DAYS=10`, `RENEWAL_AMOUNT_TOLERANCE=0.05`. If you need to tune false-positive/false-negative renewal-matching behavior, these are the two knobs — see `BUSINESS_LOGIC_RULES.md` for why these specific values were chosen.

`markMaturedDeposits()` — called nightly by `tracking/scheduler/DepositMaturityScheduler` (`fixedDelay=86_400_000`). Flips ACTIVE → MATURED once past `maturityDate`. Note this runs independently of and does not call renewal detection — a matured-but-unrenewed deposit simply sits at status MATURED until a new deposit triggers a renewal match against it.

MF redemption — `redemption/service/RedemptionService.java`

`recordRedemption(userId, holding, unitsRedeemed, nav)` — computes STCG/LTCG tax at the moment of redemption (see formula in `BUSINESS_LOGIC_RULES.md`) and persists an `MfRedemption` row with `status=ACTIVE`, `reinvestedAmount=0`.

`recordReinvestment(userId, redemptionId, amount, date, targetFund, note)` — appends a `Reinvestment` chunk row, flips `status=COMPLETED` once `reinvestedAmount >= redeemedAmount`.

`totalAwaitingRedeployment(userId)` — sums `cashRemaining` across `ACTIVE` redemptions. **Do not add this to `CashAccount` totals** — it's an earmark on cash already counted there (the sale proceeds land in a bank account, which `CashAccount.balance` already reflects), not additional cash. Adding both would double-count.

`getDeploymentPlan(userId, redemptionId)` — rule-based only (20/30/50 split with an ATR-derived pullback trigger for the "after a correction" tranche). Not live market monitoring; do not present this as an AI-driven recommendation in the UI.

Signal engine — `signal/service/SignalEngine.java`

`analyse(String symbol) → SignalPayload`. Family weights and thresholds are class constants (`W_TREND=0.25`, `W_MOMENTUM=0.25`, `W_STRUCTURE=0.25`, `W_VOLUME=0.15`, `W_VOLATILITY=0.10`, `BUY_THRESHOLD=20`, `SELL_THRESHOLD=-20`).

If adding a new factor: add it as its own family with its own weight (rebalance the others so they still sum to 1.00), not as an additional vote inside an existing family — the whole point of the family-weighted design is that correlated indicators must never silently dominate the score. See `01-architecture/ARCHITECTURE_DECISIONS.md` ADR-8.

`MarketStructureAnalyzer` provides the `structure` family score (uptrend/downtrend/ranging via swing high/low detection) — a separate class, not inlined in `SignalEngine`.

Confidence ceiling is computed from `weightAvailable / totalWeight`, where `totalWeight` is the sum of all defined family weights (currently 1.00, since `microstructure` has weight 0 and is never counted toward

the ceiling denominator — verify this doesn't change if `microstructure` is ever given a nonzero weight when a broker feed becomes available).

ATR execution (`buildExecution`): `ATR_STOP_MULTIPLE=2.0`, `ATR_TARGET_MULTIPLE=4.0` — fixed 2:1 reward:risk. Position sizing is deliberately unimplemented (`limitedBy="NOT_SIZED"`) — do not add a sizing heuristic without first wiring in a real, tracked cash-balance and equity input; the class's own documentation states inventing one would violate the no-fabrication rule.

Card optimizer — `card/optimizer/{CardOptimizerService,SpendAggregator}.java`

`SpendAggregator.aggregate(userId)` — 90-day window, buckets by `ExpenseCategory`, computes a `coverage` ratio (categorized / total spend). Every category is flagged `inferred=true` — this is merchant-name inference, never present it as MCC data.

`CardOptimizerService.analyse(userId)` — `MIN_TRUSTWORTHY_COVERAGE=0.50`. Verdict precedence is a fixed if/else chain (see `BUSINESS_LOGIC_RULES.md`); do not reorder without re-verifying the "un-trustworthy coverage always wins" rule stays first.

`rewardsRealized12m` is hardcoded `null` throughout — it is not backed by real reward-transaction history yet. Do not populate it with a projection; that would misrepresent an estimate as a measurement.

Internal transfers — `ledger/service/LedgerTransferService.java`

`applyCashEffect(transfer)` — the only method that mutates `CashAccount.balance` for a transfer. Guarded by `LedgerTransfer.applied` (boolean) so a re-save or replay can never move money twice.

`delete(user, id)` reverses by the exact equal-and-opposite operation (not by recomputation) so balances return to precisely where they were.

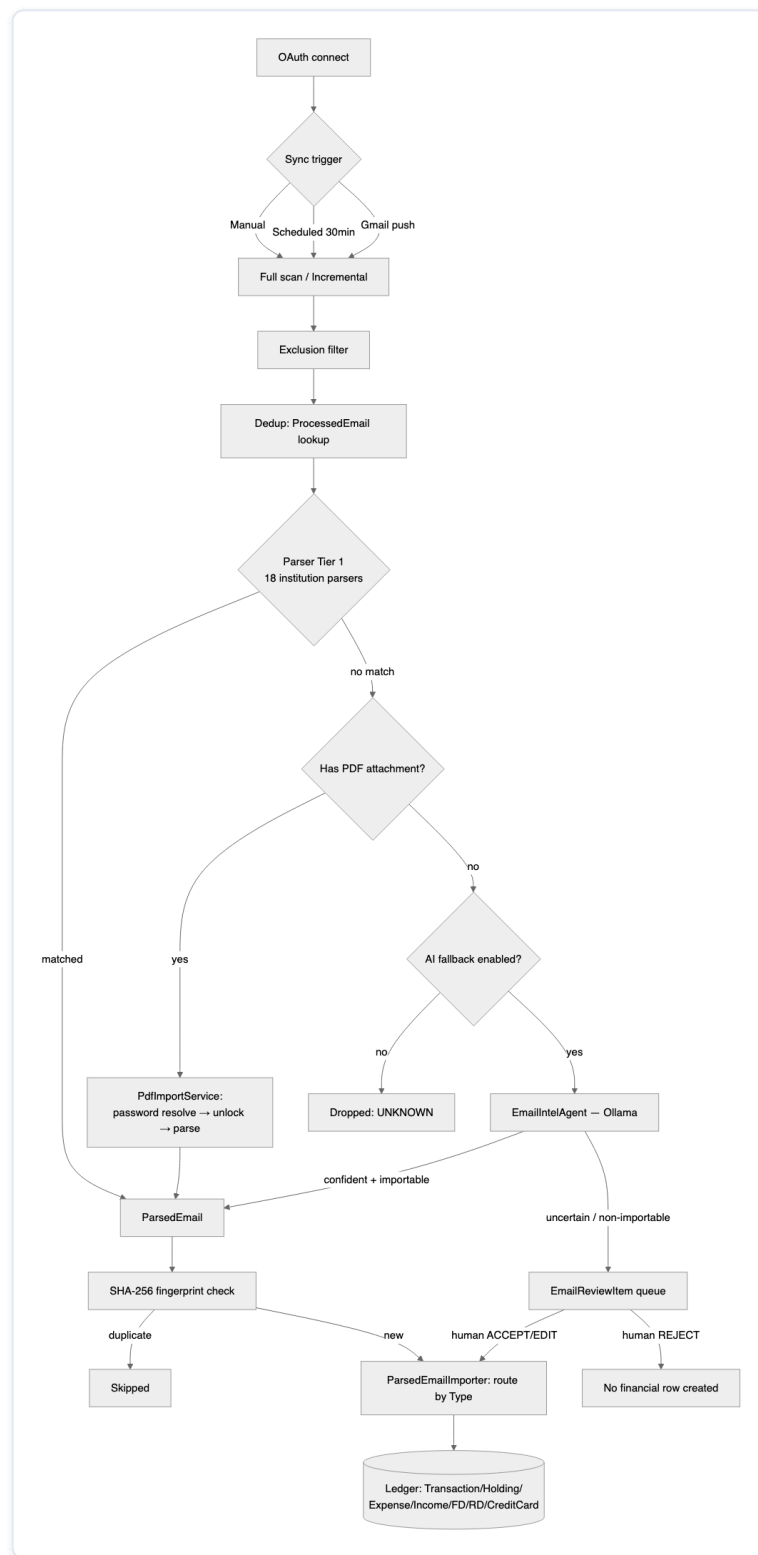
If you add a new `destinationType`, ensure the corresponding asset-creation path (wherever that asset type is actually created) does *not* also credit cash — `LedgerTransferService` already did that on the way in.

Gmail Ingestion Pipeline

 [Source file · docs/02-backend/GMAIL_INGESTION_PIPELINE.md](#)

All paths relative to `backend/src/main/java/com/marketai/`. This is the most complex subsystem in the backend — read this fully before modifying anything in `gmail/`, `sync/`, or `ai/intel/`.

Pipeline overview



1. OAuth / Auth — `gmail/service/GmailClientService.java`

- Config (:41–48) loads `clientId` / `clientSecret` / `redirectUri` via `@Value` , with a `@PostConstruct` `loadFromEnvFileIfMissing()` (:53–72) fallback reading a gitignored `gmail.env` file so credentials survive restarts without re-exporting env vars.

- `getAuthorizationUrl(userId)` (:89–97) — scope `GmailScopes.GMAIL_READONLY`, `accessType=offline`, `prompt=consent` (forces a refresh token every time, :95). `state` is signed as `"<userId>.<hmacSha256(userId)>"` (:116–126), HMAC key = OAuth client secret.
- Callback (`GmailController.callback`, :72–114): verifies state, exchanges code, fetches connected email, **upserts** `GmailToken` by `userId` — refresh token only overwritten if Google returned a new one (:95), preserving the original across re-consents that omit it.
- **Token refresh callback:** `buildGmailService(accessToken, refreshToken, TokenRefreshCallback)` (:143–176) attaches a `CredentialRefreshListener` so a mid-call silent token refresh is persisted immediately via the caller's callback — every caller (`GmailSyncService`, `SyncJobRunner`, `GmailWatchService`) wires this straight to `tokenRepo.save(token)`.
- `GmailToken.accessToken` / `refreshToken` are stored **plaintext** (2000-char columns) — no encryption at rest, unlike `SavedPdfPassword` which is AES-GCM encrypted.
- Disconnect (:316–320) deletes the `GmailToken` row only — no explicit Google-side revocation call is made.

2. Email discovery

FULL-SCAN PATH — `GMAILSYNCSERVICE.DOSYNCFORUSER` (`GMAIL/SERVICE/GMAILSYNCSERVICE.JAVA`)

- Guarded by a **per-user** `ReentrantLock` (:38, 74–83) — concurrent syncs for one user are refused outright, not queued.
- Query built in `GmailClientService.fetchRecentMessages` (:187–217): `"newer_than:" + lookbackPeriod + " (category:primary OR category:updates OR category:promotions OR category:social)"` Paginated up to 20 pages, each message hydrated with `.setFormat("full")`.
- **Exclusion filter runs before any parser:** `ExcludedSenderRepository` patterns checked against `from+subject+body` (:164–167, 205–215). Matched emails are marked `EXCLUDED` / `SKIPPED` and never reach parsers, PDF queueing, or the AI fallback.
- Per-message dedup by Gmail message ID via `ProcessedEmail` lookup (:172–186) — a prior non-FAILED row is skipped; a prior FAILED row is retried.

INCREMENTAL PATH — `GMAILINCREMENTALSYNCSERVICE`

- `fetchDelta(gmail, startHistoryId)` calls `users.history.list` filtered to `messageAdded` only, paginating up to `MAX_PAGES=50`.
- **Watermark rules** (documented in the class javadoc — do not weaken these):
- No stored `startHistoryId` → forces a full sync.
- HTTP 404 (historyId aged out of Gmail's retention window) → forces a full sync.
- Any other `history.list` failure → watermark left untouched, not advanced.
- Page-limit hit mid-drain → watermark untouched, remainder picked up next run.
- Empty result set → watermark left as-is (Gmail's index can lag a push notification).
- `advanceWatermark(current, candidate)` — pure `BigInteger` comparison, only ever moves forward.

- Consumed by `SyncJobRunner.runIncremental` — falls back to `runFull(userId, "14d")` whenever `!delta.isUsable()`. Fetches only the exact new message IDs (`syncSpecificMessages`), never a re-scan. The watermark is persisted **only when the delta was fully drained**.
- `runFull` captures the mailbox's *current* historyId **before** scanning, so anything arriving mid-scan is caught by the next incremental run rather than falling in a gap.

JOB-QUEUE LAYER — SYNC/ PACKAGE (SEE `JOBS_AND_SCHEDULERS.MD` FOR THE WORKER MECHANICS)

- `SyncJob` entity: `type` (`GMAIL_FULL_SYNC` / `GMAIL_INCREMENTAL_SYNC` / `GMAIL_RETRY_FAILED`), `status` (`QUEUED` / `RUNNING` / `SUCCEEDED` / `FAILED` / `CANCELLED`), `trigger` (`MANUAL` / `SCHEDULED` / `PUSH` / `RECOVERY`), `attempts` / `maxAttempts` (default 3).
- `SyncJobService.enqueue()` collapses duplicates — an existing `QUEUED` / `RUNNING` job of the same type for that user is reused rather than a new one inserted (critical for push bursts).
- `SyncJobRunner.run(jobId)` dispatches `GMAIL_INCREMENTAL_SYNC` → `runIncremental`; `GMAIL_FULL_SYNC` / `GMAIL_RETRY_FAILED` → `runFull`.
- The old inline `POST /api/gmail/sync` (`GmailController.triggerSync`) still exists and still runs synchronously — kept for compatibility. The async path is `POST /api/sync/gmail`.

3. Push notifications

- `GmailWatchService.registerWatch(userId)` — calls `users().watch("me", ...)` with `topicName / labelIds=[INBOX]`; stores `expiration` on `GmailToken.watchExpiration`. Seeds `lastHistoryId` **only if none exists yet** — never rewinds an existing watermark.
- `renewAllDueWatches()` renews anything missing or expiring within `RENEW_WHEN_DAYS_LEFT=3` days. Scheduled daily (`GmailSyncScheduler.renewWatches`, `fixedDelay=86_400_000`).
- Push is explicitly "latency optimisation only" (class javadoc) — the 30-minute scheduled poll is never disabled, because Gmail caps push at 1 notification/sec/user and silently drops the rest.
- `GmailPushController` (`POST /api/gmail/push?token=...`): shared-secret query param checked against `app.gmail.push.verification-token`. **Always returns 200**, even on a bad token or any internal error, to avoid a Pub/Sub retry storm on a call that can never succeed. Decodes the base64 Pub/Sub payload → `emailAddress` → matches to a `GmailToken` by `connectedEmail` → enqueues a `GMAIL_INCREMENTAL_SYNC` job (duplicate-collapsed by `SyncJobService.enqueue`). **The notification's own historyId is never used as the watermark** — the runner always re-derives it from `history.list`.
- Disabled by default: `app.gmail.push.enabled=false`.

4. Parser Tier 1 (deterministic) — `gmail/parser/`

Interface:

```
public interface EmailParser {
    boolean canParse(String from, String subject);
    List<ParsedEmail> parse(String from, String subject, String bodyText);
}
```

Spring injects `List<EmailParser> parsers`, ordered by `@Order`. **All parsers are `@Order(1)` except `BankTransactionParser`, which is `@Order(100)`** — the generic catch-all, must run last. `GmailSyncService` iterates in order and stops at the first parser whose `canParse` is true and `parse` returns non-empty.

PARSER	INSTITUTION
<code>AngelOneParser</code>	Angel One/Angel Broking
<code>AxisBankParser</code>	Axis Bank (FD/RD only)
<code>BankTransactionParser</code>	Generic bank alert catch-all (<code>@Order(100)</code>)
<code>CamsParser</code>	CAMS RTA (MF SIP/purchase/redemption)
<code>CardBillParser</code>	Card issuers (statement/bill/amount due)
<code>DividendParser</code>	RTAs (LinkInTime/KFintech/CAMS/NSDL/CDSL) + all brokers
<code>GrowwParser</code>	Groww
<code>HdfcBankParser</code>	HDFC Bank (FD/RD only)
<code>IciciBankParser</code>	ICICI Bank (FD/RD/deposit)
<code>IciciDirectParser</code>	ICICI Direct (broker)
<code>IdfcBankParser</code>	IDFC First Bank (FD/RD)
<code>KfintechParser</code>	KFintech/Karvy RTA
<code>KotakParser</code>	Kotak Securities
<code>MstockParser</code>	mStock / Mirae Asset
<code>SbiBankParser</code>	SBI (FD/RD/deposit)
<code>UpiAppParser</code>	Paytm/PhonePe/GPay
<code>UpstoxParser</code>	Upstox
<code>YesBankParser</code>	Yes Bank (FD/RD)
<code>ZerodhaParser</code>	Zerodha/Kite

Supporting: `ParsedEmail.java` (DTO; `Type` enum: `TRADE_BUY`, `TRADE_SELL`, `FD_OPEN`, `RD_OPEN`, `MF_SIP`, `MF_REDEEM`, `DIVIDEND`, `INCOME`, `EXPENSE`, `CARD_BILL`, `UNKNOWN`), `ParserUtil` (shared string matching), `ContractNoteHelper` (shared PDF contract-note row parsing), `SpendCategorizer` (expense categorization helper).

5. PDF / attachment handling

- Discovery: `GmailClientService.findPdfAttachments` walks the MIME tree; bytes are **not** downloaded at sync time, only the reference (`PdfAttachmentRef`) — `downloadAttachment` is on-demand.
- Queueing: looked up by `(userId, gmailMessageId, filename)` — filename, **not** attachmentId, because Gmail can return a different attachment ID for the same physical file on refetch. New refs become a `PendingPdf` row; `tryAutoUnlock` is attempted immediately.
- `PendingPdf` status lifecycle: `NEEDS_PASSWORD` → `PASSWORD_FAILED` / `FAILED` / `IMPORTED` / `DISMISSED`. Stores `pipelineSteps` as a JSON trace (`email_detected`, `attachment_detected`, `password_found`, `pdf_downloaded`, `pdf_unlocked`, `trades_extracted`, `holdings_updated`, `dashboard_updated`) for the Contract Note Debug View.
- **Password resolution** (`PdfImportService.tryAutoUnlock`), three tiers in order: 1. Exact match: `SavedPdfPassword` for this PDF's `providerKey`. 2. Cross-provider PAN candidates: if the provider is in the hardcoded `PAN_PASSWORD_DOMAINS` set (25+ Indian brokers/RTAs/AMCs using PAN-uppercase convention), tries all saved passwords whose *own* provider is also in that set — handles multi-account households. 3. Hint-based sharing: if `passwordHint` mentions "pan", falls back to any saved password whose hint also mentions "pan".
- On success, saves a copy of the password under the *new* provider key too, so future statements from that provider unlock automatically.
- `SavedPdfPassword.encryptedPassword` — AES-256/GCM via `gmail/security/PasswordCipher.java`, key from `app.security.pdf-password-key` (random/ephemeral if unset).
- `PasswordHintExtractor` — regex priority order: PAN-first-4+DOB → PAN+DOB → PAN-only → Customer ID → Folio → DOB-only; falls back to a hardcoded `KNOWN_PROVIDER_DEFAULTS` map (CAMS/mStock/NSE/KFintech → "PAN uppercase") when the email gives no explicit instruction.
- PDFBox 3.x: `Loader.loadPDF(bytes, password)` throws `InvalidPasswordException` on a wrong password. On success, `PDFTextStripper().getText(doc)` extracted text runs through the **same parser chain**, then a direct `ContractNoteHelper` fallback, then the AI extractor as last resort.

6. Tier 2 AI fallback — `ai/intel/EmailIntelAgent.java`

- Triggered only when `parsed.isEmpty()` after all regex parsers, gated by `app.gmail.ai-fallback.enabled` (default true) and `emailIntelAgent.isEnabled()`.
- **Pre-filter**: `looksFinancial()` checks a `MONEY_SIGNALS` keyword list before ever calling the LLM — non-financial mail short-circuits at zero model cost.
- **Prompt** (`SYSTEM_INSTRUCTION`): fixed enum of 30 classification labels, a `confidence` (0-1), `extractedFields` (amount/date/merchant/symbol/quantity/price/units/nav/folio/fundName/bank/ cardLast4/ paymentMethod/counterparty), `reasoning`, and `evidence` (the literal source sentence). Explicit prompt rules: never invent values (use null), use `INTERNAL_TRANSFER` / `SELF_TRANSFER` for self-account movements (never an expense), give the exact NSE ticker, lower confidence on ambiguous/forwarded mail.
- Body truncated to `MAX_BODY_CHARS=6000`; routed through `LlmProviderRouter.complete()`, parsed via `LlmJsonParser` (strict readers — never silently coerce a malformed value).
- **EmailIntelType** — 26 semantic labels mapped to a coarser `ParsedEmail.Type` via `importAs()`. Several map to `null` on purpose (`MF_SWITCH`, `FD_MATURITY`, `RD_INSTALLMENT`, `INTERNAL_TRANSFER`,

`SELF_TRANSFER`, `OTP_OR_ALERT`, `PROMOTIONAL`, `STATEMENT_ONLY`, `UNKNOWN`) — these can **never** be auto-imported (`isImportable()` is false).

- **Confidence threshold:** `app.llm.min-confidence` default **0.85**. `confidence == null` is treated identically to below-threshold — absence of a score is never permission to import. Below threshold **or** a non-importable type → `REVIEW_REQUIRED`.
- **Field requirements per type** (enforced in `toParsedEmail`): trades need `symbol+price+quantity` all non-null; MF SIP/redeem need `amount`; FD needs `principal`; RD needs `amount`; everything else needs `amount`. A confident classification that still fails these checks routes to review rather than importing partial data.
- **Deterministic-safety guardrail — symbol resolution:** `resolveSymbol(candidate)` only accepts an **exact** match or the **sole** search hit from `MarketDataService.searchStocks`. An ambiguous/unrecognized ticker is refused (forces the trade to review) rather than risking a mis-attributed holding.
- Every classification (accepted, reviewed, or unparseable) is recorded via `AiAuditService` for the audit trail (`task=EMAIL_CLASSIFY`).

7. Review queue — `ai/review/`

- `EmailReviewItem` — one row per `(user_id, gmail_message_id)`, storing `proposedType`, `confidence`, `reviewReason`, `reasoning`, `evidence`, `extractedFields` (raw JSON), and status (`PENDING` / `ACCEPTED` / `EDITED` / `REJECTED`).
- Routing: `EmailIntelAgent`'s `IMPORT` outcome goes straight to `ParsedEmailImporter`; `REVIEW_REQUIRED` / `UNRESOLVED` calls `emailReviewService.enqueue()` and the email is marked `SKIPPED` (type `REVIEW_REQUIRED`) in `ProcessedEmail` — **not** imported.
- `enqueue()` is idempotent — a resolved item is never resurrected; headline fields are taken only from the already-validated `result.getParsed()` payload, never re-derived from raw model text.
- **Decision flow** (`POST /api/review/{id}/decision`):
 - `REJECT` → status `REJECTED`, no import.
 - `ACCEPT` → imports using the stored proposed fields as-is.
 - `EDIT` → imports using request-supplied corrected fields, falling back to stored values.
- `importDecided` rejects non-importable types silently as "a judgement, no financial row created" — protects against e.g. accepting an `INTERNAL_TRANSFER` as a spend. Requires a non-null amount or throws 400.
- An accepted/edited review item still passes the SHA-256 fingerprint gate exactly like a normal import (§8) — the review queue is not a fingerprint-gate bypass.

8. Deduplication / idempotency — `ParsedEmailImporter.java`

Two independent layers:

1. **SHA-256 fingerprint gate** (primary): `TransactionFingerprinter.fingerprint(pe)` hashes `type | date | amount | price | principal | monthlyAmount | units | quantity | symbol | folio | fundName | bank | merchant | cardLast4` — **deliberately excludes** Gmail message ID, sender, and free-text description, so a forwarded/resent copy still collides. Checked via `fingerprintRepo.existsByUserIdAndFingerprint` before any table-specific routing. A new

`ImportedTransactionFingerprint` row is saved only on a **successful** import — a failed import stays retryable.

- Per-domain content checks** (defense-in-depth): `isDuplicateExpense` (sourceEmailId OR amount+date+merchant match), `isDuplicateIncome` (similar, plus a dividend-specific rule), FD (`bank+principal+startDate`), RD (`bank+monthlyAmount+startDate`), trades/MF (`portfolioService.isDuplicateTrade` — requires an existing holding, checks `symbol+date+quantity+price`). - `CARD_BILL` has no dedup beyond the fingerprint gate — it's an overwrite-style update (`currentDue / currentDueDate`), naturally idempotent by design. - `POST /api/gmail/resync` wipes both `ProcessedEmail` and `ImportedTransactionFingerprint` for the user, so a full resync can re-import anything manually deleted — subsequent re-import is then gated purely on whether the domain record still exists (per-domain checks above).

9. Ledger commit — `ParsedEmailImporter.routeImport`

PARSEDEMAIL.TYPE	WHAT GETS WRITTEN
TRADE_BUY	<code>portfolioService.addHolding(...)</code> on the user's first portfolio (auto-created if none) — creates/updates <code>Holding</code> + <code>Transaction</code>
TRADE_SELL	If a matching <code>Holding</code> exists: <code>sellHolding(...)</code> . If not: booked as <code>Income</code> with <code>IncomeSource.CAPITAL_GAIN</code> instead (no holding to sell against)
MF_SIP / MF_REDEEM	Also <code>addHolding</code> , synthetic <code>.MF</code> -suffixed symbol from the fund name. Rejected outright if <code>FinancialDataValidator.looksLikeUnverifiableFundName</code> flags the name as boilerplate
FD_OPEN	<code>trackingService.addFd(...)</code> then <code>detectAndLinkRenewal</code> — every FD_OPEN is checked for a renewal match
RD_OPEN	<code>trackingService.addRd(...)</code> then <code>detectAndLinkRdRenewal</code>
DIVIDEND	<code>Income</code> with <code>IncomeSource.DIVIDEND</code> , <code>payer=symbol</code>
INCOME	<code>Income</code> with <code>IncomeSource.fromLabel(...)</code>
EXPENSE	<code>Expense</code> with <code>ExpenseCategory.fromLabel(...)</code>
CARD_BILL	Looks up <code>CreditCard</code> by last-4 then issuer name; updates <code>currentDue / currentDueDate</code> — no new record if no card on file
UNKNOWN / default	No-op — silently dropped (should be unreachable, since both parsers and the review queue gate on importable types upstream)

Developer guide: adding a new bank/broker parser

- Implement `EmailParser`** in a new class under `gmail/parser/` : `java @Component @Order(1) //`
specific-institution parsers use 1; only `BankTransactionParser` uses 100 (catch-all, must run

```
last) public class NewBrokerParser implements EmailParser { @Override public boolean
canParse(String from, String subject) { ... } @Override public List<ParsedEmail>
parse(String from, String subject, String bodyText) { ... } }
```

2. **Registration is automatic** — Spring component-scans the `@Component` and injects it into the `List<EmailParser>` wherever it's a constructor dependency. No manual registry.
3. **Naming:** `<Institution>Parser.java`, matching the existing set.
4. **canParse convention:** AND of (a) `ParserUtil.containsIgnoreCase(from, <sender domain keywords>)` and (b) `ParserUtil.containsIgnoreCase(subject, <subject keywords>)`.
5. **parse convention:** return `ParsedEmail.builder()...build()` populating only the fields relevant to the target `Type`. Return an **empty list** (never throw) when required fields can't be extracted — the sync loop treats empty as "matched but recorded nothing" and tries the next parser.
6. **PAN password domain:** if the new broker/AMC uses the standard PAN-uppercase PDF password convention, add its sender domain to `PdfImportService.PAN_PASSWORD_DOMAINS`.
7. **Write a unit test** (pattern: `backend/src/test/java/com/marketai/gmail/parser/CamsParserTest.java`) — instantiate the parser directly (no Spring context needed, parsers have no dependencies). Test: (a) `canParse` true for a realistic sender/subject, false for unrelated mail; (b) `parse` on a realistic body produces the expected fields; (c) `parse` on a body missing required fields returns an empty list, never throws or guesses.
8. **No further registration needed** — the sync pipeline, dedup, and ledger-commit layers are all generic over `ParsedEmail.Type`, so a correctly-typed output flows through `ParsedEmailImporter` exactly like any existing parser's.

Jobs & Schedulers

 [Source file · docs/02-backend/JOBS_AND_SCHEDULERS.md](#)

@Scheduled **inventory (every scheduled method in the backend)**

FILE:LINE	TRIGGER	METHOD
amfi/service/AmfiNavService.java:47	cron="0 30 21 * * *"	refresh()
mf/scheduler/ MfNavHistoryScheduler.java:30	cron="0 0 22 * * *"	refreshHeldSchemeHistory()
news/service/NewsService.java:63	fixedRateString="\$ {app.news.refresh- rate-ms:1800000}" (30 min)	fetchLatestMarketNews()
portfolio/scheduler/ HoldingReconciliationScheduler.java:25	fixedDelay=86_400_000, initialDelay=60_000 (daily)	scheduledReconciliation()
tracking/scheduler/ DepositMaturityScheduler.java:23	fixedDelay=86_400_000, initialDelay=30_000 (daily)	scheduledMaturityCheck()
market/service/ MarketDataService.java:204	fixedRateString="\$ {app.market.refresh- rate-ms:900000}" (15 min — no YAML entry , pure annotation default)	refreshMarketData()
market/service/ MarketDataService.java:297	cron="0 30 18 * * MON- FRI", zone="Asia/ Kolkata"	scheduledSectorIndexHistoryRefresh
sync/worker/SyncJobWorker.java:73	fixedDelay=5_000, initialDelay=10_000	poll()

FILE:LINE	TRIGGER	METHOD
<code>sync/worker/SyncJobWorker.java:105</code>	<code>fixedDelay=600_000,</code> <code>initialDelay=600_000</code> (10 min)	<code>reclaimStale()</code>
<code>gmail/scheduler/ GmailSyncScheduler.java:40</code>	<code>fixedDelay=1_800_000</code> (30 min)	<code>scheduledSync()</code>
<code>gmail/scheduler/ GmailSyncScheduler.java:61</code>	<code>fixedDelay=86_400_000,</code> <code>initialDelay=120_000</code> (daily)	<code>renewWatches()</code>

Note: `app.market.refresh-rate-ms` and several other `@Value("${x:default}")` defaults inlined at the annotation site have no corresponding entry in `application.yml` at all — see `04-devops/ENVIRONMENT_VARIABLES.md` for the full list of config that only exists as a Java-side default.

Async job queue (`sync/` package) — the mechanism behind Gmail sync

This is the general-purpose durable job queue; today its only job type is Gmail sync, but the design (`SyncJobType` enum, atomic claim, retry/reclaim) is not Gmail-specific.

CONCURRENCY

`SyncJobWorker` builds a fixed-size `ThreadPoolExecutor(concurrency, concurrency, 0L, MILLISECONDS, LinkedBlockingQueue<>())` with **daemon** threads named `sync-worker`. `concurrency` = `app.sync.worker.concurrency` (default **2**). Deliberately small: the bottleneck is the local LLM classifier (~8s/email measured), and parallelizing on one machine makes each job slower rather than the total faster — it also keeps one in-flight job per user under Gmail's 6,000-units/min per-user quota.

IN-FLIGHT TRACKING

A `ConcurrentHashMap.newKeySet<Long>()` of job IDs this JVM currently owns, so the 5s poller never re-dispatches a job it's already running locally.

STARTUP RECLAIM

`@PostConstruct start()` immediately calls `jobService.requeueStale(Duration.ZERO)` — anything left `RUNNING` belongs to a previous process that died; reclaimed unconditionally at boot.

ATOMIC CLAIMING — THE CONCURRENCY-SAFETY MECHANISM

```
UPDATE SyncJob SET status = RUNNING, claimedBy = :worker, claimedAt = :now,
  startedAt = COALESCE(startedAt, :now), attempts = attempts + 1
WHERE id = :id AND status = QUEUED
```

`@Modifying(clearAutomatically=true, flushAutomatically=true)`, returns rows-affected. Exactly one worker's `UPDATE` matches even if two workers read the same `QUEUED` row simultaneously — the status check lives in the `WHERE` clause specifically so a read-then-write race in application code (which would let both proceed and double-import) is structurally impossible.

DUPLICATE-ENQUEUE GUARD

`SyncJobService.enqueue()` checks `existsByUser_IdAndTypeAndStatusIn(userId, type, [QUEUED, RUNNING])` before inserting — collapses a burst of Pub/Sub at-least-once push notifications into one sync per mailbox, not one per notification.

RETRY & STALE-JOB RECLAIM

- `SyncJobService.fail(jobId, error)`: sets `lastError` (truncated 1000 chars); requeues to `QUEUED` if `attempts < maxAttempts` (default 3), else terminal `FAILED` + `finishedAt`. Most failures are treated as transient (token refresh, Gmail 5xx, slow model call).
- `requeueStale(Duration)`: finds `RUNNING` jobs with `claimedAt` older than the threshold; requeues if attempts remain, else `FAILED` with `"Worker did not finish; exceeded retry budget."`. Called both at startup (`Duration.ZERO`) and every 10 minutes (`Duration.ofMinutes(staleAfterMinutes)`, default 90).

FAILURE ISOLATION

The `pool.submit()` lambda wraps `runner.run(jobId)` in try/catch — an escaping `Throwable` is caught, logged, and routed to `jobService.fail()` specifically because an uncaught exception on a pool thread would silently kill that thread and permanently shrink capacity.

ENTRY POINTS THAT ENQUEUE A JOB

SOURCE	TYPE	TRIGGER
<code>POST /api/sync/gmail</code>	<code>GMAIL_FULL_SYNC</code> or <code>GMAIL_INCREMENTAL_SYNC</code>	MANUAL
<code>GmailSyncScheduler.scheduledSync()</code>	<code>GMAIL_INCREMENTAL_SYNC</code>	SCHEDULED
<code>GmailPushController.receive()</code>	<code>GMAIL_INCREMENTAL_SYNC</code>	PUSH
(implicit) stale-job reclaim	same type, requeued	<code>RECOVERY</code> conceptually (see <code>SyncTrigger.RECOVERY</code> , though the reclaim path reuses the original job's trigger rather than rewriting it)

Legacy inline sync path (still present)

`POST /api/gmail/sync` (`GmailController.triggerSync`) runs `GmailSyncService.doSyncForUser` synchronously, blocking the HTTP request for the full duration of the scan (including any AI classification calls).

Kept for backward compatibility — `POST /api/sync/gmail` is the intended path for new integrations since it returns immediately with a pollable job.

API Specification

 [Source file · docs/02-backend/API_SPECIFICATION.md](#)

Base URL: `http://localhost:8080` (dev). Interactive docs at `/swagger-ui.html` when running.

Auth: every endpoint except the explicit allowlist below requires `Authorization: Bearer <accessToken>`. The user is resolved via `@AuthenticationPrincipal User` — no endpoint accepts a `userId` path/query parameter, so one user cannot read another's data by changing an ID.

Unauthenticated (permitAll) paths: `/api/auth/**`, `/api/gmail/callback`, `/api/gmail/push` (protected instead by a shared-secret `?token=` query param), `/api-docs/**`, `/swagger-ui/**`, `/actuator/health`.

Note: several GET endpoints (global reference data — `amfi`, `analyst`, `card.catalog/categories`, `forecast`, `market`, `news`, `recommendation.benchmarks`, `signal`, `technical`) do not take a `User` parameter but are still gated by the default `anyRequest().authenticated()` rule — they rely entirely on the filter chain, not on defensive code in the controller.

Auth — `/api/auth`

METHOD	PATH	BODY	AUTH
POST	<code>/register</code>	<code>RegisterRequest</code>	Unauthenticated
POST	<code>/login</code>	<code>LoginRequest</code> → <code>AuthResponse</code>	Unauthenticated
POST	<code>/refresh</code>	<code>RefreshTokenRequest</code>	Unauthenticated
POST	<code>/logout</code>	—	Technically requires a valid JWT (<code>@AuthenticationPrincipal UserDetails</code>) despite the path being allowlisted

AI — `/api/ai`

METHOD	PATH	BODY	RETURN
POST	<code>/analyse-stock</code>	<code>AiRequest</code>	<code>AiResponse</code>
POST	<code>/market-summary</code>	—	<code>AiResponse</code>
POST	<code>/portfolio-review/{portfolioId}</code>	—	<code>AiResponse</code>

METHOD	PATH	BODY	RETURN
POST	/chat	AiRequest	AiResponse

AI Review Queue — /api/review

METHOD	PATH	PARAMS/BODY	RETURN
GET	/	all=false	List<EmailReviewItem>
GET	/count	—	{pending: number}
POST	/id/decision	ReviewDecisionRequest (ACCEPT/EDIT/REJECT)	EmailReviewItem

AMFI — /api/mf-nav

METHOD	PATH	PARAMS	RETURN
GET	/	name	AmfiNavResult or 404

Analyst — /api/analyst

METHOD	PATH	PATH VARS	PARAMS	RETURN
GET	/symbol	symbol	name (opt)	AnalystAssessment

Actions — /api/actions, /api/todays-actions

METHOD	PATH	PATH VARS	BODY	RETURN
GET	/api/actions	—	—	List<ActionItemDto>
POST	/api/actions	—	ActionUpdateRequest	ActionItemDto
PATCH	/api/actions/{id}/note	id	{note}	ActionItemDto
GET	/api/todays-actions	—	—	TodaysActionsResponse

Cards — `/api/cards`

METHOD	PATH	PATH VARS	BODY	RETURN
GET	<code>/catalog</code>	—	—	<code>List<CatalogEntry></code>
GET	<code>/categories</code>	—	—	<code>List<String></code>
GET	<code>/</code>	—	—	<code>List<CardResponse></code>
POST	<code>/</code>	—	<code>CardRequest</code>	<code>CardResponse</code>
PUT	<code>/ {id}</code>	<code>id</code>	<code>CardRequest</code>	<code>CardResponse</code>
DELETE	<code>/ {id}</code>	<code>id</code>	—	204
PUT	<code>/ {id} /points</code>	<code>id</code>	<code>{pointsBalance}</code>	<code>CardResponse</code>
POST	<code>/recommend</code>	—	<code>RecommendRequest</code>	<code>List<RecommendResult></code>
GET	<code>/points-tips</code>	—	—	<code>List<PointsTip></code>

Expenses — `/api/expenses`

METHOD	PATH	PARAMS	BODY	RETURN
GET	<code>/</code>	<code>year,</code> <code>month</code>	—	<code>List<ExpenseResponse></code>
POST	<code>/</code>	—	<code>ExpenseRequest</code>	<code>ExpenseResponse</code> (201)
PUT	<code>/ {id}</code>	—	<code>ExpenseRequest</code>	<code>ExpenseResponse</code>
DELETE	<code>/ {id}</code>	—	—	204
GET	<code>/miscategorized-investments</code>	—	—	<code>List<ExpenseResponse></code>
DELETE	<code>/miscategorized-investments</code>	—	—	<code>{removed: number}</code>
GET	<code>/summary</code>	<code>year,</code> <code>month</code>	—	<code>Map<String, Object></code>

Forecast — `/api/forecast`

METHOD	PATH	PARAMS	RETURN
GET	<code>/indices</code>	—	<code>Map<String,String></code>
GET	<code>/</code>	<code>symbol, horizon=2W, name</code>	<code>ForecastResponse</code>

Gmail — `/api/gmail`

METHOD	PATH	PATH VARS	PARAMS/BODY	RETURN
GET	<code>/auth-url</code>	—	—	<code>{url}</code>
GET	<code>/callback</code>	—	<code>code, state</code>	redirects (allowlisted)
GET	<code>/status</code>	—	—	<code>GmailStatusResponse</code>
POST	<code>/sync</code>	—	—	<code>GmailSyncResult</code> (inline/blocking — prefer <code>/api/sync/gmail</code>)
GET	<code>/history</code>	—	<code>limit=100</code>	<code>List<ProcessedEmail></code>
GET	<code>/pending-pdfs</code>	—	—	<code>List<PendingPdf></code>
POST	<code>/pending-pdfs/{id}/unlock</code>	<code>id</code>	<code>{password}</code>	<code>Map<String,Object></code>
DELETE	<code>/pending-pdfs/{id}</code>	<code>id</code>	—	204
GET	<code>/reconciliation-report</code>	—	—	<code>ReconciliationReportDto</code>
GET	<code>/contract-note-debug</code>	—	—	<code>List<PendingPdf></code>
POST	<code>/reprocess-failed</code>	—	—	<code>Map<String,Object></code>
GET	<code>/debug-pdf/{id}</code>	<code>id</code>	—	<code>Map<String,Object></code>
GET	<code>/excluded-senders</code>	—	—	<code>List<ExcludedSender></code>
POST	<code>/excluded-senders</code>	—	<code>{pattern, label}</code>	<code>ExcludedSender</code>
DELETE	<code>/excluded-senders/{id}</code>	<code>id</code>	—	204/404

METHOD	PATH	PATH VARS	PARAMS/BODY	RETURN
GET	/saved-passwords	—	—	List<SavedPasswordResponse>
PUT	/saved-passwords/{id}	id	{password}	204
DELETE	/saved-passwords/{id}	id	—	204
POST	/resync	—	—	GmailSyncResult (wipes ProcessedEmail + fingerprints for the user)
POST	/retry-failed	—	—	GmailSyncResult
DELETE	/disconnect	—	—	204
POST	/push	—	?token=, Pub/ Sub body	200 always (unauthenticated , shared-secret token)

Goals — /api/goals

METHOD	PATH	PATH VARS	BODY	RETURN
GET	/	—	—	List<GoalResponse>
POST	/	—	GoalRequest	GoalResponse
PUT	/ {id}	id	GoalRequest	GoalResponse
DELETE	/ {id}	id	—	204

Income — /api/income

METHOD	PATH	PARAMS	BODY	RETURN
GET	/	year, month	—	List<IncomeResponse>
GET	/by-source	source, year	—	List<IncomeResponse>
POST	/	—	IncomeRequest	IncomeResponse
PUT	/ {id}	—	IncomeRequest	IncomeResponse
DELETE	/ {id}	—	—	204
GET	/summary	year, month	—	Map<String, Object>

Ledger — `/api/ledger`

METHOD	PATH	PATH VARS	BODY	RETURN
GET	<code>/accounts</code>	—	—	<code>List<CashAccount></code>
POST	<code>/accounts</code>	—	<code>CashAccountRequest</code>	<code>CashAccount</code>
PUT	<code>/accounts/{id}/balance</code>	<code>id</code>	<code>{balance}</code>	<code>CashAccount</code>
GET	<code>/transfers</code>	—	—	<code>List<LedgerTransfer></code>
POST	<code>/transfers</code>	—	<code>TransferRequest</code>	<code>LedgerTransfer</code>
DELETE	<code>/transfers/{id}</code>	<code>id</code>	—	204
GET	<code>/cash-total</code>	—	—	<code>{totalCash}</code>

Market — `/api/market`

METHOD	PATH	PATH VARS	PARAMS	RETURN
GET	<code>/overview</code>	—	—	<code>MarketOverviewDto</code>
GET	<code>/quote/{symbol}</code>	<code>symbol</code>	—	<code>QuotedDto</code>
GET	<code>/search</code>	—	<code>q</code>	<code>List<Stock></code>
GET	<code>/history/{symbol}</code>	<code>symbol</code>	<code>from, to</code> (ISO date, required)	<code>List<PriceHistory></code>
POST	<code>/history/{symbol}/fetch</code>	<code>symbol</code>	<code>range=1y</code>	202

Net Worth — `/api/networth`

METHOD	PATH	RETURN
GET	<code>/series</code>	<code>List<NetWorthSnapshot></code>
POST	<code>/snapshot</code>	<code>NetWorthSnapshot</code>

News — `/api/news`

METHOD	PATH	PATH VARS	PARAMS	RETURN
GET	<code>/</code>	—	<code>page=0, size=20</code> (capped 50)	<code>Page<News></code>
GET	<code>/symbol/{symbol}</code>	<code>symbol</code>	—	<code>List<News></code>

Portfolio — `/api/portfolios`

METHOD	PATH	PATH VARS	PARAMS/BODY	RETURN
POST	<code>/</code>	—	<code>{name, description}</code>	<code>Portfolio</code> (201)
GET	<code>/</code>	—	—	<code>List<Portfolio></code>
GET	<code>/id/summary</code>	<code>id</code>	—	<code>PortfolioSummaryDto</code>
POST	<code>/id/holdings</code>	<code>id</code>	<code>AddHoldingRequest</code>	<code>Holding</code> (201)
DELETE	<code>/portfolioId/holdings/{holdingId}</code>	both	—	204
DELETE	<code>/portfolioId/holdings</code>	<code>portfolioId</code>	<code>type=all\ stocks\ mf</code>	<code>Map<String,Integer></code>
PUT	<code>/portfolioId/holdings/{holdingId}</code>	both	<code>{quantity, averageCost, currentPrice, investedAmount, broker, folio, buyDate, xirr}</code>	<code>Holding</code>
POST	<code>/recalculate</code>	—	—	<code>Map<String,String></code>
GET	<code>/integrity-check</code>	—	—	<code>IntegrityReportDto</code>
POST	<code>/merge-duplicate-symbols</code>	—	—	<code>MergeSummaryDto</code>
POST	<code>/fix-mismatched-tickers</code>	—	—	<code>MergeSummaryDto</code>
POST	<code>/rebuild</code>	—	—	<code>Map<String,Object></code>

METHOD	PATH	PATH VARS	PARAMS/BODY	RETURN
POST	<code>/portfolioId/holdings/{holdingId}/sell</code>	both	<code>{quantity, salePrice}</code>	204
GET	<code>/portfolioId/holdings/{holdingId}/transactions</code>	both	—	<code>List<TransactionDto></code>
GET	<code>/mf-transactions</code>	—	<code>days=7</code> (clamped 1–365)	<code>List<TransactionDto></code>

Recommendation / Wealth — `/api/recommendation`, `/api/wealth`

METHOD	PATH	PATH VARS	PARAMS/BODY	RETURN
GET	<code>/api/recommendation/benchmarks</code>	—	—	<code>Map<String,Double></code>
GET	<code>/api/recommendation/{symbol}</code>	<code>symbol</code>	<code>name, pnlPercent, holdingValue, totalPortfolioValue</code> (opt)	<code>AnalystAssessment</code>
POST	<code>/api/recommendation/mf</code>	—	<code>MfRecommendationRequest</code>	<code>AnalystAssessment</code>
GET	<code>/api/wealth/summary</code>	—	—	<code>PortfolioContext</code> (single source of truth for net worth)

Reconciliation — `/api/reconciliation`

METHOD	PATH	RETURN
GET	<code>/report</code>	<code>ReconciliationReportDto</code>

Distinct from `/api/gmail/reconciliation-report` (import-history-based) — this is a broader cross-domain check (`ReconciliationService.checkAll`).

Redemptions — `/api/redemptions`

METHOD	PATH	PATH VARS	BODY	RETURN
GET	<code>/</code>	—	—	<code>List<MfRedemption></code>
GET	<code>/{"id"}/deployment-plan</code>	<code>id</code>	—	<code>DeploymentPlan</code>
POST	<code>/{"id"}/reinvestments</code>	<code>id</code>	<code>ReinvestmentRequest</code>	<code>MfRedemption</code>

Reminders — `/api/reminders`

METHOD	PATH	RETURN
GET	<code>/</code>	<code>List<ReminderResponse></code>

[PARTIALLY IMPLEMENTED] — read-only; no create/update/delete endpoint exists.

Recurring Investments — `/api/recurring-investments`

METHOD	PATH	PATH VARS	BODY	RETURN
GET	<code>/</code>	—	—	<code>List<RecurringInvestmentResponse></code>
POST	<code>/</code>	—	<code>RecurringInvestmentRequest</code>	<code>RecurringInvestmentResponse</code> (201)
DELETE	<code>/{"id"}</code>	<code>id</code>	—	204

Signals — `/api/signals`

METHOD	PATH	PATH VARS	RETURN
GET	<code>/{"symbol"}</code>	<code>symbol</code>	<code>SignalPayload</code>

Sync Jobs — `/api/sync`

METHOD	PATH	PATH VARS	PARAMS	RETURN
POST	<code>/gmail</code>	—	<code>full=false,</code> <code>lookback</code>	<code>SyncJobDto</code> (202) — prefer this over <code>/api/gmail/sync</code>

METHOD	PATH	PATH VARS	PARAMS	RETURN
GET	/jobs/{id}	id	—	SyncJobDto or 404
GET	/jobs	—	limit=10 (capped 50)	List<SyncJobDto>

Tax — /api/tax

METHOD	PATH	PARAMS	RETURN
GET	/summary	fyStartYear (opt)	TaxResponse

Technical — /api/technical

METHOD	PATH	PATH VARS	RETURN
GET	/symbol	symbol	TechnicalAnalysisDto

Tracking (FD/RD/Loan/EPF/Other) — /api/tracking

METHOD	PATH	BODY	RETURN
GET	/summary	—	TrackingSummaryResponse
GET/POST/ PUT/DELETE	/fd, /fd/{id}	FdRequest	FdResponse
POST	/fd/{id}/close	{actualAmount} (opt)	FdResponse
GET/POST/ PUT/DELETE	/rd, /rd/{id}	RdRequest	RdResponse
POST	/rd/{id}/close	{ } (opt)	RdResponse
GET/POST/ PUT/DELETE	/loan, /loan/{id}	LoanRequest	LoanResponse
GET/POST/ PUT/DELETE	/epf, /epf/{id}	EpfRequest (no @Valid — inconsistent with FD/RD/Loan/Other)	EpfResponse

METHOD	PATH	BODY	RETURN
GET/POST/ PUT/DELETE	<code>/other</code> , <code>/other/{id}</code>	<code>OtherAssetRequest</code>	<code>OtherAssetResponse</code>

Reserved but unimplemented

`/api/admin/**` — configured in `SecurityConfig` as `hasRole("ADMIN")`, but **no controller maps under this path**. `[PARTIALLY IMPLEMENTED – dead security rule]`.

05

End-to-end flows

One concrete example traced start to finish, for each of the three critical paths.

01 Gmail to Portfolio

02 Transaction Processing

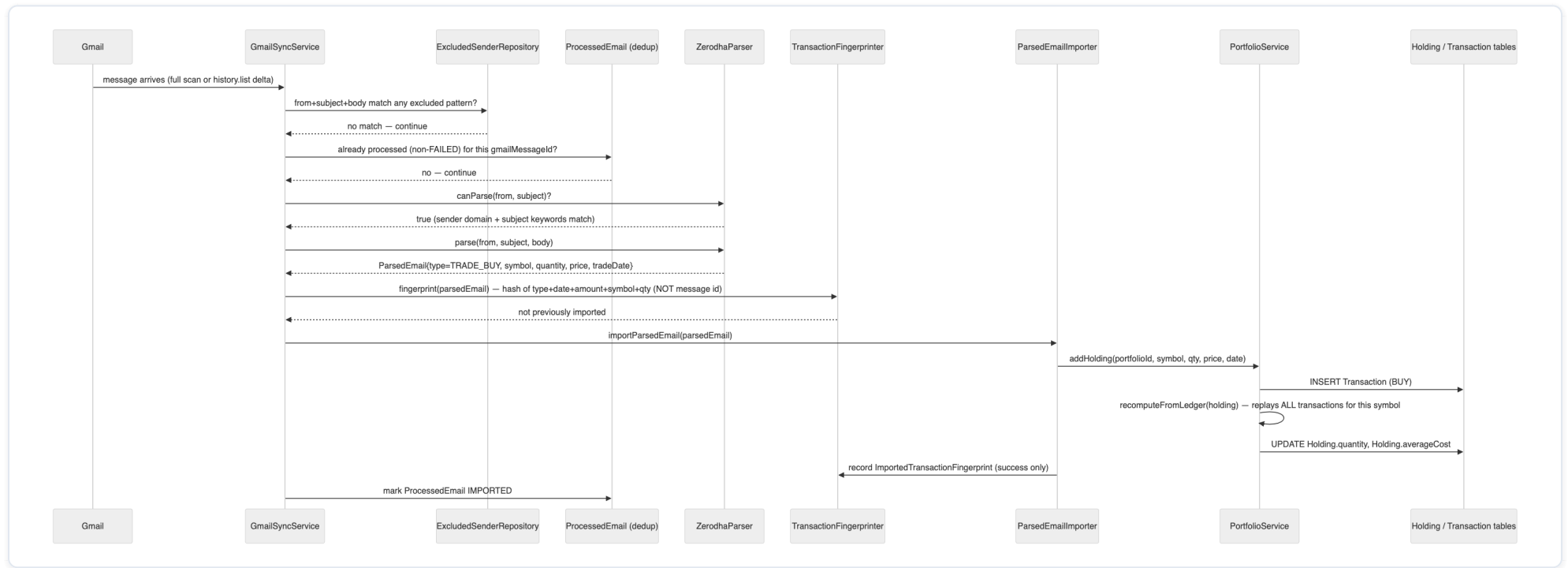
03 Card & AI Recommendations

Gmail to Portfolio

 [Source file · docs/05-flows/GMAIL_TO_PORTFOLIO_FLOW.md](#)

End-to-end trace of a single email from arrival to appearing as a holding. This is a condensed, narrative version of `02-backend/GMAIL_INGESTION_PIPELINE.md` — read that document for full file:line detail; this one is for tracing one concrete example.

Example: a Zerodha "trade confirmation" email for a stock buy



What happens if the parser doesn't recognize the email

If no parser's `canParse()` matches, and the email has no PDF attachment, and AI fallback is enabled:

`EmailIntelAgent` classifies it via the local LLM. If confidence ≥ 0.85 **and** the classified type is importable (not e.g. `INTERNAL_TRANSFER`), it's treated exactly like a parser's output from this point on — same fingerprint check, same `ParsedEmailImporter` call.

If confidence < 0.85 , or the type is non-importable, or the LLM call itself fails: the email becomes an

`EmailReviewItem` instead. **No `Transaction` is ever written for a reviewed-but-not- yet-decided item.** A human must `ACCEPT / EDIT / REJECT` it via `POST /api/review/{id}/decision` — an `ACCEPT / EDIT` decision then calls `ParsedEmailImporter` through the exact same path shown above, including the fingerprint check (so accepting a review item that was somehow also separately imported another way still cannot double-book it).

What happens if the same email arrives twice (forwarded, or a duplicate notification)

The Gmail message ID differs (a forward has a new ID), but `TransactionFingerprinter` hashes only the *financial content* (type+date+amount+symbol+quantity — not message ID or sender). The second occurrence's fingerprint matches the first, `ParsedEmailImporter` finds the existing `ImportedTransactionFingerprint` row, and short-circuits with a no-op **before** any table-specific routing runs. Nothing is written twice.

What happens if the holding already exists

`recomputeFromLedger` doesn't add to the existing `quantity / averageCost` directly — it re-reads **every** `Transaction` row for that holding (this new BUY plus every prior BUY/SELL) and recomputes both fields from scratch via weighted-average-cost replay. This is why a manual correction to a holding is never a direct field edit in this system — it must go through the ledger (a corrective `Transaction`), or the next sync/reconciliation pass will silently overwrite the manual correction back to what the ledger implies.

Where net worth picks this up

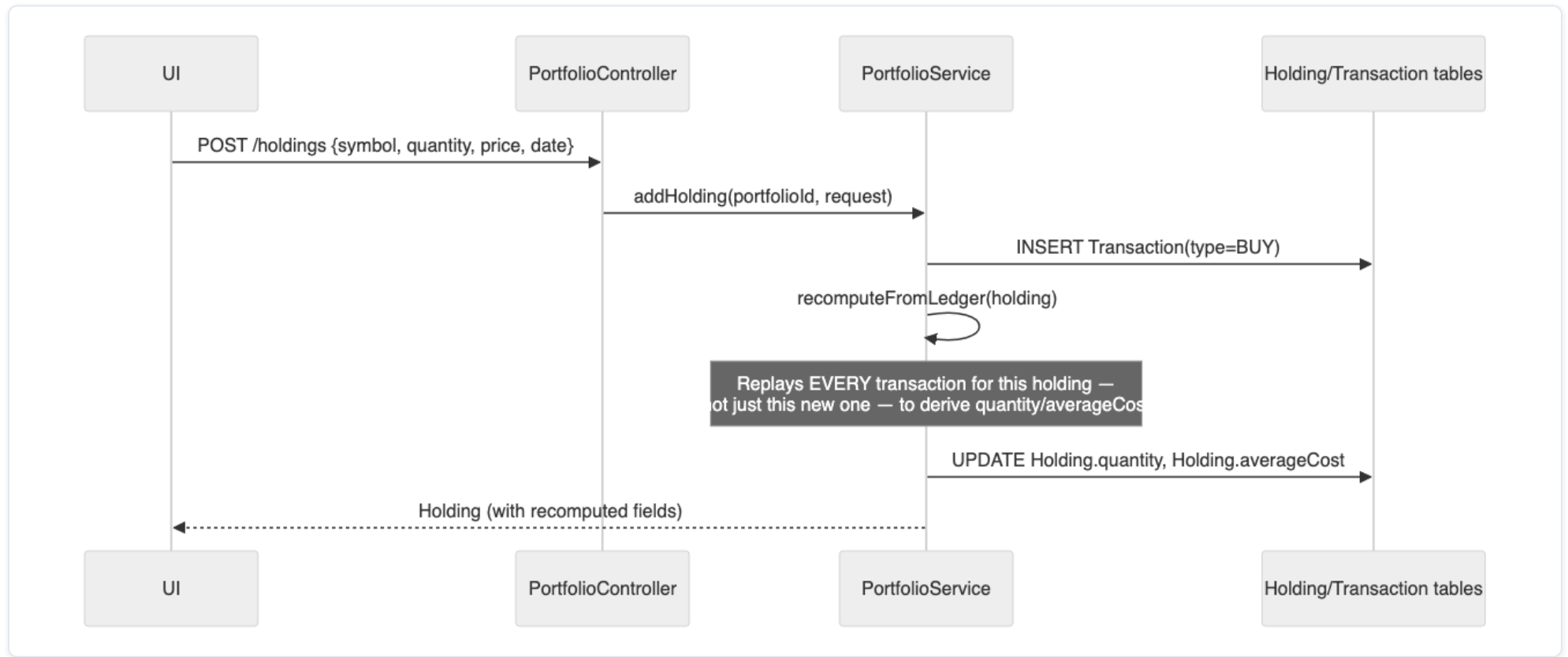
The new `Holding.currentValue` (quantity × latest market price) flows into

`PortfolioContextService.build()` the next time `/api/wealth/summary` is called — there is no separate "recompute net worth" step triggered by the import; net worth is always computed fresh from current holdings on request, never cached/stale from the import event.

Transaction Processing

 [Source file · docs/05-flows/TRANSACTION_PROCESSING.md](#)

Manual buy (via `POST /api/portfolios/{id}/holdings`)



Manual sell (via `POST /api/portfolios/{portfolioId}/holdings/{holdingId}/sell`)

```
sellHolding(portfolioId, holdingId, quantity, salePrice)
  → INSERT Transaction(type=SELL, quantity, salePrice)
  → recomputeFromLedger(holding):
      replay all transactions in date order
      each SELL reduces netQty and proportionally reduces totalCost
      (weighted-average-cost model, NOT FIFO/LIFO lot matching)
  → if netQty ≤ 0 after replay: DELETE the holding (fully sold)
  → else: UPDATE quantity/averageCost to the replayed values
```

Realized P&L is not stored on the `Transaction` row directly — it is derived at read time by comparing the sale price against the average cost *as of the moment of that sale* during the ledger replay, not queried from a stored "profit/loss" column.

Gmail-imported trade (see `GMAIL_TO_PORTFOLIO_FLOW.md` for the full trace)

Same `recomputeFromLedger` call, reached via `ParsedEmailImporter.importTrade` → `portfolioService.addHolding / sellHolding` instead of the controller. **The recompute logic is identical regardless of entry path** — manual entry and Gmail import both funnel through the same ledger-replay method, which is what guarantees they can never disagree about a holding's current quantity/cost.

What happens when a SELL has no matching holding

If `ParsedEmailImporter` processes a `TRADE_SELL` `ParsedEmail` and no `Holding` exists for that symbol (e.g., the corresponding BUY was never imported, or was for a security not tracked in this portfolio), it does **not** create a negative-quantity holding or silently drop the transaction. Instead: `importSellAsTransaction` books it as an `Income` row with `IncomeSource.CAPITAL_GAIN` — the money is recorded as realized income even though there's no holding to attribute it to.

Nightly reconciliation (drift detection)



This job exists because a holding's stored fields *should* always equal what the ledger implies — if they ever diverge (a bug, a direct DB edit, a migration artifact), this job silently corrects it and logs the fact that a correction happened. It does not alert externally; the log line is the only signal (see `04-devops/MONITORING_AND_LOGGING.md`).

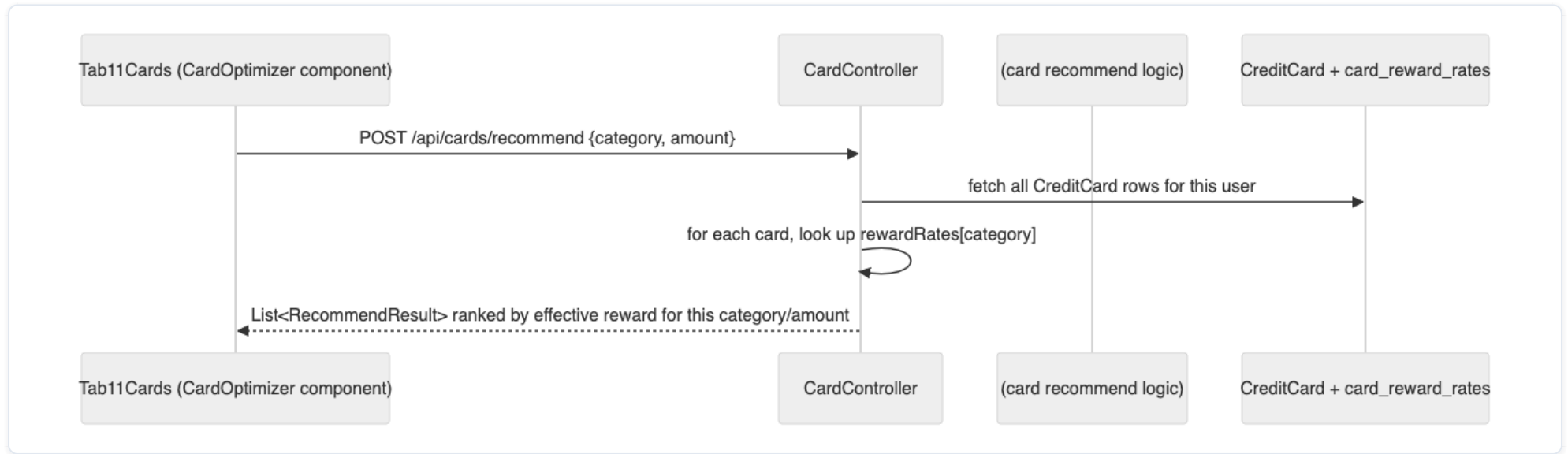
Manual holding correction (the one case that looks like a direct edit but isn't)

`PUT /api/portfolios/{portfolioId}/holdings/{holdingId}` accepts a direct `{quantity, averageCost, ...}` patch from the UI. This is the **one legitimate exception** to "never directly set Holding fields" — but even here, the underlying implementation still routes through the same ledger-consistency model conceptually: a manual correction is only meaningful until the next `recomputeFromLedger` pass (nightly reconciliation, or the next buy/sell on that holding) recomputes from the transaction history and overwrites it. If a manual correction needs to persist, the correct fix is to also correct the underlying `Transaction` history, not just the `Holding` snapshot.

Card & AI Recommendations

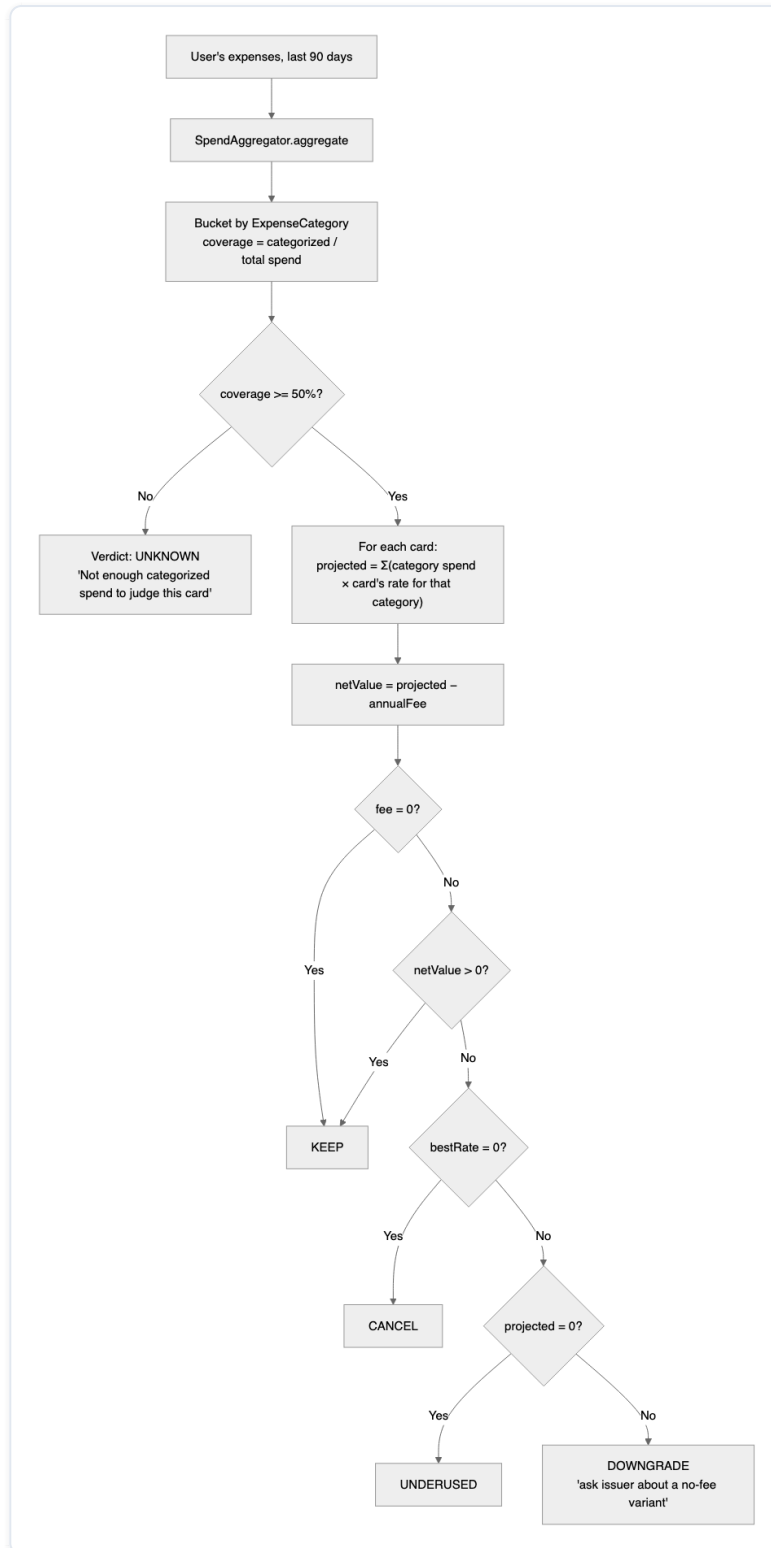
 Source file · docs/05-flows/CARD_AI_RECOMMENDATION_FLOW.md

"Which card should I use for this purchase?" (`POST /api/cards/recommend`)



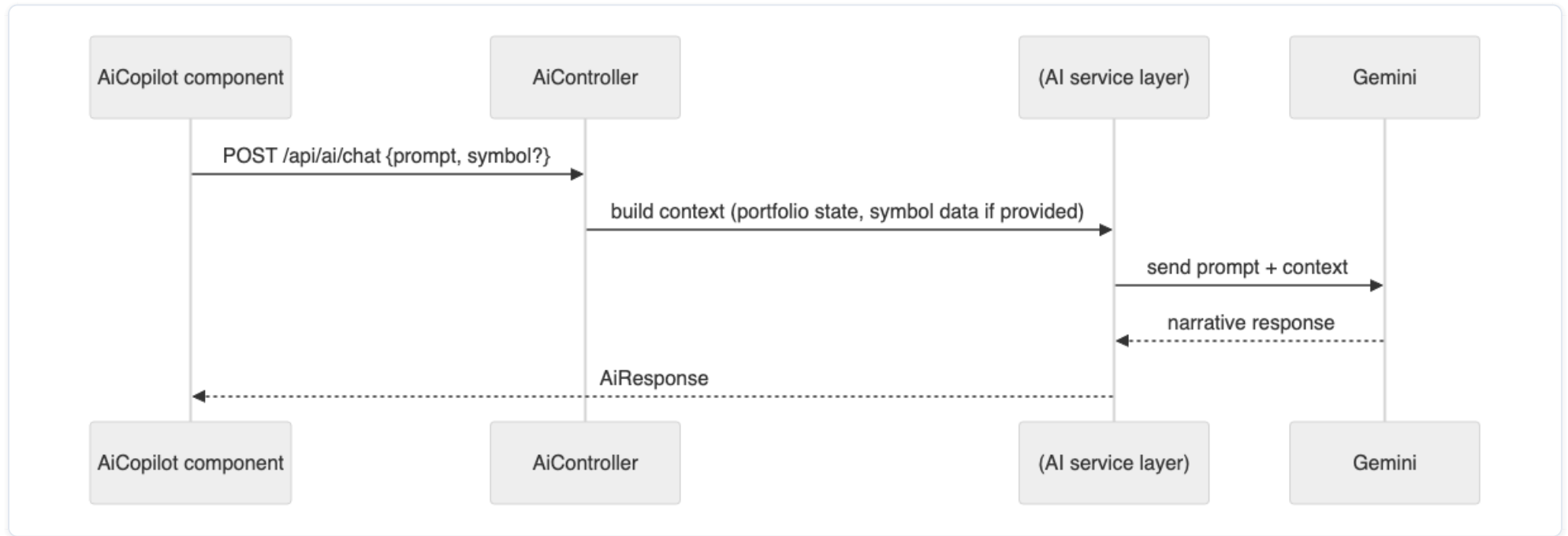
Reward rates come from `CreditCard.rewardRates` (an `@ElementCollection` map, category→rate) — manually maintained per card, not derived from any external catalog or issuer feed. A separate, more structured mechanism (`CardRewardRule`, effective-dated with `isStale()`) also exists in the schema for versioned rate history, but the simple `recommend` endpoint reads the live map on `CreditCard` directly.

"Should I keep, downgrade, or cancel this card?" (`CardOptimizerService.analyse`)



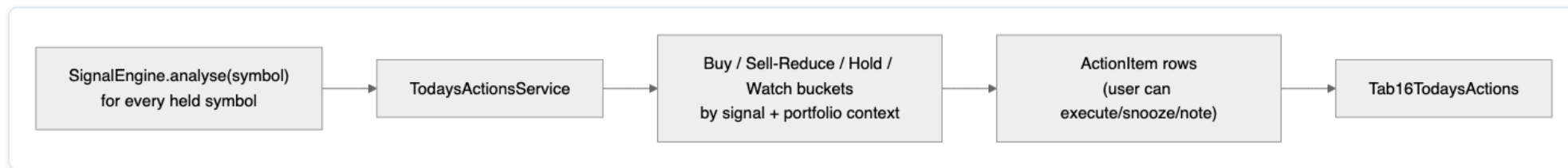
Key honesty constraints, both deliberate: 1. Categories are **inferred from merchant names**, never presented as real card-network MCC data — `SpendAggregator` marks every category `inferred=true`. 2. `rewardsRealized12m` is **always** `null` — the system does not yet have enough per-card reward-transaction history to report a *measured* figure, and the code deliberately refuses to substitute a projection for a measurement in that field.

AI Advisor chat / portfolio review (`aiApi.chat` , `aiApi.portfolioReview`)



This path is distinct from the local-first `EmailIntelAgent` / `LlmProviderRouter` used for Gmail classification (see `02-backend/GMAIL_INGESTION_PIPELINE.md` §6) — `AiController`'s chat/review endpoints call Gemini directly for narrative generation, not through the same provider-router abstraction used for structured email classification. Both ultimately depend on `GEMINI_API_KEY` being configured if Gemini is the active/only provider.

| Today's Investment Actions — the daily signal-to-action bridge



`ActionItem` records what the user *did* with a recommendation (executed/skipped/snoozed) — it does **not** place any trade. Marking an action "executed" is a manual acknowledgment; the actual portfolio change (if any) still has to come through the normal transaction path (manual entry or Gmail import) described in `TRANSACTION_PROCESSING.md`. This system never trades on the user's behalf.

06

Operations

Running it: environment, deployment, and what to watch once it's live.

01 DevOps Onboarding

02 Environment Variables

03 Deployment

04 Monitoring & Logging

DevOps Onboarding

 [Source file · docs/04-devops/DEVOPS_ONBOARDING.md](#)

Infrastructure overview

`docker-compose.yml` (repo root) — 4 services:

SERVICE	IMAGE	PORTS	HEALTHCHECK
<code>postgres</code>	<code>postgres:16-alpine</code>	<code>5432:5432</code>	<code>pg_isready</code>
<code>redis</code>	<code>redis:7-alpine</code>	<code>6379:6379</code>	<code>redis-cli ping</code>
<code>backend</code>	built from <code>./backend</code>	<code>8080:8080</code>	<code>wget --spider http://localhost:8080/actuator/health</code>
<code>frontend</code>	built from <code>./frontend</code>	<code>3000:80</code>	none defined

`backend` depends on `postgres` + `redis` via `service_healthy` conditions; `frontend` depends on `backend` with no healthcheck condition (plain dependency — frontend can start before backend is actually healthy).

Postgres initializes from `./database/schema.sql` then `./database/seed.sql` on first boot (mounted as `docker-entrypoint-initdb.d/01-schema.sql / 02-schema.sql`).

Named volumes: `postgres_data`, `redis_data`.

Prerequisites

TOOL	VERSION
Docker + Docker Compose	any recent
JDK	25 (for local backend dev outside Docker — see the Java-version mismatch note below)
Node	20 (matches <code>frontend/Dockerfile</code> 's <code>node:20-alpine</code>)
Maven	3.6+

⚠️ Known gap: no CI/CD pipeline exists

Repo-wide search confirmed: no `.github/workflows/*` (project-owned), no `Jenkinsfile`, no `.gitlab-ci.yml` anywhere in this repository. `[PLANNED / NOT IMPLEMENTED]`. Every build/test/ deploy today is manual. See `CI_CD_PIPELINE.md` for what would need to be built.

⚠️ Known gap: Java version mismatch

`pom.xml` requires/targets **Java 25** (`java.version / maven.compiler.release=25`). `backend/Dockerfile` builds and runs on `eclipse-temurin:21-*--alpine`. Verify which is authoritative before trusting the Docker build — as of this audit, building the current `pom.xml` inside that container image would fail unless the Dockerfile has since been bumped to a JDK 25 base image.

Quick start (Docker Compose, full stack)

```
docker compose up -d
```

Backend becomes healthy once `/actuator/health` responds (checked every 30s, 5 retries). Frontend serves the Vite production build via nginx once its container starts (no health-gated wait on the backend).

TLS truststore (corporate network only)

If backend outbound HTTPS calls (Yahoo Finance, AMFI, Gemini) fail with PKIX/SSL errors, your network likely intercepts TLS via a corporate proxy. Run:

```
backend/tools/regen-truststore.sh
```

This rebuilds `backend/truststore.p12` (the current JDK's own `cacerts` + your machine's trusted corporate root certs, exported from the macOS keychain by CN match). Re-run when the JDK is upgraded or the corporate CA rotates. See `pom.xml`'s `spring-boot-maven-plugin` config for the `-Djavax.net.ssl.*` JVM arguments this file supports — drop them entirely if you're not behind such a proxy.

Environment Variables

 Source file · docs/04-devops/ENVIRONMENT_VARIABLES.md

Extracted directly from `backend/src/main/resources/application.yml` (both `dev` and `prod` profile documents) plus `frontend/.env`. Every `${VAR:default}` pattern in the codebase is listed.

Critical caveat, verified in code: Spring's multi-document YAML means each `---`-separated profile document is **independent** — a key set only in the `dev` document has **no value at all** when running with `SPRING_PROFILES_ACTIVE=prod` unless it's also redefined in the `prod` document or has a `${VAR:default}` fallback inlined at the Java `@Value` use site. Several keys fall into this gap — marked below.

Backend

VARIABLE	DESCRIPTION	REQUIRED (Y/N)	DEFAULT	ROLE
<code>SPRING_PROFILES_ACTIVE</code>	Active profile	N	<code>dev</code>	Backend
<code>DB_HOST</code>	Postgres host	Y (prod) / N (dev)	<code>localhost</code>	Backend/ DevOps
<code>DB_PORT</code>	Postgres port	Y (prod) / N (dev)	<code>5432</code>	Backend/ DevOps
<code>DB_NAME</code>	Database name	Y (prod) / N (dev)	<code>marketai_db</code>	Backend/ DevOps
<code>DB_USER</code>	Database user	Y (prod) / N (dev)	<code>marketai</code>	Backend/ DevOps
<code>DB_PASSWORD</code>	Database password	Y (prod) / N (dev)	<code>marketai_pass</code>	Backend/ DevOps
<code>REDIS_HOST</code>	Redis host	Y (prod) / N (dev)	<code>localhost</code>	Backend/ DevOps
<code>REDIS_PORT</code>	Redis port	Y (prod) / N (dev)	<code>6379</code>	Backend/ DevOps
<code>REDIS_PASSWORD</code>	Redis password	Y (prod) / N (dev)	<code>""</code> (empty)	Backend/ DevOps
<code>JWT_SECRET</code>	JWT signing secret	Y (prod, no default) / N (dev)	dev: none — auto-generates and persists to gitignored <code>gmail.env</code> if unset	Backend

VARIABLE	DESCRIPTION	REQUIRED (Y/N)	DEFAULT	ROLE
<code>GEMINI_API_KEY</code>	Google Gemini API key	Y (prod, no default) / N (dev)	''	Backend
<code>NEWS_API_KEY</code>	NewsAPI key (prod news source)	Y (prod, no default)	— not present in dev block at all; dev uses free RSS feeds	Backend
<code>CORS_ALLOWED_ORIGINS</code>	Allowed CORS origins (dev only)	N	<code>http://localhost:3000,http://localhost:5173,http://localhost:5174</code>	Backend
<code>FRONTEND_URL</code>	Where OAuth callback redirects the browser; also prod's single CORS origin	Y (prod, no default) / N (dev)	dev: <code>http://localhost:5174</code>	Backend
<code>PDF_PASSWORD_ENC_KEY</code>	Base64 32-byte AES-256 key for encrypting saved statement passwords	N	random/ephemeral if unset (generate with <code>openssl rand -base64 32</code>)	Backend
<code>EMAIL_CLIENT_ID</code>	Google OAuth client ID	N (but Gmail sync won't work without it)	''	Backend
<code>EMAIL_CLIENT_SECRET</code>	Google OAuth client secret	N (same caveat)	''	Backend
<code>EMAIL_REDIRECT_URI</code>	Must exactly match the OAuth client's registered redirect URI	N	<code>http://localhost:8080/api/gmail/callback</code>	Backend
<code>EMAIL_PUSH_ENABLED</code>	Enable Gmail push (watch + Pub/Sub)	N	<code>false</code>	Backend
<code>EMAIL_PUSH_TOPIC</code>	Google Cloud Pub/Sub topic name	N (required if push enabled)	''	Backend
<code>EMAIL_PUSH_TOKEN</code>	Shared-secret query token for the push webhook	N (required if push enabled)	''	Backend
<code>LLM_PROVIDER</code>	<code>ollama</code> <code>gemini</code> <code>none</code>	N	<code>ollama</code>	Backend
<code>OLLAMA_BASE_URL</code>	Ollama server URL	N	<code>http://localhost:11434</code>	Backend

VARIABLE	DESCRIPTION	REQUIRED (Y/N)	DEFAULT	ROLE
<code>OLLAMA_MODEL</code>	Ollama model name	N	<code>qwen2.5:7b</code>	Backend

Backend — dev-only YAML keys with no `prod` document entry

These resolve purely to their Java-side `@Value("${x:default}")` inline default when `SPRING_PROFILES_ACTIVE=prod` — confirmed for the sync-worker settings; treat the rest as the same pattern unless verified otherwise:

KEY	EFFECTIVE PROD VALUE	SOURCE OF DEFAULT
<code>app.sync.worker.concurrency</code>	<code>2</code>	<code>SyncJobWorker.java</code> <code>@Value("\${app.sync.worker.concurrency:2}")</code>
<code>app.sync.worker.stale-after-minutes</code>	<code>90</code>	<code>SyncJobWorker.java</code> <code>@Value("\${app.sync.worker.stale-after-minutes:90}")</code>
<code>app.gmail.push.*</code> (all 3 keys)	as documented above	inline defaults
<code>app.llm.*</code> (all keys)	as documented above	inline defaults
<code>app.recommendation.weights.*</code>	technical=30, momentum=20, valuation=20, sentiment=30	only in <code>dev</code> document
<code>app.security.pdf-password-key</code>	random/ephemeral	only in <code>dev</code> document
<code>gmail.client-id</code> / <code>client-secret</code> / <code>redirect-uri</code> / <code>frontend-url</code>	as documented above	only in <code>dev</code> document
<code>springdoc.*</code>	Swagger UI paths	only in <code>dev</code> document
<code>spring.cache.type</code>	<code>simple</code>	only in <code>dev</code> document

Also has no YAML entry in either profile: `app.market.refresh-rate-ms` — driven entirely by the inline `@Scheduled(fixedRateString = "${app.market.refresh-rate-ms:900000}")` default (15 min) in `MarketDataService.java:204`. Cannot be tuned via config without a code change.

Backend — hardcoded, not configurable via env var

VALUE	WHERE
<code>app.yahoo-finance.base-url = https://query1.finance.yahoo.com</code>	both profiles
<code>app.yahoo-finance.timeout-seconds = 10</code>	both profiles
<code>app.gemini.base-url = https://generativelanguage.googleapis.com/v1beta</code>	both profiles
<code>app.gemini.model = gemini-1.5-flash</code>	both profiles
<code>app.jwt.expiration-ms = 86400000</code> (24h), <code>app.jwt.refresh-expiration-ms = 604800000</code> (7d)	both profiles
<code>app.llm.ollama.timeout-seconds = 120</code> , <code>app.llm.ollama.temperature = 0.0</code>	dev only (see gap above)
<code>app.llm.min-confidence = 0.85</code>	dev only (see gap above)
<code>hibernate.jdbc.time_zone = Asia/Kolkata</code>	dev only
<code>spring.jpa.hibernate.ddl-auto = update</code> (dev) / <code>validate</code> (prod)	both, different values

Frontend

VARIABLE	DESCRIPTION	REQUIRED (Y/N)	DEFAULT	ROLE
<code>VITE_API_URL</code>	Backend base URL	N	<code>http://localhost:8080</code>	Frontend
<code>BACKEND_URL</code>	Used by <code>docker-compose.yml</code> to set the frontend build's <code>VITE_API_URL</code>	N	<code>http://localhost:8080</code>	DevOps

Note: `VITE_API_URL` is baked into the static build at **build time** (Vite env vars are compile-time only) — changing it at runtime requires a rebuild, not just an env var change on the running container.

`.env.example` gap (verified against `application.yml`)

The repo-root `.env.example` declares: `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `REDIS_HOST`, `REDIS_PORT`, `REDIS_PASSWORD`, `JWT_SECRET`, `SPRING_PROFILES_ACTIVE`, `GEMINI_API_KEY`, `NEWS_API_KEY`, `FRONTEND_URL`, `BACKEND_URL`.

It **omits** (all of which `application.yml` reads): `GMAIL_CLIENT_ID`, `GMAIL_CLIENT_SECRET`, `GMAIL_REDIRECT_URI`, `GMAIL_PUSH_ENABLED`, `GMAIL_PUSH_TOPIC`, `GMAIL_PUSH_TOKEN`, `LLM_PROVIDER`, `OLLAMA_BASE_URL`, `OLLAMA_MODEL`, `PDF_PASSWORD_ENC_KEY`, `CORS_ALLOWED_ORIGINS`. [PARTIALLY

IMPLEMENTED – a new engineer following `.env.example` alone will hit silent defaults for all of these.]

Deployment

 [Source file · docs/04-devops/DEPLOYMENT_GUIDE.md](#)

⚠ Read this first: Java version mismatch

`pom.xml` declares `java.version / maven.compiler.release=25`. `backend/Dockerfile` builds and runs on `eclipse-temurin:21-*-*alpine`. **Verify which is authoritative before deploying** — as audited, the Maven build inside that container image would fail against the current `pom.xml` unless the Dockerfile has since been updated to a JDK 25 base image. This is not a hypothetical edge case; it's a directly observed discrepancy between two files in the repo.

Docker images (verified contents)

BACKEND/DOCKERFILE (MULTI-STAGE)

```
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN apk add --no-cache maven && mvn clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=build /app/target/*.jar app.jar
RUN chown appuser:appgroup app.jar
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-XX:+UseContainerSupport", "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]
```

Runs as non-root `appuser`. JVM flags scope container memory (`UseContainerSupport / MaxRAMPercentage=75.0`). **No `truststore.p12` is copied into the image** — the corporate-proxy TLS JVM args in `pom.xml`'s `spring-boot-maven-plugin` are a local build-time concern, not carried into the container's runtime `ENTRYPOINT`. If the container's egress also passes through a TLS-inspecting proxy, this would need to be added.

FRONTEND/DOCKERFILE (MULTI-STAGE)

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL=http://localhost:8080
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

`VITE_API_URL` is a **build-time ARG** — baked into the static JS bundle. Changing the backend URL for an already-built image requires a rebuild with a new `--build-arg`, not an env var change on the running container.

Docker Compose (local/single-host deployment)

```
docker compose up -d
```

See `DEVOPS_ONBOARDING.md` for the full service table. This is the only orchestration artifact in the repo — **no Kubernetes manifests, Helm charts, or cloud IaC (Terraform/CDK/Pulumi) were found**. A production deployment target beyond a single Docker host is `[PLANNED / NOT IMPLEMENTED]`.

Database migrations

`spring.jpa.hibernate.ddl-auto`: - `dev`: `update` — Hibernate auto-migrates the schema on startup. - `prod`: `validate` — **Hibernate will refuse to start if the schema doesn't already match the entity model**. Production schema must be migrated by some other means before deploying new entity changes — no Flyway/Liquibase migration tool was found in `pom.xml`'s dependencies, and `database/schema.sql` / `seed.sql` are only wired into the Postgres container's *first-boot* init scripts (`docker-entrypoint-initdb.d`), not a repeatable migration tool.

Practical implication: deploying a change that adds/modifies an entity to a prod-profile environment requires manually applying the equivalent DDL to the production database before (or as part of) the deploy — there is currently no automated migration step.

Environment configuration for prod

See `ENVIRONMENT_VARIABLES.md` for the complete matrix, including which variables have **no default in the prod YAML profile document at all** (several `app.sync.*`, `app.gmail.push.*`, `app.llm.*` keys) — these silently fall back to whatever default is hardcoded at the Java `@Value` annotation site, not to anything in

`application.yml`'s `prod` block. Confirm each of these resolves to an intentional value before a production deploy, since the YAML file alone does not make this visible.

Health check

`GET /actuator/health` — used by Docker Compose's `backend` healthcheck (`wget -q --spider http://localhost:8080/actuator/health` , every 30s, 5 retries) and is `permitAll` in `SecurityConfig` (no auth required).

Monitoring & Logging

 [Source file · docs/04-devops/MONITORING_AND_LOGGING.md](#)

Health checks

GET /actuator/health — permitAll (no auth). Used by: - docker-compose.yml 's backend healthcheck (30s interval, 5 retries). - Manual verification during onboarding (curl http://localhost:8080/actuator/health).

No other actuator endpoints (/actuator/metrics , /actuator/info , etc.) were confirmed as exposed — springdoc / spring-boot-starter-web is present but a broader actuator surface was not verified as enabled beyond /actuator/health appearing in the SecurityConfig allowlist. [PARTIALLY IMPLEMENTED / NOT VERIFIED — confirm actuator exposure config (management.endpoints.web.exposure.include) if deeper observability is needed; no such property was found in application.yml , meaning Spring Boot's actuator defaults apply (health only, by default, beyond what's explicitly enabled).]

Logging configuration (application.yml)

PROFILE	COM.MARKETAI LEVEL	ORG.SPRINGFRAMEWORK.SECURITY	ROOT
dev	DEBUG	INFO	(unset — Spring default)
prod	INFO	(unset)	WARN

Console pattern (dev): %d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n .

No structured/JSON logging, no log aggregation sink (no Logback appender for a log shipper, no ELK/Loki config) was found — logs go to stdout/console only, which is standard for a container-orchestrated deployment (Docker/K8s log drivers pick up stdout) but means any centralized log search depends entirely on whatever the deployment platform provides around the container, not on anything configured in this codebase.

What the system logs today (application-level signal, not infra monitoring)

These are the built-in "loud failure" points already in the code — treat them as the monitoring signals worth alerting on until real observability tooling is added:

SIGNAL	WHERE	WHY IT MATTERS
"AMFI NAV refresh: X bytes returned but no scheme rows parsed"	<code>AmfiNavService.refresh()</code>	MF NAV data silently stopped updating
Gmail watch registration failure	<code>GmailWatchService.registerWatch</code> (ERROR level)	Push notifications will lapse — watches expire after 7 days with no error from Gmail itself , so this app-level log is the only signal
<code>SyncJob</code> terminal <code>FAILED</code> status with <code>lastError</code>	<code>sync_jobs</code> table, queryable via <code>GET /api/sync/jobs</code>	A Gmail sync exhausted its 3 retries
"Worker did not finish; exceeded retry budget"	<code>SyncJobService.requeueStale</code>	A sync worker died mid-job and the job couldn't be salvaged
Per-scheme/per-ticker isolated failures	<code>MfNavHistoryScheduler</code> , <code>MarketDataService.scheduledSectorIndexHistoryRefresh</code>	One bad symbol doesn't abort the batch, but repeated single-symbol failures are worth surfacing
<code>AiAuditTrail</code> rows with <code>status != ACCEPTED</code>	<code>ai_audit_trail</code> table	Every rejected/low-confidence AI classification is recorded — a spike here signals either a parser regression or a genuinely unusual batch of emails

Recommended additions (not yet built)

- Wire `management.endpoints.web.exposure.include` to add `/actuator/metrics`, `/actuator/prometheus` (if adopting Prometheus) for real infra-level monitoring. `[PLANNED / NOT IMPLEMENTED]`.
- Alert on the `SyncJob` `FAILED`-status count and on `GmailWatchService` renewal failures — both already produce the right log signal, just nothing consumes it externally today.
- Structured logging (JSON) would make the existing signal list above machine-parseable by a log aggregator without code changes beyond the Logback encoder config.

07

Why it is built this way

The competitive research and gap analysis behind every design choice above.

01 Research: How To Read It

02 Product Gap Matrix

03 Technical Gap Matrix

Research: How To Read It

 [Source file · docs/research/README.md](#)

Competitive and technical research underpinning the Wealth OS product upgrade. Produced during the RESEARCH → AUDIT → BENCHMARK phases, **before** any design or code.

Read in order:

#	DOCUMENT	WHAT IT ANSWERS
1	COMPETITIVE_RESEARCH.md	What competitors actually do, how, and with what data. Includes the COPY / IMPROVE / DIFFERENTIATE / REJECT disposition for every feature studied (§9), India tax rules (§10), and every unverified claim (§11)
2	COMPETITOR_FEATURE_MATRIX.md	Product · Feature · How it works · Data required · Wealth OS equivalent · Our improvement
3	PRODUCT_GAPS.md	Area · Wealth OS Today · Competitors · Gap · Priority (P0–P3)
4	TECHNICAL_GAPS.md	System · Current · Problem · Industry Pattern · Proposed Architecture

How to read the labels

Every research claim carries its evidence grade, preserved from the source agent:

- **[A]** documented, with a URL or a live HTTP probe
- **[B]** inference from observable product behaviour
- **[C]** speculation

Repository claims cite `file:line` and were verified against the code, not assumed. Implementation status uses the same tags as the rest of `/docs`: `[IMPLEMENTED]` · `[PARTIALLY IMPLEMENTED]` · `[PLANNED / NOT IMPLEMENTED]` · `[NOT BUILT]`.

Standing constraints these documents serve

Do not fabricate, estimate, duplicate, silently drop, or overwrite financial records.

EVERY FINANCIAL NUMBER MUST BE TRACEABLE. EVERY RECOMMENDATION MUST HAVE A REASON. EVERY IMPORT MUST BE RECONCILABLE. EVERY DECISION MUST USE VERIFIED DATA.

Two research findings map directly onto these:

- **Grounded verbatim source spans** (COMPETITIVE_RESEARCH §6.3) are the mechanism that makes "every financial number traceable" true at the field level rather than the email level.
- **Binary pass/fail checks** (§2.5, Simply Wall St) are what makes "every recommendation has a reason" explainable — a weighted z-score composite cannot be explained; "failed 4 of these 6 named checks" can.

Status

Research phase **complete** — 14 agents, all reported. Design phase (docs/architecture/) not yet started.

No code has been written against these findings.

Product Gap Matrix

 Source file · docs/research/PRODUCT_GAPS.md

"Wealth OS Today" is verified against the code as of this audit — file:line cited, not assumed. "Competitors" is sourced from `COMPETITIVE_RESEARCH.md`.

Priority definitions (from the spec)

	SCOPE
P0	Financial truth · Gmail ingestion · dedup · reconciliation · portfolio + net-worth correctness
P1	Investment + MF + card intelligence · portfolio risk · Action Center
P2	Advanced AI · tax · scenario · goals
P3	Nice-to-have

A gap is **P0 only if getting it wrong shows the user a wrong number**. Several attractive features below are deliberately P1/P2 despite being differentiators, because they are additive rather than corrective.

P0 — Financial truth

AREA	WEALTH OS TODAY	COMPETITORS	GAP	PR
Ledger as single writer	✅ Intact. <code>Holding.quantity</code> / <code>averageCost</code> written only at <code>PortfolioService.java:258–259</code> inside <code>recomputeFromLedger</code>. Verified: no other writer exists	Zerodha Console contradicts itself between two XIRR surfaces	None — this is our strongest asset. Protect it with a test that fails if a second writer appears	P0 (g)
Net-worth attribution	⚠️ <code>NetWorthSnapshot</code> does store a dated series (<code>snapshotDate</code> , <code>totalAssets</code> , <code>netWorth</code>) via <code>NetWorthService.record()</code> . What's absent is decomposition — no contribution/ withdrawal/ revaluation fields	Nobody ships it. Modified Dietz / TWR / MWRR are standard but unused in Indian retail	"Your net worth rose ₹X: ₹A saved, ₹B market, ₹C spent." The hard part — transfer-pair detection — is already solved by <code>LedgerTransferService</code>	P0
Return-metric correctness	⚠️ <code>XirrCalculator</code> exists; one metric regardless of horizon; no benchmark-XIRR	Varsity/Kuvera publish a horizon→metric decision table; Zerodha suppresses XIRR under 1yr	Wrong metric for the horizon is a wrong number. Also: switches must be paired outflow/inflow or they double-count ; XIRR can have multiple roots on SIP+STP chains	P0

AREA	WEALTH OS TODAY	COMPETITORS	GAP	PRIORITY
Refuse-to-answer	⚠️ Partial — <code>SignalEngine.java:275</code> sets <code>limitedBy("NOT_SIZED")</code> , but no general mechanism	VR publishes 5 rating exclusions ; Zerodha shows "-" rather than a misleading XIRR	No first-class <code>DataSufficiency</code> type. Every computed number should be able to decline	P0
Reconciliation coverage	⚠️ <code>ReconciliationService.checkAll</code> covers 3 of ~17 listed checks	Industry standard: matched / unmatched / partially-matched / typed exceptions	14 checks missing; no exception typing; no partial-match concept	P0
Economic identity (cross-source dedup)	⚠️ <code>TransactionFingerprinter</code> hashes financial content, correctly excluding message id — survives forwards	Plaid models restatement via <code>pending_transaction_id</code> and admits matching sometimes fails	Same trade via contract-note PDF and broker email has different bytes → content hash can't match. Need composite key + tolerance, and UTR/RRN harvesting to make it exact	P0
Source traceability	❌ No grounded source spans. Numbers can be traced to an <i>email</i> , not to a <i>span within it</i>	Nobody does this either	"EVERY FINANCIAL NUMBER MUST BE TRACEABLE" is not fully satisfied by email-level provenance	P0
Extraction confidence	⚠️ Flat 0.85 gate on model-reported confidence	Research: logprobs give only 0.705 ROC AUC on DocILE	We gate on the signal shown to be weak. Need arithmetic validation + parser/LLM agreement	P0
Ingestion status model	⚠️ <code>ProcessedEmail</code> + <code>PendingPdf</code> + <code>EmailReviewItem</code> cover the states informally	—	Spec requires 10 explicit statuses (DISCOVERED... REQUIRES_REVIEW) as one observable model	P0
Review queue	✅ <code>EmailReviewItem</code> + <code>ReviewStatus</code> + decision endpoint	No evidence any Indian app offers one	None — we're ahead. Surface it as a feature, not plumbing	P0
AMFI feed	✅ Fixed — binds columns by header name	—	Action: point at <code>portal.amfiindia.com</code> directly rather than relying on the 302 from <code>www.</code>	P0

P1 — Intelligence

AREA	WEALTH OS TODAY	COMPETITORS	GAP	PRIORITY
Stock factor model	❌ Wrong. <code>AnalystService.java:142</code> = tech 30 + mom 20 + val 20 + sentiment 30 . No quality factor, no growth factor	VR: Quality 25 / Growth 20 / Valuation 35 / Momentum 20, sentiment not used at all	Missing the two factors that most determine whether a business is worth owning; 30% on the softest signal	P1 (top)

AREA	WEALTH OS TODAY	COMPETITORS	GAP	PRIORITY
Factor multicollinearity	✗ Momentum-family indicators pairwise $p > 0.9$ → momentum effectively ~4:1 over-weighted	Measured: HML–CMA 0.68 drives FF's own redundancy finding; RMW/ CMA/UMD are the near- orthogonal trio	Naive per-indicator averaging double-counts. And accruals recur in Piotroski, Beneish and QMJ — triple-counted unless consciously slotted once	P1
Sector-aware scoring	✗ One uniform formula for every stock	VR and Simply Wall St independently use a separate ratio set for banks/financials	Applying a generic formula to a bank is simply wrong — ROA/ NIM/provision coverage/ CAR/SGR gap instead	P1
Rating eligibility	✗ Scores everything	VR excludes: untraded last month, <3y history, negative net worth, accumulated losses, bottom 1% mcap	No "refuse to rate" path	P1
Red flags	✗ Not modelled	Tickertape scores them as a separate channel ; Zerodha Nudge fires them pre- trade	ASM/GSM/pledge/ default-probability should veto , never contribute a weighted score	P1
Explainable thesis	⚠ <code>Factor</code> list with notes exists	VR separates free quant rating from paid Stock Advisor — they don't pretend the score is a thesis	Need the same separation: decomposed factors = screen; thesis = a different artifact	P1
Position sizing	✗ <code>SignalEngine.java:275</code> deliberately <code>suggestedAmount(null)</code> , <code>limitedBy("NOT_SIZED")</code>	—	Honest placeholder, but the spec wants sizing first-class	P1
Uncertainty-scaled conviction	✗ Not modelled	Morningstar: 5- star discount 20/30/40/50/75% by uncertainty tier	Margin of safety should scale with uncertainty. Drive it off data quality we can measure , not analyst judgement we lack	P1

AREA	WEALTH OS TODAY	COMPETITORS	GAP	PRIORITY
MF overlap	✗ Not implemented. <code>PortfolioContextService.java:227</code> correctly documents why; <code>mf/entity/</code> contains only <code>MfNavHistory.java</code>	Advisorkhoj (counts, no methodology), Tickertape (MF-only)	Feasible: 3–6 weeks + 1–3 days/month maintenance. The comment stays accurate until built	P1
Direct stocks ↔ MF overlap	✗	Nobody	Unoccupied, and it is exactly our user's situation	P1
Card: cap counters	✗ <code>CardRewardRule.monthlyCapAmount</code> exists but no <code>UtilizationCounter</code>	FinArt ranks by ₹ after caps	Four counters on four clocks. Without them the ranking is wrong whenever a cap is near	P1
Card: MCC	✗ Zero MCC files. <code>SpendAggregator</code> infers category from merchant name, honestly marked <code>inferred=true</code>	FinArt chose category-level deliberately; ccreward uses MCC	Honest today. MCC would improve precision but FinArt's reasoning — issuers rarely publish the map — applies to us too	P1
Card: UPI/ RuPay gate	✗ Not modelled	Nobody models it well either	Only RuPay works on UPI; P2P/C2C/P2PM/ MF/IPO blocked. Recommending a Visa card for a UPI rail is a correctness bug	P1
Card: exclusions	✗ Zero exclusion files	Kotak publishes five lists for five purposes	One exclusion list per card is the most common modelling error . Need <code>EligibleSpendSet</code> keyed by purpose	P1
Card: point valuation	⚠ Single rate	Most tools use best-case transfer value as headline	Axis runs two currencies with a ~5x gap . Need a vector by redemption route	P1
Portfolio concentration	⚠ <code>AssetAllocationRing</code> displays, doesn't alert	Zerodha's new 50% single-stock/sector nudge	Needs look-through — direct + MF + smallcase combined	P1
Risk metrics	✗ None at portfolio level	VR publishes 15; Kuvera/ INDmoney publish none	Lead with max drawdown + downside capture. Alpha/beta/R ² on 36 observations are uninformative	P1

AREA	WEALTH OS TODAY	COMPETITORS	GAP	PRIORITY
Rebalancing triggers	✗ Display only	Kuvera: 5% band, "deviation beats time-based"	Vanguard supports a 200/175bp trigger/destination rule	P1
Benchmarks	✗ No benchmark model	SEBI two-tier TRI; AMFI publishes the tier-1 mapping table	Three peer-group definitions exist and disagree for flexicap/multicap/BAF. Store all three	P1
Action Center	✓ <code>ActionItem</code> + status + endpoints	—	Present; needs the new signal contract wired in	P1 (extend)
Data health	⚠ 3 of ~17 checks	—	Same gap as reconciliation, user-facing	P1

P2 — Tax, scenario, goals

AREA	WEALTH OS TODAY	COMPETITORS	GAP	PRIORITY
Capital-gains engine	⚠ <code>tax/service/TaxService.java</code> exists and is already on the correct rates — 12.5% LTCG above ₹1,25,000, 20% STCG, applied to the FY aggregate . But it estimates a <i>range</i> from <code>Income</code> rows grouped by source string, with no lot-level LT/ST split. Honestly labelled "an estimate range, not tax advice"	Zerodha TLH report; Kuvera Tax Harvesting (Kuvera's page still shows the stale ₹1 lakh)	Needs lot-level LT/ST classification, grandfathering, exit load, surcharge/cess. The rate constants and per-FY aggregation are already right — do not "fix" them	P2 (correctness-critical)
Statutory renumbering	✗	—	Income-tax Act 2025 replaces the 1961 Act from 1 Apr 2026: 111A→196, 112A→198. Rates unchanged; store section refs as data, not literals	P2
Grandfathering / surcharge / cess	✗	—	Cost = max(actual, FMV 31 Jan 2018), capped at sale price. Surcharge capped 15% + 4% cess — a bare 12.5% is wrong for high earners	P2

AREA	WEALTH OS TODAY	COMPETITORS	GAP	PRIORITY
Specified-MF classifier	✗	—	The most error-prone rule in the set — definition changed from "≤35% equity" to ">65% in debt/MMI" from FY2025-26. Verify against the Finance (No.2) Act 2024 before coding	P2
Exit load	✗	—	Not standardized — per-SID, SEBI-capped at 2%, credited back to the scheme. Store per scheme; apply FIFO at unit-lot level	P2
Tax-gain harvesting	✗	Kuvera: ₹15,625/yr	The Indian play inverts the US default — realise LTCG up to ₹1.25L tax-free and rebuy. No wash-sale rule makes it viable (verify independently)	P2
Deferral prompt	✗	Nobody ships it	"Wait N days and this lot turns long-term / exits the load window, saving ₹X" — the highest-value output of the whole tax feature	P2
Tax-aware rebalancing	✗	Nobody in India	Unoccupied	P2
FD/RD lifecycle	✓ Implemented + tested	—	Present	—
MF profit-booking engine	✗	—	Depends on the tax engine landing first	P2

P3 — Nice-to-have

AREA	WEALTH OS TODAY	COMPETITORS	GAP	PRIORITY
smallcase overlap	✗	Verified absent from smallcase, structurally unlikely to appear	Constituents are public via rebalance CSVs. Real differentiator, but only if the user holds smallcases	P3

AREA	WEALTH OS TODAY	COMPETITORS	GAP	PRIORITY
Card-terms changelog	✗	TechnoFino's forum is the only structured change-tracking in the market	Public, dated, provenance-stamped changelog = cheap trust moat	P3
Investor-return gap	✗	Morningstar <i>Mind the Gap</i>	"Your timing cost you X%" — behavioural insight	P3
Forward P/E, PEG	✗	Tickertape, Trendlyne (licensed)	REJECTED for v1 — not obtainable free. Shipping it would mean fabricating or scraping stale estimates	Rejected
Account Aggregator	✗	Jupiter (FIU via Finvu)	REJECTED — retracted from the roadmap. Regulated-entity gate, ₹5–25 lakh, cards excluded entirely	Rejected
SMS parsing	✗	axio, CheQ (outsourced)	REJECTED — Google restricts to default handlers; dead end for a new entrant	Rejected

What changed in our own recommendations

Recorded so the reversals are explicit rather than silent.

EARLIER POSITION	NOW	WHY
Account Aggregator is the highest-value roadmap item	Retracted	FIU requires a regulated entity, ₹5–25 lakh first-year, no hobbyist tier — and credit cards are excluded from AA entirely
—	Dhan is the highest-value data integration	Free API with account, daily history back to inception . Kite charges ₹500/mo for the same data
MF overlap may be infeasible	Feasible — 3–6 weeks	SEBI mandates free monthly disclosure. Cost is ETL + scheme-master, not access
MF Central/CAMS could supply user holdings	Closed	AMFI directed MF Central to stop sharing investor data with third-party apps (Sept 2025). CAS-first is now the only viable path — which is what we already built
Zerodha rates stocks	Wrong	Zerodha rates nothing. That's Tickertape , owned by smallcase; Rainmatter is an <i>investor</i> in smallcase, not its owner

Technical Gap Matrix

 [Source file · docs/research/TECHNICAL_GAPS.md](#)

Systems-level counterpart to `PRODUCT_GAPS.md`. "Current" is verified against the code; "Industry Pattern" is sourced from `COMPETITIVE_RESEARCH.md`.

"Proposed Architecture" is a recommendation, not a decision. Per the spec's ordering (RESEARCH → AUDIT → BENCHMARK → **DESIGN** → IMPLEMENT), these feed the design docs under `docs/architecture/` and are not yet committed to.

1. Document & ingestion layer

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Document abstraction	No generic <code>Document</code> entity. Only <code>PendingPdf</code> . 18 parsers, each with independent logic	Adding an issuer means a new parser class; no shared lifecycle, status, provenance or retry. Nothing enumerates "all documents in flight"	Classify → route → extract, with the document as a first-class entity carrying <code>(doc_type, issuer, confidence)</code>	<code>FinancialDocument</code> root entity with the 10 spec'd statuses ; parsers become <code>DocumentExtractor</code> strategies registered against <code>(issuer, doc_type)</code> ; <code>PendingPdf</code> becomes a projection
Classification	<code>canParse(from, subject)</code> polled across 18 parsers	O(n) predicate scan; no confidence; no fallback ordering; adding a parser can silently shadow another	Cheap cascade: sender domain + DKIM → subject regex → attachment MIME/producer → first-page fingerprint → model last . Jupiter's published query is stages 1–2 only	Ordered <code>ClassificationStage</code> chain returning <code>(type, issuer, confidence, matchedStage)</code> ; model invoked only when earlier stages abstain

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Extraction confidence	Flat 0.85 gate on model-reported confidence	Research shows logprobs give only 0.705 ROC AUC on DocILE — "a frontier LLM confidently transcribing OCR noise produces high log-probabilities for a wrong answer"	Multi-signal: cross-field arithmetic validation, deterministic-parser/LLM agreement, self-consistency across passes, grounding checks	<code>ConfidenceVector</code> { <code>arithmetic</code> , <code>parserAgreement</code> , <code>selfConsistency</code> , <code>grounding</code> } with an explicit combination rule; calibrate the threshold against a real sample before go-live , don't pick 0.85 by feel
Traceability	Values trace to an <i>email</i> , not a <i>span</i>	"EVERY FINANCIAL NUMBER MUST BE TRACEABLE" is not satisfied by email-level provenance	Mandatory verbatim source spans , mechanically verified to exist in the source text. Buys hallucination defence and user-facing provenance in one move	Every extracted field carries { <code>value</code> , <code>sourceSpan</code> , <code>page</code> , <code>bbox?</code> }; extraction fails if the span isn't found in the source. Non-negotiable
CAS parsing	Generic PDF path	Reinventing a solved format; CAS is high-volume and structured	<code>casparser</code> (MIT) parses CAMS + KFinTech + NSDL/CDSL eCAS to typed models	Dedicated CAS extractor (port or shell out); reserve the LLM pipeline for the long tail
PDF passwords	<code>SavedPdfPassword</code> + <code>PendingPdf</code> retry	One global strategy	Per-issuer rules: CAS = PAN for all types except KFinTech (user-defined); HDFC = Customer ID; ICICI = DOB/PAN fragments; SBI netbanking = account no. but YONO = DOB	<code>PasswordCandidateProvider</code> per issuer, tried in order, user-prompt on exhaustion. Never a deterministic generator for user-facing secrets — that constraint is separate and absolute
OCR	Not gated	—	Gate on text-layer presence: chars-per-page below threshold → OCR	Rare path for Indian statements (overwhelmingly text-layer + ruled tables)

2. Identity, dedup & reconciliation

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Ingest identity	<code>TransactionFingerprinter</code> — SHA-256 over financial content, correctly excluding Gmail message id so forwards collide	Good, but single-granularity: no notion of row-within-document	Stripe: compare params to original, error on mismatch, cache results including errors . TigerBeetle: transfer id <i>is</i> the idempotency key	Extend to <code>SHA-256(gmail_message_id attachment_hash row_index)</code> at the ingest layer, keeping the existing fingerprint at the economic layer. Two hashes, two purposes
Economic identity	Content hash only	Same trade via contract-note PDF and broker email has different bytes — the hash cannot match them. This is the central dedup gap	Composite natural key with tolerance: <code>(account, instrument, direction, quantity, amount ± ε, date ± N days)</code> . Where a strong external reference exists — UTR, ISO 20022 <code>EndToEndId</code> , RRN — prefer it and skip fuzzy matching entirely	<code>FinancialEventId</code> with a three-tier resolution external reference → <code>contract_note_id</code> + key + tolerance → fuzzy match. Harvest UTR/RRN aggressively — it is the highest-value signal on the page and turns fuzzy into exact
Restatement	Not modelled — a corrected document would be a new row or a conflict	No way to express "the same event, restated"	Plaid: <code>pending_transaction_id</code> links the posted row to its pending version; pending appears in <code>removed</code> , posted in <code>added</code> . And Plaid admits matching sometimes fails	<code>supersedes_id</code> links the restatement and links to the failure path explicitly because matching will fail
Reconciliation	<code>ReconciliationService.checkAll</code> — 3 of ~17 checks; binary pass/fail	No exception typing, no partial matches, no investigation payload	matched / unmatched / partially-matched / exceptions , typed by cause (timing difference, amount variance, unidentified). Banding ≥95 auto / 85–94 auto+flag / 70–84 human / <70 distinct	<code>ReconciliationContext</code> registry (17+ checks); <code>ReconciliationError</code> with <code>type</code> , <code>score</code> , <code>source_rows</code> needed to investigate. Also adopt matching (blocking) vs reconciliation (commit, comparative) controls
Ledger invariant	✅ Intact — <code>Holding.quantity / averageCost</code> written only at <code>PortfolioService.java:258–259</code>	The invariant is enforced by convention, not by a test	—	Add an ArchUnit/reflection test that fails the build if a writer appears. This is the strongest asset and it is currently undefended

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Event sourcing	CRUD + ledger replay	Replay is already event-sourcing-shaped for holdings	Double-entry "turns tampering into an arithmetic alarm." But Azure: projections become effectively immutable — <i>"even a simple bug fix can cause a butterfly effect as it runs against every transaction since the beginning of time"</i>	Follow the published guidance: event-sourcing ledger only; keep CR elsewhere. Do not cor whole system

3. Scoring engine

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Factor composite	AnalystService.java:142 — tech×0.30 + mom×0.20 + val×0.20 + sent×0.30	No quality factor. No growth factor. 30% on sentiment, which VR doesn't use for rating at all. Structurally misspecified	VR: Quality 25 / Growth 20 / Valuation 35 / Momentum 20	9 spec'd dimensions built on rank → z-score → average within bucket → equal-weight buckets
Weighting method	Fixed hand-chosen weights	Arbitrary, undocumented provenance	AQR, MSCI, S&P all converge on equal-weight buckets, explicitly rejecting optimisation: <i>"a potential nightmare of choices and dimensionality... We choose the latter"</i>	Fixed equal-weight buckets. Do NOT use PCA, Gram-Schmidt or covariance-optimised weights — measured evidence: HML–RMW and RMW–CMA flip sign between 1963–99 and 2000–26. The covariance matrix you estimate is not the one you live with
Multicollinearity	RSI/CCI/Stochastic/ROC/MACD-line pairwise p>0.9 , naively averaged → momentum ~4:1 over-weighted	Silent mis-weighting; the composite doesn't measure what it claims	Measured: HML–CMA 0.68 is what makes HML redundant in FF5. RMW/CMA/UMD are the near-orthogonal trio (0.02, 0.05, –0.02)	Collapse each family to one representative, then blend across near-orthogonal axes. Measured payoff: HML+CMA (p=0.68) → Sharpe 0.41, zero gain; HML+UMD (p=–0.19) → 0.67, nearly double

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Double-counted accruals	N/A (not yet built)	A trap to avoid: accruals appear independently in Piotroski (CFO>ROA), Beneish (TATA) and QMJ (ACC)	—	Assign accruals exactly one slot. Same class of error as the momentum over-weighting
Quality factor	Absent	The single biggest scoring gap	MSCI: exactly 3 descriptors — ROE, D/E, 5y earnings variability; winsorise 5/95 → z-score → negate D/E and variability → average; exclude the security entirely if ROE is missing. AQR QMJ is the 4-bucket deep version	Start with MSCI's three (cheapest defensible quality factor in existence), plus Novy-Marx GP/A = (REVT-COGS)/AT — <i>"the cleanest accounting measure of true economic profitability"</i>
Sector handling	One formula for all stocks	Wrong for banks/NBFCs	VR and Simply Wall St independently maintain a separate ratio set for financials: ROA, NIM, provision coverage, CAR, SGR gap; PE → PE-to-SGR	<code>SectorScoringProfile</code> strategy, selected by sector. Two independent sources converging makes this non-optional
Rating eligibility	Scores everything	Produces a confident number on unratable stocks	VR's 5 published exclusions	<code>RatingEligibility</code> gate returning <code>INELIGIBLE(reason)</code> — a first-class outcome, not a null

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Red flags	Not modelled	—	Tickertape scores them as a separate channel ; Zerodha Nudge fires pre-trade. Both treat them as disqualifiers, not score contributions	RedFlag channel with veto semantics. A pledged, ASM-listed stock should not be rescued by a good valuation score
Valuation bands	Not modelled	—	Morningstar: 5-star discount 20/30/40/50/75% scaling with uncertainty, seeded from TTM daily return SD	Borrowable free — it needs only price volatility. Drive our uncertainty tier off measurable data quality (bars available, parser confidence, history length)
Percentile normalisation	N/A	Look-ahead risk	Asness et al.: naive backtests embed look-ahead because <i>"you didn't know the 10y mean in real time"</i>	Expanding window, never full-sample. Any percentile must use only data available at that date
Regime overlay	None	—	Faber 10-month SMA: compound 9.32%→10.18%, max DD 83.7%→42.2%. Daniel-Moskowitz panic state: WML beta falls 0.518 (t=-28.4), Sharpe 0.682→1.194 OOS	200DMA/10-month market filter only. Regime-modulating a <i>stock-level</i> signal is supported only for momentum — Cederburg et al. (72 of 103 strategies lose) is direct evidence against generalising
Rejected inputs	Sentiment at 30%	—	Fed model/ BEER: E/P alone adj. R ² 29.6% vs 1.4% for E/P-Y	Drop sentiment from the rating. Never use BEER as a score input. Drop forward P/E and PEG — the data isn't obtainable free

4. Card engine

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Reward model	<code>CardRewardRule</code> + <code>SpendAggregator</code> ; rate × spend	Stateless. Cannot express "caps exhausted but ₹40k short of a fee waiver"	The correct answer is <code>argmax</code> of marginal value = capped rate + $\Delta(\text{milestone})$ + $\Delta(\text{fee waiver})$ – surcharge	<code>MarginalValueEngine</code> over live counters. Build the counters first; the rate table is the easy part
Counters	<code>monthlyCapAmount</code> field, no counter entity	Four state dimensions on four clocks; SBI uses statement cycle, HDFC uses calendar month	—	<code>UtilizationCounter(userCard, capOrMilestone, windowStart, windowEnd, consumed, source: DERIVED\ USER_CONFIRMED)</code>
Exclusions	Zero exclusion files	The most common modelling error is one exclusion list per card. Kotak publishes five lists for five purposes , with per-variant carve-outs, and says it "is not exhaustive"	—	<code>EligibleSpendSet</code> keyed by purpose (REWARD / MILESTONE / FEE_WAIVER / SPECIFIC_BENEFIT) with card-variant overrides
Rule versioning	None	Devaluations arrive with 30–60 days' notice. Kotak's own PDF carries an "effective 1 Apr 2026" clause. Backtesting requires "what was true last March?"	FinArt stamps a per-card "verified on" date	Effective dating on every rule, plus <code>supersedes_id</code>. Non-negotiable

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Provenance	None	Where issuers don't publish MCC maps, we are inferring — FinArt chose category-level modelling for exactly this reason	—	<code>source_url,</code> <code>source_type(MITC\ TNC_PDF\ </code> <code>PRODUCT_PAGE\ COMMUNITY),</code> <code>verified_on,</code> <code>confidence(HIGH\ MED\ LOW)</code> on every rule/cap/milestone row. The SBI Cashback cap disagreement across three sources is the whole argument for this
UPI/RuPay route	Not modelled	Recommending a Visa card for a UPI rail is a correctness bug	Only RuPay on UPI; P2P/ C2C/P2PM/ ATM/IPO/MF blocked; acquirer blocks 6010–6013, 6011, 6051, 6211, 4829, 7322, 74xx, 78xx/ 7995/9406	<code>RouteEligibility(card, route,</code> <code>allowed, blockedMccs[],</code> <code>blockedTxnTypes[],</code> <code>perTxnLimit, dailyLimit,</code> <code>interchangeFloor,</code> <code>smallMerchantExempt)</code>
Reward uncertainty	Point estimate	Whether a ₹5,000 UPI payment earns anything can depend on the merchant's turnover band — unobservable to the user and to us	NPCI parity mandate exempts issuers earning no interchange	Model as a confidence band, not a number
Point valuation	Single rate	Axis runs two currencies with a ~5x gap (EDGE Points ₹0.20 vs EDGE Miles ~₹1+)	Best-case transfer value as headline is <i>"the single biggest source of inflated claims"</i>	<code>value_per_point</code> vector by redemption route with <code>{rate,</code> <code>liquidity, minRedemption,</code> <code>expiry, transferRatio}</code> . Headline = user's declared behaviour; show conservative / realistic / ceiling

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Category inference	Merchant-name inferred, honestly <code>inferred=true</code>	Lower precision than MCC	FinArt deliberately chose category-level "because issuers rarely publish the exact MCC map"	Keep the honesty flag. MCC where obtainable, category otherwise — and say which was used

5. Data acquisition

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
AMFI NAV	Fixed — binds by header name, 16MB codec buffer	Depends on following a 302 from <code>www.amfiindia.com</code>	Feed moved to <code>portal.amfiindia.com/spages/NAVAL1.txt</code> ; header now carries new Plan / Option columns that break positional parsers	Point at <code>portal.</code> directly. Our header-name binding already survives the column change — this is the fix validating itself
Price history	Existing market-data service	—	Dhan: free with account, daily back to inception. Kite: ₹500/mo for the same. Upstox: daily only 1 year — weak	Dhan primary, Kite backup. Independent check against <code>nsearchives</code> bhavcopy
NSE access	—	<code>www.nseindia.com</code> is Akamai-fenced and blocks datacenter IPs outright; cookie-priming does not help	<code>nsearchives.nseindia.com</code> is open — bhavcopies, equity master, index constituents, corp-action archives	Archives from the server; JSON APIs only from a residential IP. Use <code>jugaad-data</code> (maintained) not <code>nsepy</code> (dead since 2020)

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Fundamentals	None	No free structured source; building an XBRL ingestor across ~5,000 listed cos is a multi-month project	Screener.in HTML: 12 years annual + quarterly + ratios + shareholding, unauthenticated 200, and <code>/company/<SYM>/</code> is not disallowed by robots.txt	Polite scrape (1 req/2–3s), cache aggressively, re-scrape only post-results. Find the live ToS before any redistribution — <code>/terms/</code> 404'd. Personal use plausibly fine; republishing is not
Filings	None	—	<code>api.bseindia.com</code> returns 200 JSON, no datacenter-IP block , needs only a UA + Referer	BSE for announcements and scrip master; NSE archives for corp actions
MF holdings	None	No central machine-readable source. Per-AMC XLSX, inconsistent headers, WAFs, JS-gated pages	SEBI mandates monthly disclosure within 10 days (debt: fortnightly within 5)	First-class subsystem, not a feature. 3–6 weeks + 1–3 days/month. ISIN as join key. Top-40 truncation (Zerodha's approach) is a legitimate v1 if disclosed
Analyst estimates	None	—	No free source exists. Refinitiv/ Bloomberg/CapIQ = ₹lakhs to \$20–30k/yr	Scope out. Trailing/ TTM + historical 3/5/10y CAGR. A PEG on <i>historical</i> growth is defensible if labelled honestly
User MF holdings	CAS + Gmail	—	AMFI directed MF Central to stop sharing investor data with third-party apps (Sept 2025); AA gates on regulatory status	CAS-first is now the only viable architecture — and it is what we already have. No change needed; the moat just got deeper

6. Cross-cutting

SYSTEM	CURRENT	PROBLEM	INDUSTRY PATTERN	PROPOSED ARCHITECTURE
Data sufficiency	Ad-hoc (NOT_SIZED)	Each subsystem invents its own "can't answer"	VR's 5 exclusions; Zerodha suppressing XIRR; Simply Wall St's auto-fail on <10y dividend history	First-class <code>Insufficient(reason, whatWouldFixIt)</code> result type used by every computed number
Explainability	<code>Factor</code> list with notes	Weighted z-scores are hard to explain	Simply Wall St: 6 binary pass/fail checks per axis. "Failed 4 of 6 health checks, here they are" is a reason	Binary checks as the explanation layer over the continuous score. Directly serves "EVERY RECOMMENDATION MUST HAVE A REASON"
PII in LLM calls	Sanitisation required by standing constraint	—	Jupiter/CRED/INDmoney all scope-limit; CheQ holds CASA certification	Keep the sanitiser on the path to any fallback provider; <code>AiAuditService</code> redaction already covers secrets
Rate limiting / politeness	—	Scraping Screener and 40+ AMC sites	—	Central <code>PoliteFetcher</code> with per-host rate limits, caching and a real UA. A 403 from an AMC is a scheduling problem, not an error to retry hard